

Chapter 11:

Geometry-Based Flow Visualization

Prof. Dr.-Ing. Thomas Schultz

URL: <http://cg.cs.uni-bonn.de/schultz/>

E-Mail: schultz@cs.uni-bonn.de

Office: Friedrich-Hirzebruch-Allee 6, Room 2.117

June 24/30, 2026

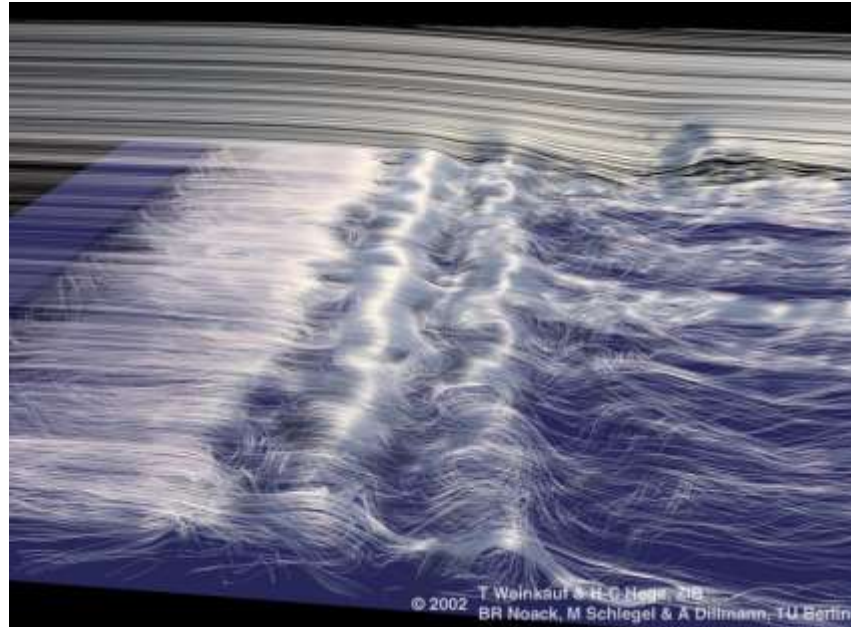
Section 11.1:

Introduction to Vector Field Visualization

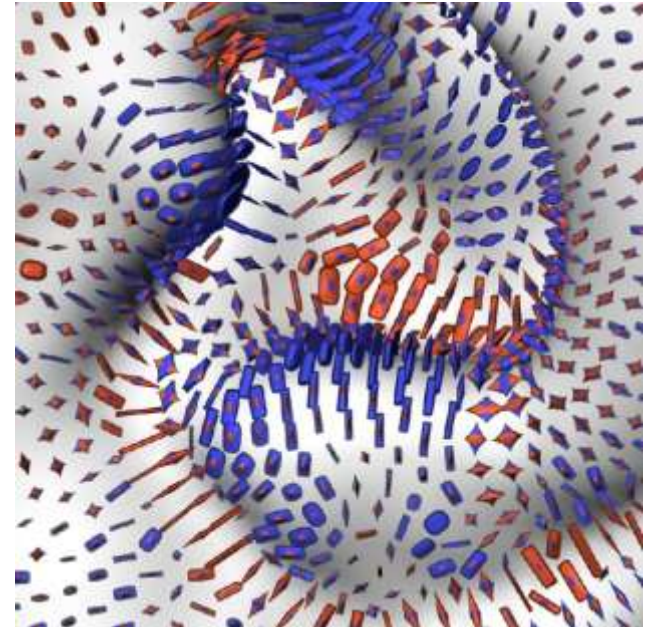
Where We Are In Scientific Visualization



Scalar Fields



Vector Fields



Tensor Fields

$$f: D \rightarrow R^m \quad \text{with} \quad D \subseteq R^n$$

Real-World Flow Visualization



<https://ecomodder.com/forum/showthread.php/tufts-38870.html>

wool tufts



http://autospeed.com/cms/A_108677/article.html



smoke injection



http://autospeed.com/cms/A_108677/article.html

smoke nozzles

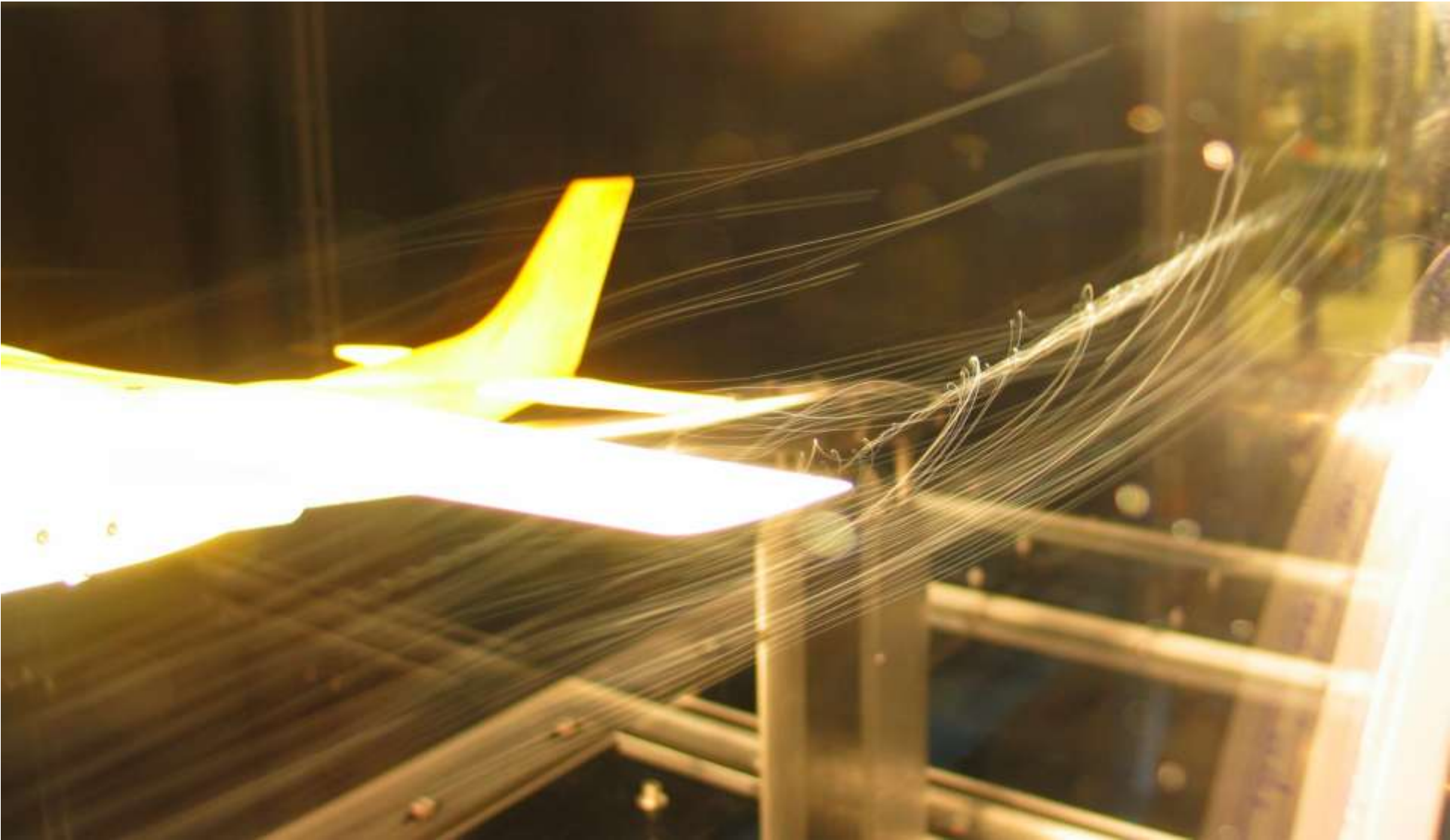


[NASA, J. Exp. Biol.]



http://autospeed.com/cms/A_108677/article.html

smoke nozzles



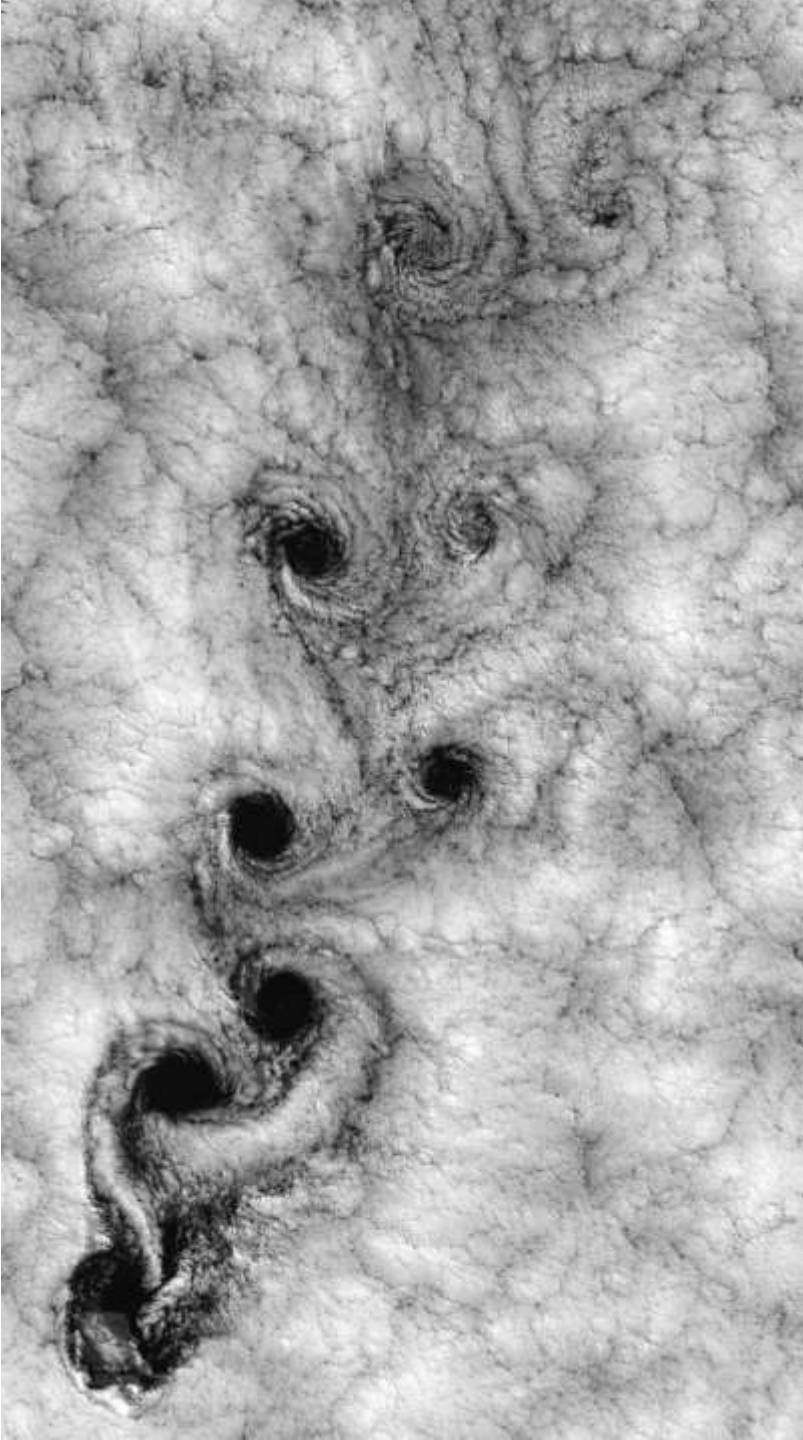
A wind tunnel model of a Cessna 182 showing a wingtip vortex.
Tested in the RPI (Rensselaer Polytechnic Institute) Subsonic Wind Tunnel.
By Ben FrantzDale (2007).



Smoke angel

A C-17 Globemaster III from the 14th Airlift Squadron, Charleston Air Force Base, S.C. flies off after releasing flares over the Atlantic Ocean near Charleston, S.C., during a training mission on Tuesday, May 16, 2006. The "smoke angel" is caused by the vortex from the engines.

(U.S. Air Force photo/Tech. Sgt. Russell E. Cooley IV)

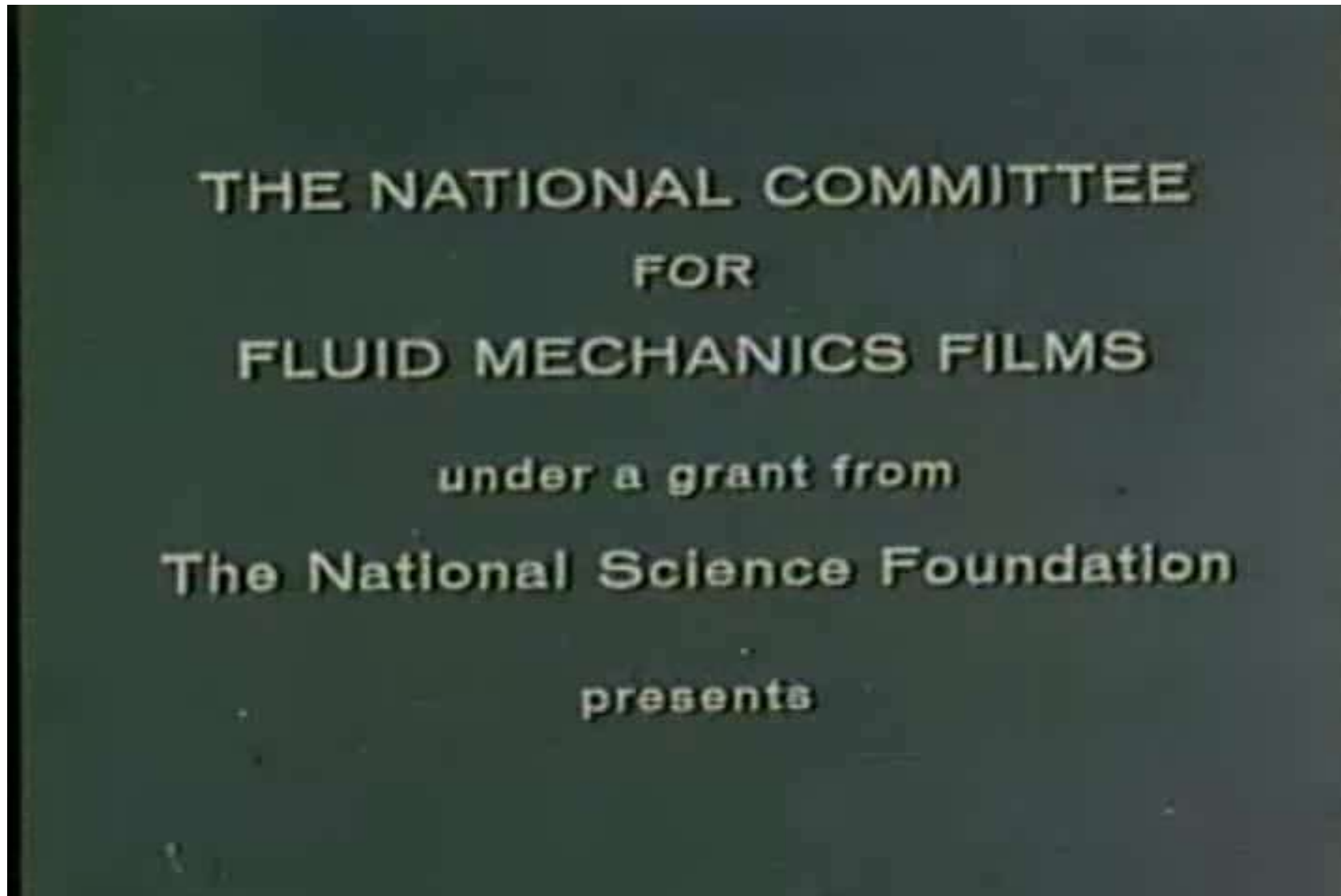


Clouds (satellite image)

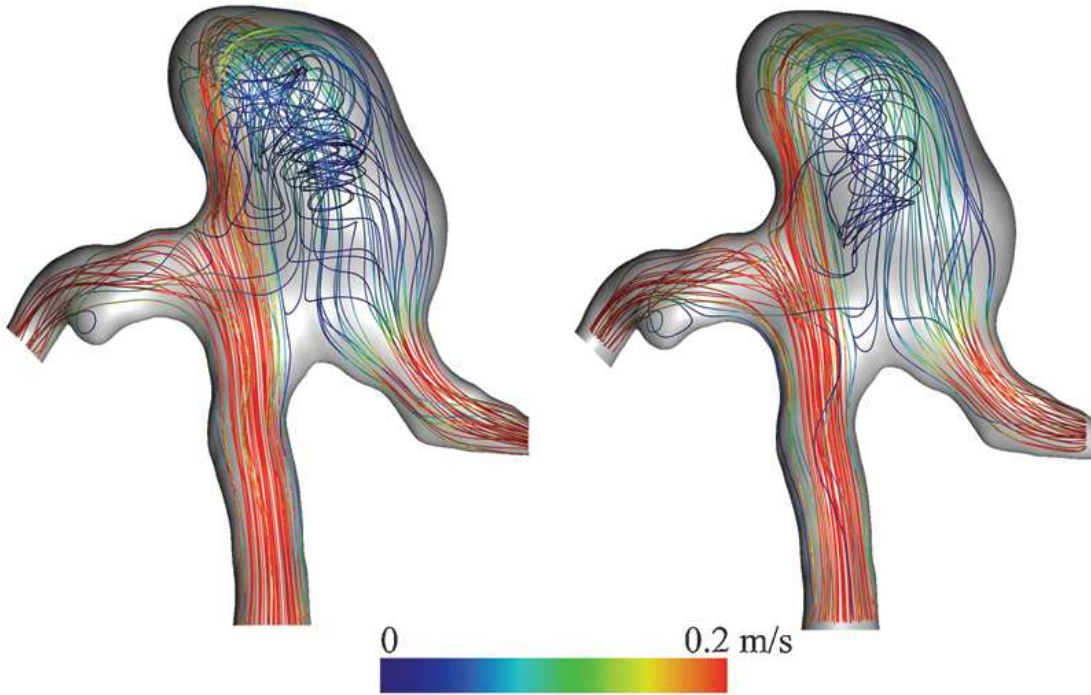
Juan Fernandez Islands

<http://de.wikipedia.org/wiki/Bild:Vortex-street-1.jpg>

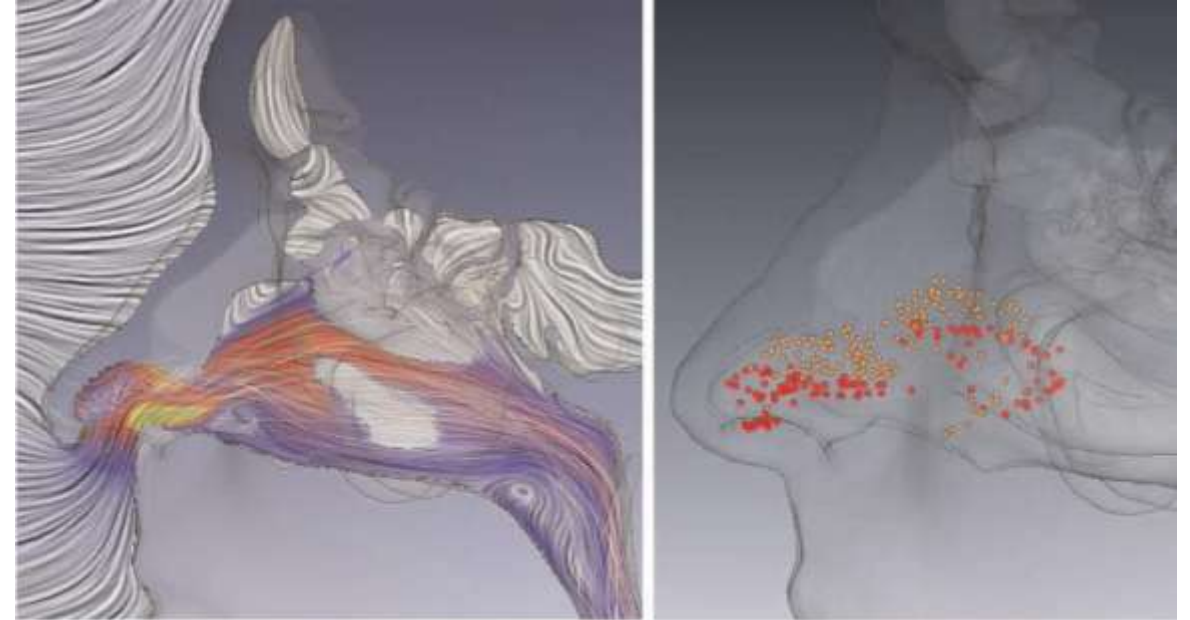
Flow Visualization in a Laboratory



CFD Visualizations in Medicine



Simulated Blood Flow in an Aneurysm
[Goubergrits et al. 2013]



Simulated Nasal Air Flow
[Zachow et al. 2009]

Strategies for Vector Field Visualization

- **Geometry-based Techniques (this chapter)**
 - Define geometry (e.g., curves or surfaces) that is directly related to the flow and can be rendered with traditional techniques
 - Might resemble real-world visualization (e.g., smoke injection)
- **Image-based Techniques (next chapter)**
 - Create a dense texture that aligns with flow structures
- **Feature-based Techniques (if time permits)**
 - Extract physically meaningful patterns (features) that are of particular interest for some analysis task and visualize those

Steady vs. Unsteady Vector Fields

vector field

parameter-
independent

$$\mathbf{v} : \mathbb{E}^n \rightarrow \mathbb{R}^m$$
$$\mathbf{v}(x, y) = \begin{pmatrix} u(x, y) \\ v(x, y) \end{pmatrix}$$

steady vector field

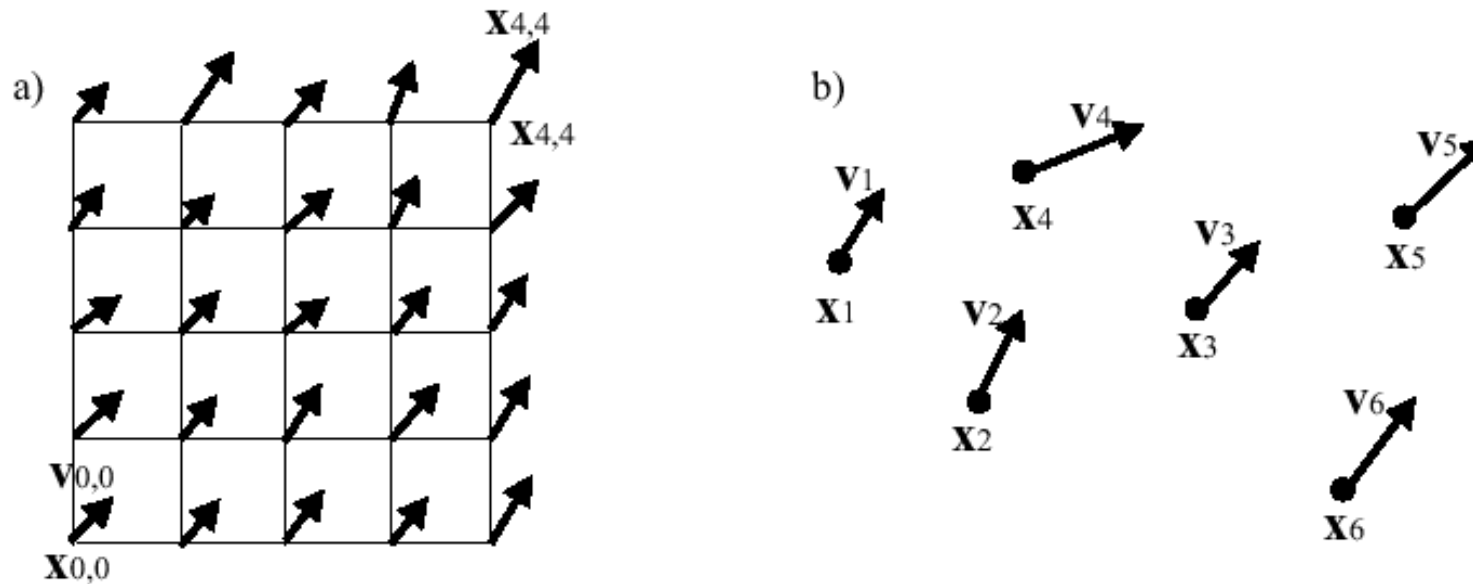
one-parameter-
dependent

$$\mathbf{v} : \mathbb{E}^{n+1} \rightarrow \mathbb{R}^m$$
$$\mathbf{v}(x, y, t) = \begin{pmatrix} u(x, y, t) \\ v(x, y, t) \end{pmatrix}$$

unsteady vector field

Representation of Flow Data

- **Flow data:**
 - vector data on a 2D or 3D grid
 - in addition, scalar data may be defined per grid point
 - pressure, density, etc.



flow data; a) on regular grid; b) scattered flow data

Origins of Flow Data

- Obtaining flow data:

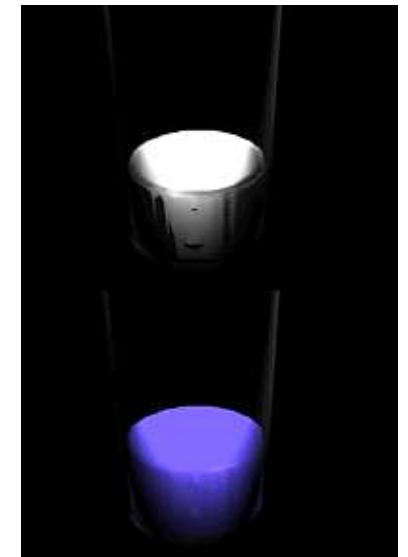
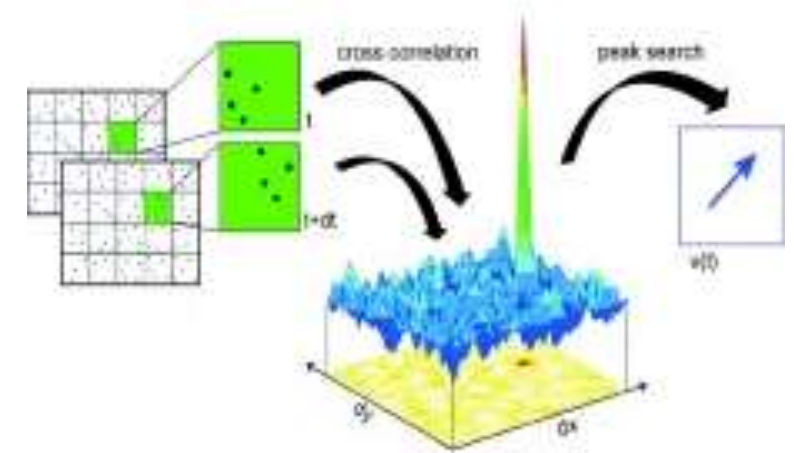
- by measurement:

- sensors at certain sample points of a fluid/gas in motion give flow velocity at these points



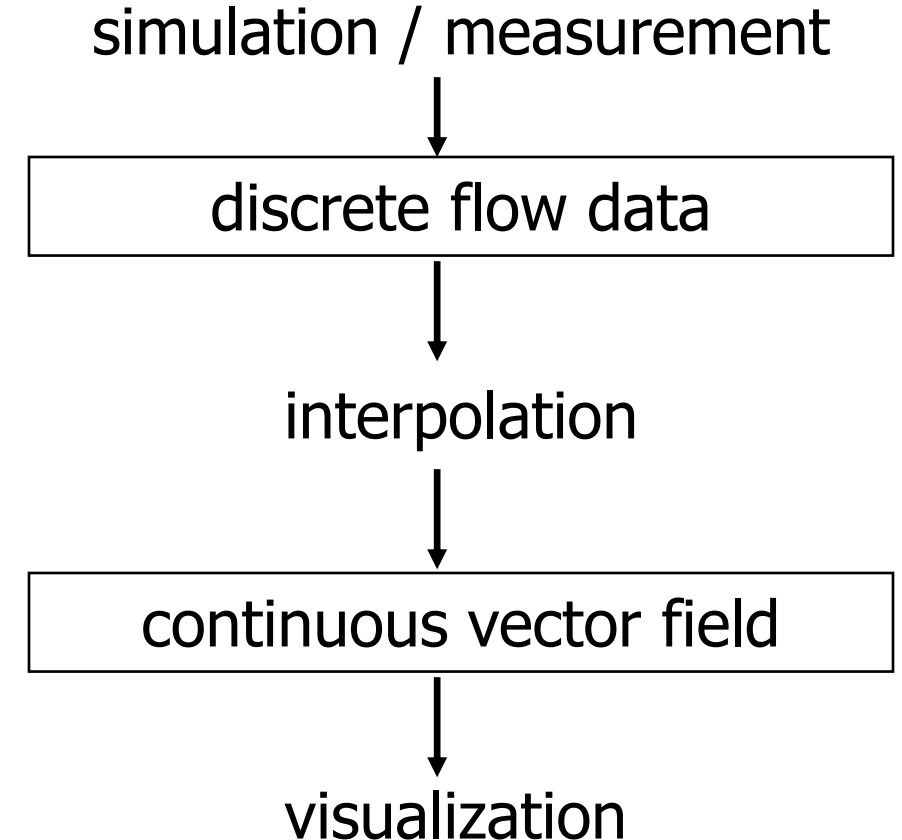
- optical methods: Particle Image Velocimetry (PIV)

- by simulation: solving the Navier-Stokes equations



Vector Field Interpolation

- **Discrete flow data** is converted to a **continuous vector field** by **component-wise interpolation** of the vectors inside the cells.
 - e.g., bi-/trilinear interpolation in grids, finite differences for differentiation
 - Radial Basis Functions for Scattered Data



Vector Field Jacobian

scalar field

$$s : \mathbb{E}^n \rightarrow \mathbb{R}$$

$$s(\mathbf{x})$$

with $\mathbf{x} \in \mathbb{E}^n$

vector field

$$\mathbf{v} : \mathbb{E}^n \rightarrow \mathbb{R}^m$$

$$\mathbf{v}(\mathbf{x}) = \begin{pmatrix} c_1(\mathbf{x}) \\ \vdots \\ c_m(\mathbf{x}) \end{pmatrix}$$

with $\mathbf{x} \in \mathbb{E}^n$

$$\mathbf{v}(x, y) = \begin{pmatrix} u(x, y) \\ v(x, y) \end{pmatrix}$$

2D vector field

tensor field

$$\mathbf{T} : \mathbb{E}^n \rightarrow \mathbb{R}^{m \times b}$$

$$\mathbf{T}(\mathbf{x}) = \begin{pmatrix} c_{11}(\mathbf{x}) & \dots & c_{1b}(\mathbf{x}) \\ \vdots & & \vdots \\ c_{m1}(\mathbf{x}) & \dots & c_{mb}(\mathbf{x}) \end{pmatrix}$$

with $\mathbf{x} \in \mathbb{E}^n$

$$\mathbf{J}(x, y) = \begin{pmatrix} u_x & u_y \\ v_x & v_y \end{pmatrix}$$

Jacobian of 2D vector field

Vector Field Jacobian

vector field

$$\mathbf{v} : \mathbb{E}^n \rightarrow \mathbb{R}^m$$

tensor field

$$\mathbf{T} : \mathbb{E}^n \rightarrow \mathbb{R}^{m \times b}$$

If vector field is continuously differentiable, we can compute its partial derivatives and arrange them in a 2x2 matrix:


$$\mathbf{v}(x, y) = \begin{pmatrix} u(x, y) \\ v(x, y) \end{pmatrix}$$

2D vector field

$$\mathbf{J}(x, y) = \begin{pmatrix} u_x & u_y \\ v_x & v_y \end{pmatrix}$$

Jacobian of 2D vector field

Divergence

Divergence of $\mathbf{v}$

- is a scalar field

$$\operatorname{div} \mathbf{v} = \operatorname{tr}(\mathbf{J}) = \frac{\delta u}{\delta x} + \frac{\delta v}{\delta y} + \frac{\delta w}{\delta z} = u_x + v_y + w_z$$

- *Interpretation:* observe transport of a small ball around a point
 - expanding volume $\rightarrow$ positive divergence
 - contracting volume $\rightarrow$ negative divergence
 - constant volume $\rightarrow$ zero divergence

$$\operatorname{div} \mathbf{v} \equiv 0 \Leftrightarrow \mathbf{v} \text{ is } \textit{incompressible}$$

Interpretation of Curl

- **Curl of $\mathbf{v}$** indicates how the field swirls at a point
- paddle wheel model
 - insert paddle wheel in a flow
 - orient such that its rate of rotation is maximal
 - ➔ **curl $\mathbf{v}$** is parallel to main rotation axis
 - ➔ **$|\text{curl } \mathbf{v}|$** is corresponds to rate of rotation
- golf ball model
 - consider golf ball in $\mathbf{v}$
 - is transported and rotates
 - ➔ **curl $\mathbf{v}$** is parallel to main rotation axis
 - ➔ **$|\text{curl } \mathbf{v}|$** is corresponds to rate of rotation

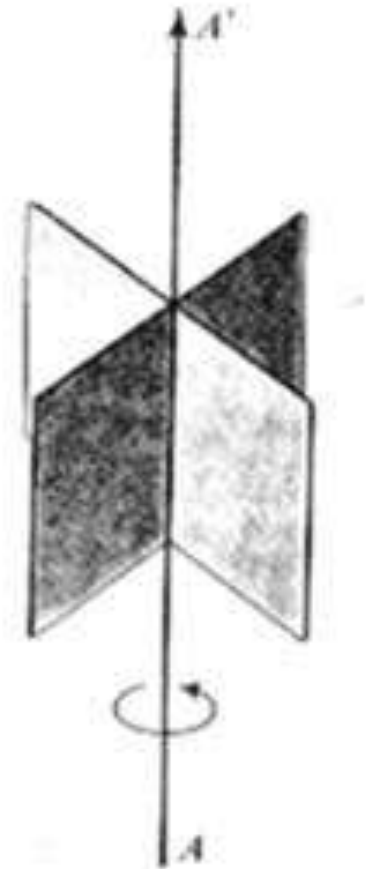


Figure 53

Curl

Curl of 3D vector field $\mathbf{v}$

- is another vector field
- computed from the Jacobian off-diagonals
- also called rotation (rot) or vorticity

$$\mathbf{curl} \, \mathbf{v} = \begin{pmatrix} w_y - v_z \\ u_z - w_x \\ v_x - u_y \end{pmatrix} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ \frac{\partial}{\partial x} & \frac{\partial}{\partial y} & \frac{\partial}{\partial z} \\ u & v & w \end{vmatrix}$$

Nabla Operator: ∇

The Nabla operator

- abbreviation: ∇
- symbolically written as:

$$\nabla = \mathbf{i} \frac{\delta}{\delta x} + \mathbf{j} \frac{\delta}{\delta y} + \mathbf{k} \frac{\delta}{\delta z} = \begin{pmatrix} \frac{\delta}{\delta x} \\ \frac{\delta}{\delta y} \\ \frac{\delta}{\delta z} \end{pmatrix}$$

- allows us to write the other operators as:

$$\mathbf{grad} f = \nabla f$$

$$\mathbf{div} \mathbf{v} = \nabla \cdot \mathbf{v}$$

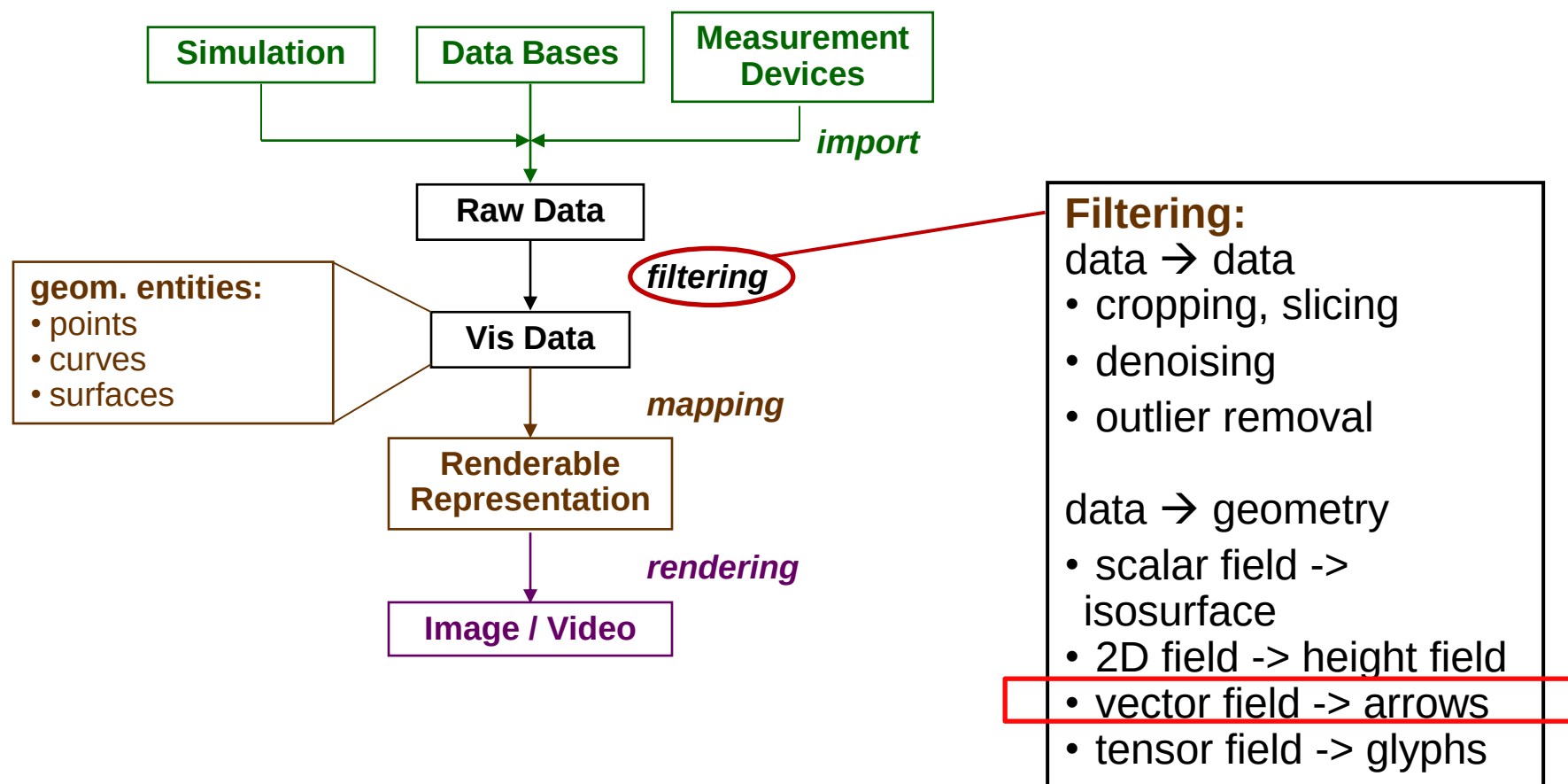
$$\mathbf{curl} \mathbf{v} = \nabla \times \mathbf{v}$$

Summary: Introduction to Vector Field Vis

- **Flow Visualization** is the main application of vector field visualization
 - Applications in engineering, flight safety, meteorology, biology, medicine etc.
 - Measurement / simulation of flow data
 - Reconstruction of continuous vector fields by component-wise interpolation
- **Classification** of flow vis techniques into
 - 2D/3D steady/unsteady
 - Geometry-/image-/feature-based
- Vector field Jacobian, divergence, curl

Section 11.2: Vector Field Glyphs

Where We Are in the Vis Pipeline



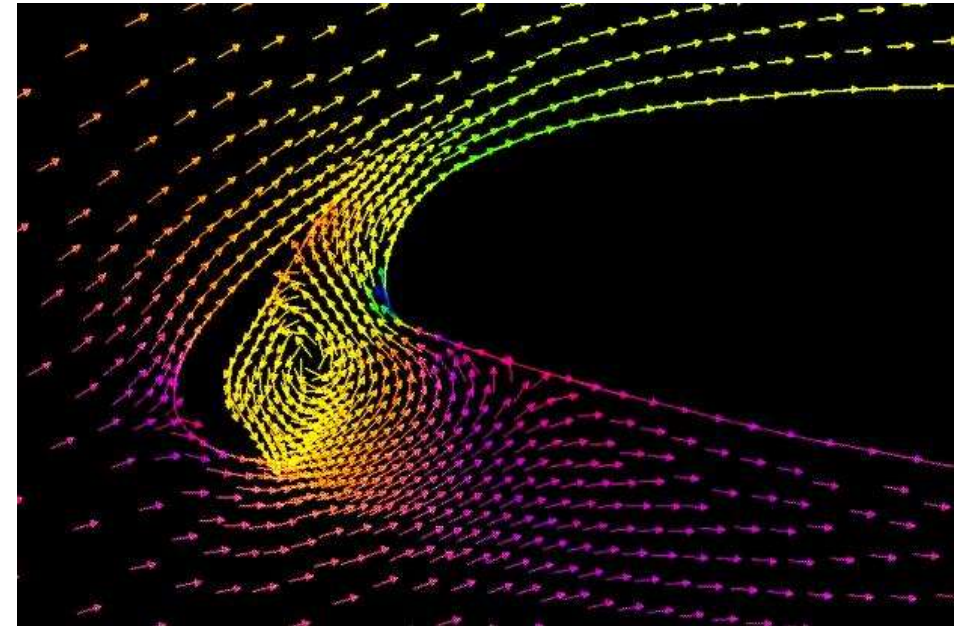
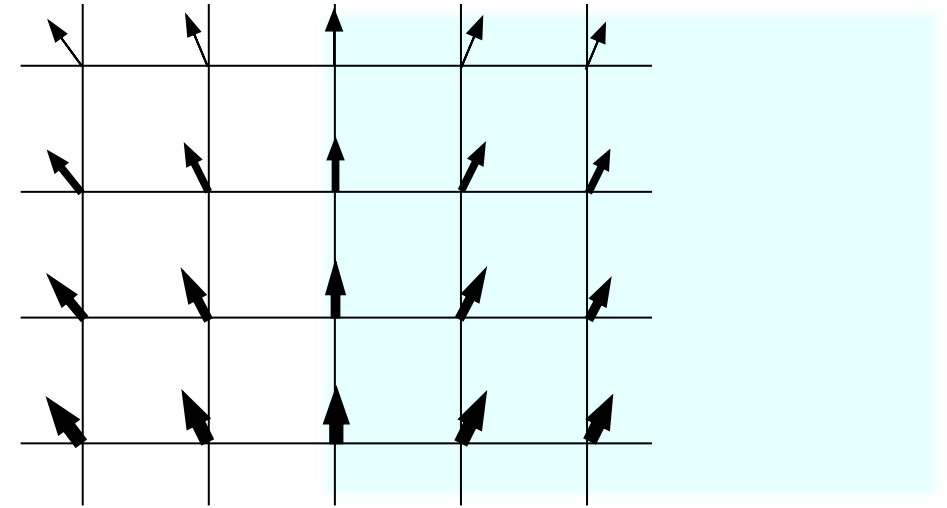
Counts as filtering in VTK; conceptually, is often counted as mapping

Arrows as Vector Field Icons

Arrows visualize

- Direction of vector field:
 - Orientation of arrows
- Magnitude:
 - Length of arrows
- Color coding of additional quantities

Sometimes called “hedgehog plot”



Layering Additional Information

- [Kirby et al 99]: visualize multiple quantities associated with 2D flow data by layering; inspired by artistic paintings

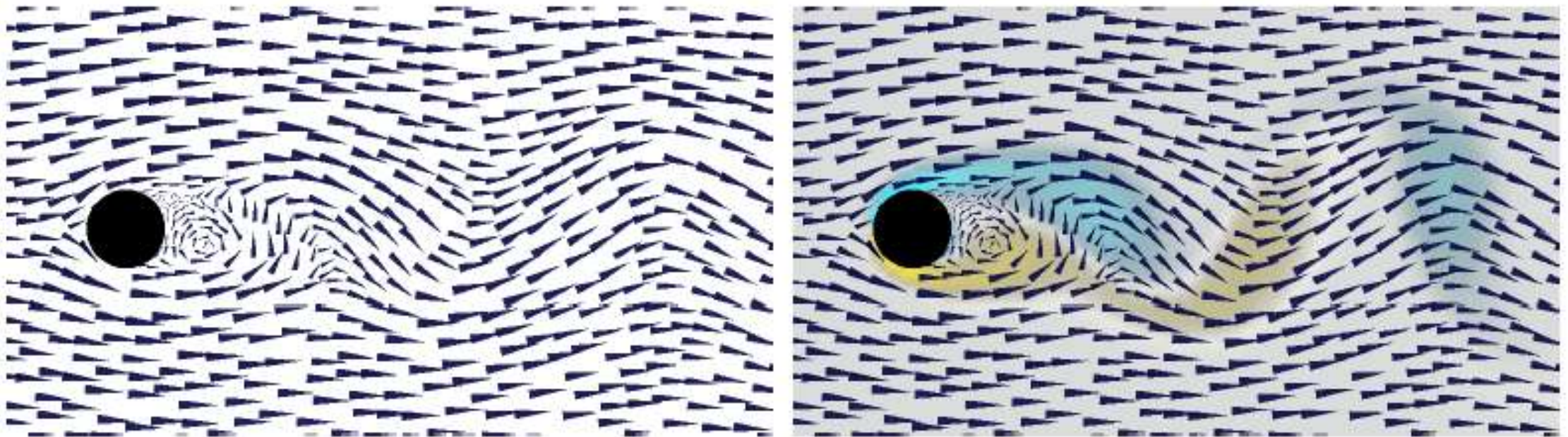
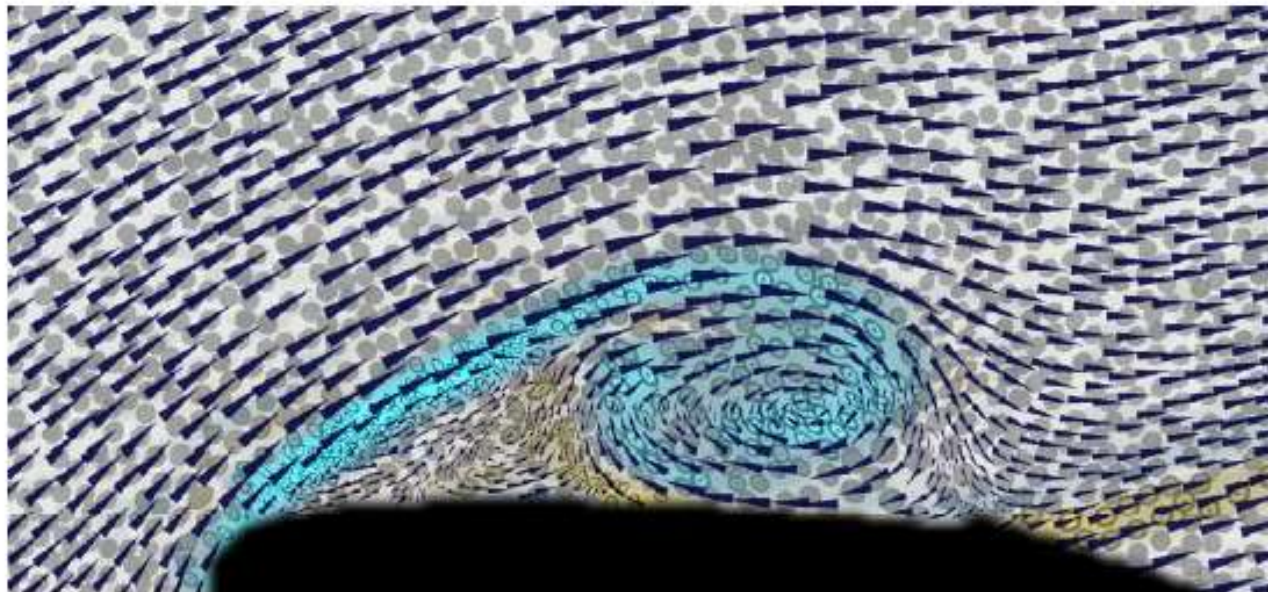
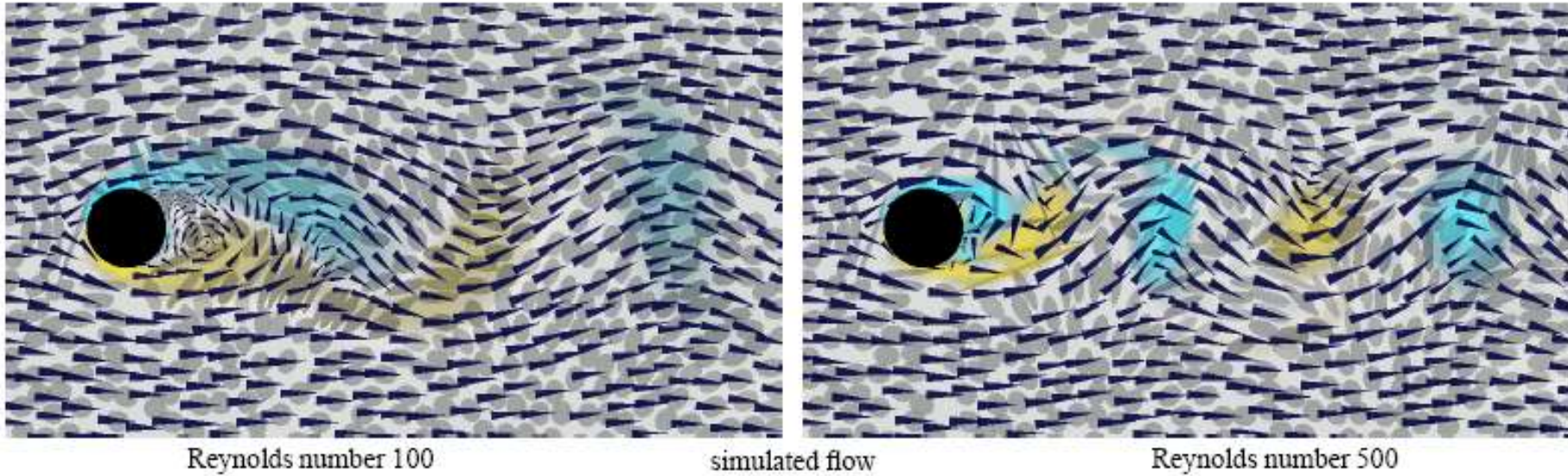


Figure 1: Typical visualization methods for 2D flow past a cylinder at Reynolds number 100. On the left, we show only the velocity field. On the right, we simultaneously show velocity and vorticity. Vorticity represents the rotational component of the flow. Clockwise vorticity is blue, counterclockwise yellow.

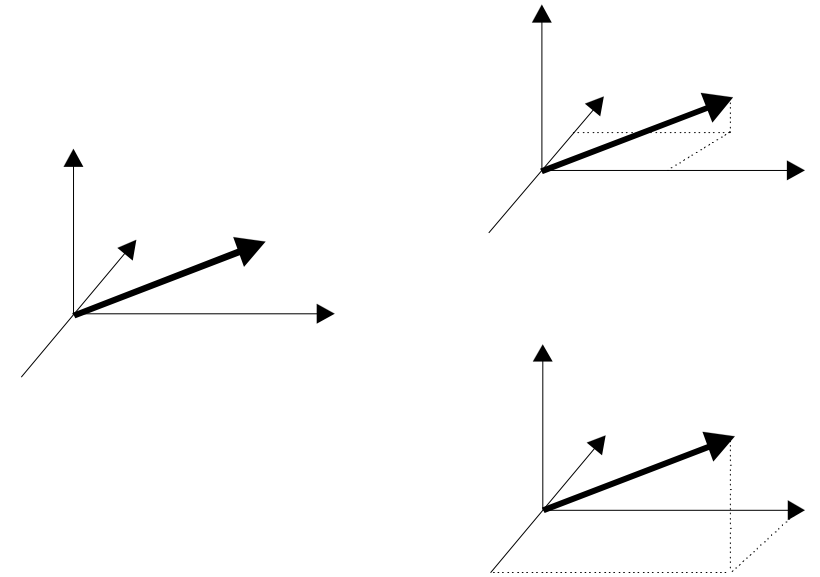
Example: Layering Additional Information



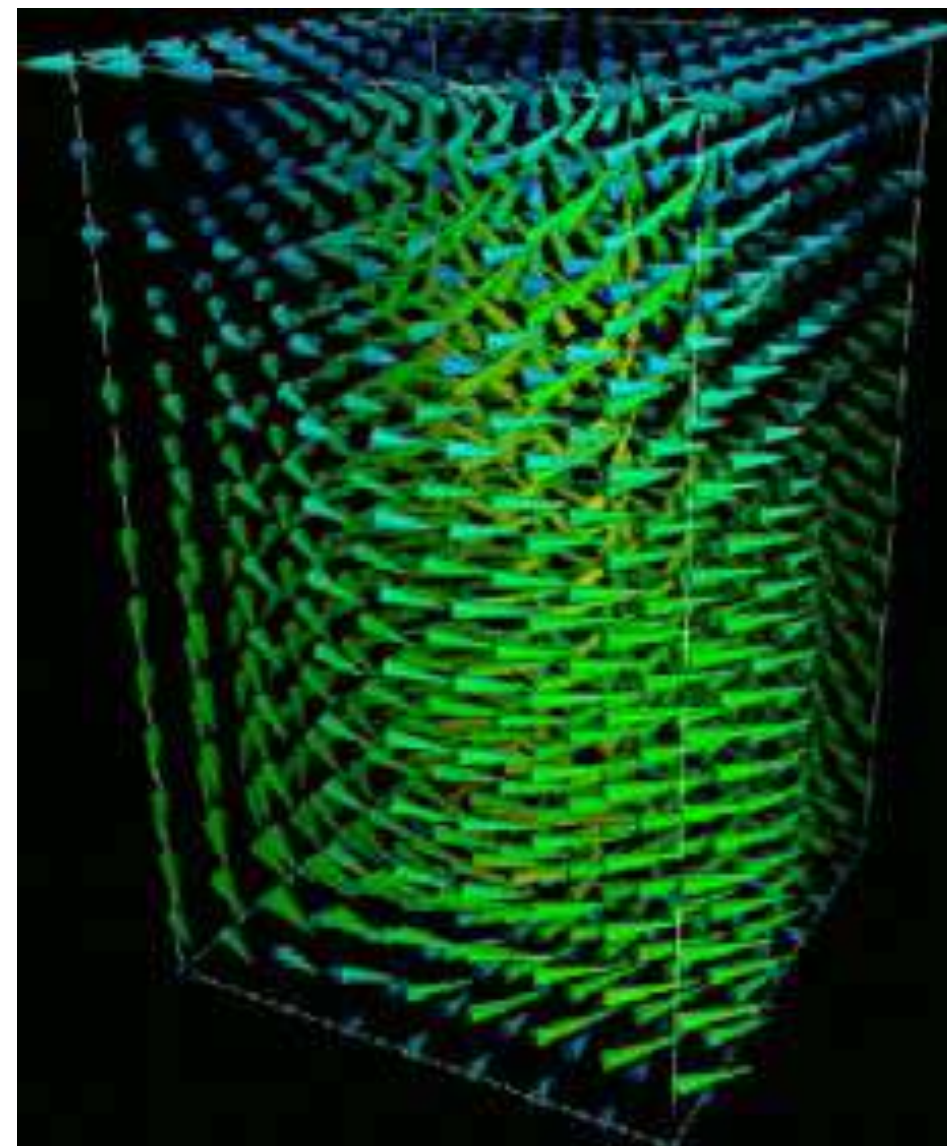
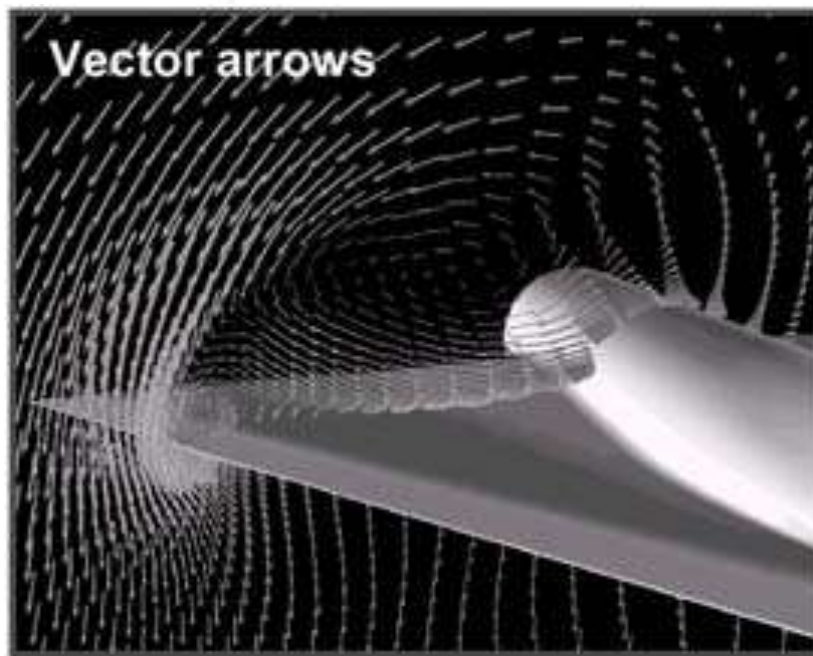
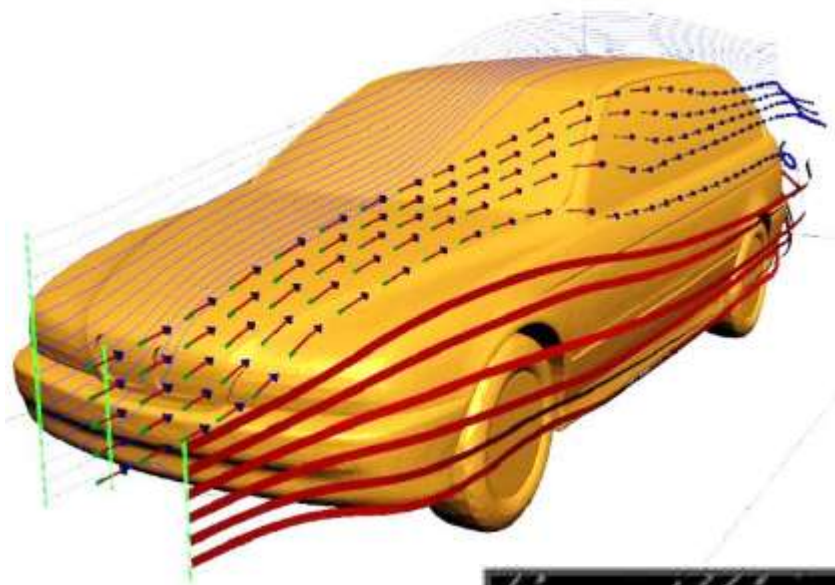
KEY	
data	visualization
velocity	arrow direction
speed	arrow area
vorticity	underpainting/ellipse color (blue=cw, yellow=ccw), and ellipse texture
	contrast
rate of strain	$\log(\text{ellipse radii})$
divergence	ellipse area
shear	ellipse eccentricity

Arrow Plots in 3D

- Occlusion problem → careful seeding
- Ambiguity problem
- Solutions:
 - 3D icons (cylinder + cone)
 - Color code direction
 - Highlight parts of arrow plot which points into a user defined direction

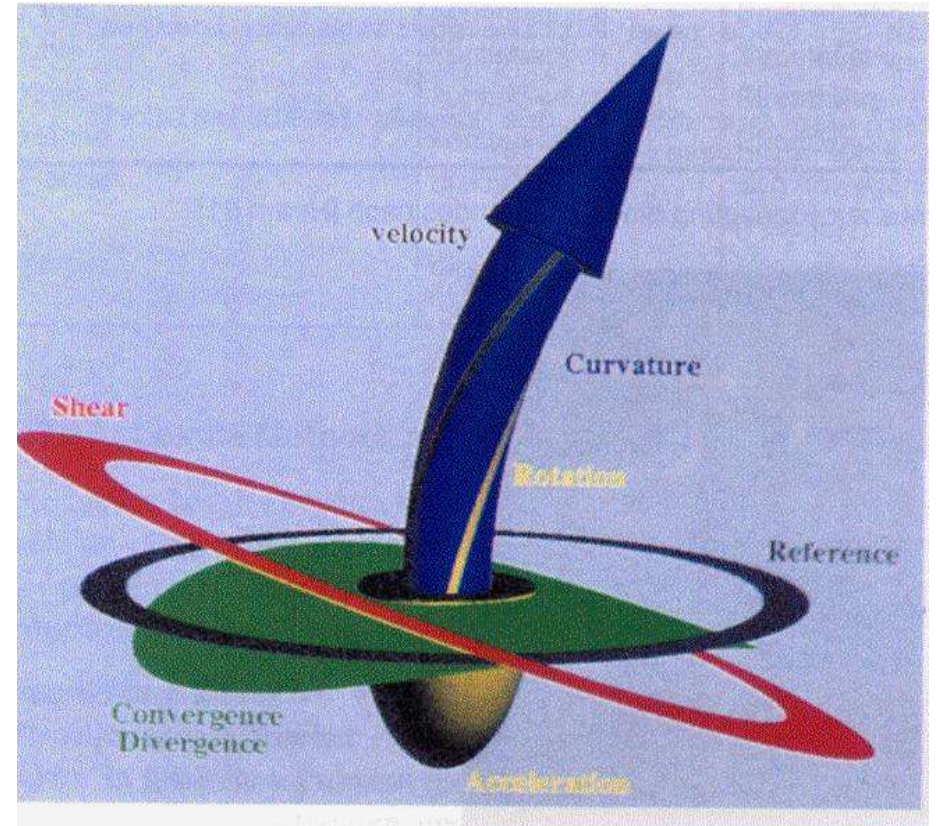


3D Arrow Plots: Examples



Advanced Glyphs: Flow Probe

- More sophisticated glyphs can visualize additional features of the vector field
 - E.g., including information derived from the Jacobian
 - Even more careful seeding (e.g., interactive)



Flow Probe by [de Leeuw, van Wijk 93]

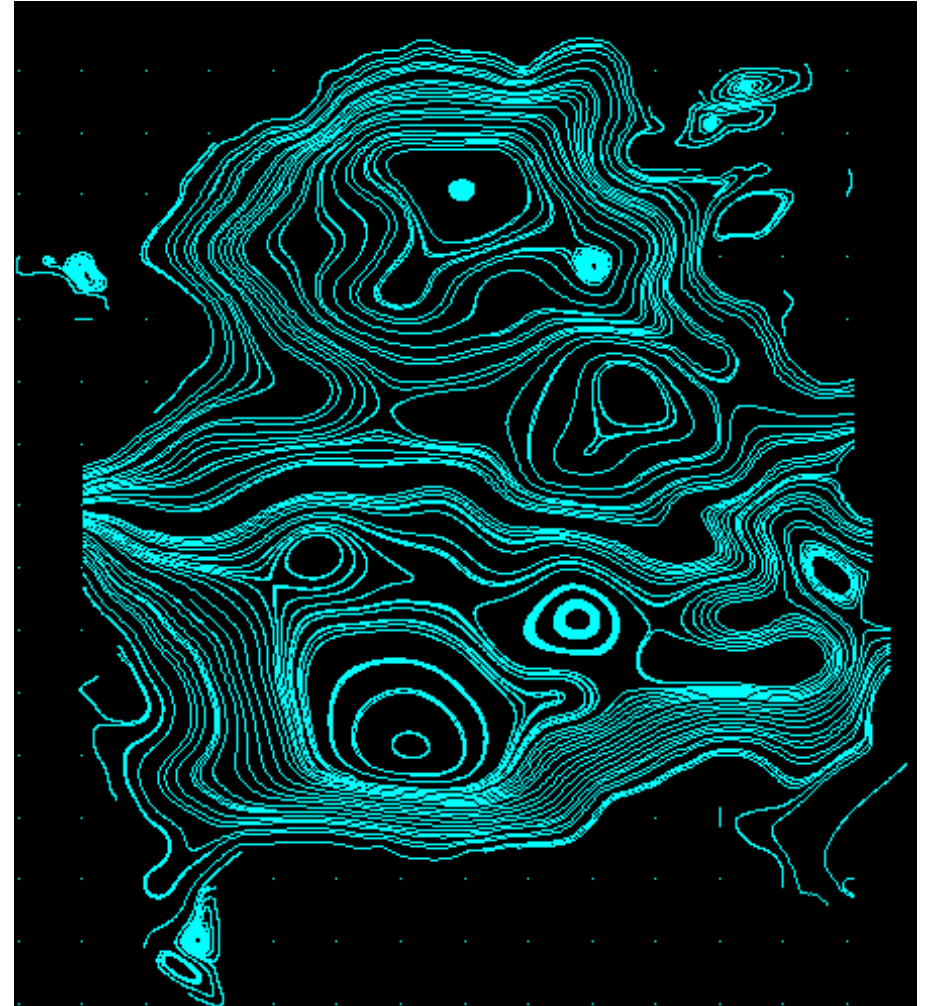
Glyphs: Advantages and Disadvantages

- + Simple
- + Can convey additional information
- + Can be used in 3D
- Seeding (placement) can be non-trivial
- Global structures may not be readily apparent
- Suffers from occlusion (especially in 3D)
- Poor results if magnitude of velocity varies greatly
(Remedy: Color code and/or compress magnitude)
- (- Heavy load in the graphics subsystem)

Section 11.3: Characteristic Curves

Motivation: Characteristic Curves

- **Characteristic Curves / Integral Curves:**
 - “natural” visualization technique
 - give immediate insight into the flow
- **We will now discuss:**
 - How to formally define these curves?
 - How to compute these curves?
 - Which lines to draw? (seeding)
 - Spatial impression in 3D?



Four Types of Characteristic Curves

- **Stream line:** curve tangential to the vector field in each point at a fixed time
- **Path line:** describes the motion of a particle over time in an unsteady flow field
- **Streak line:** location of all particles set out at a fixed point, continuously over time
- **Time line:** location of all particles set out along a certain line at a fixed time

Tangent Curves

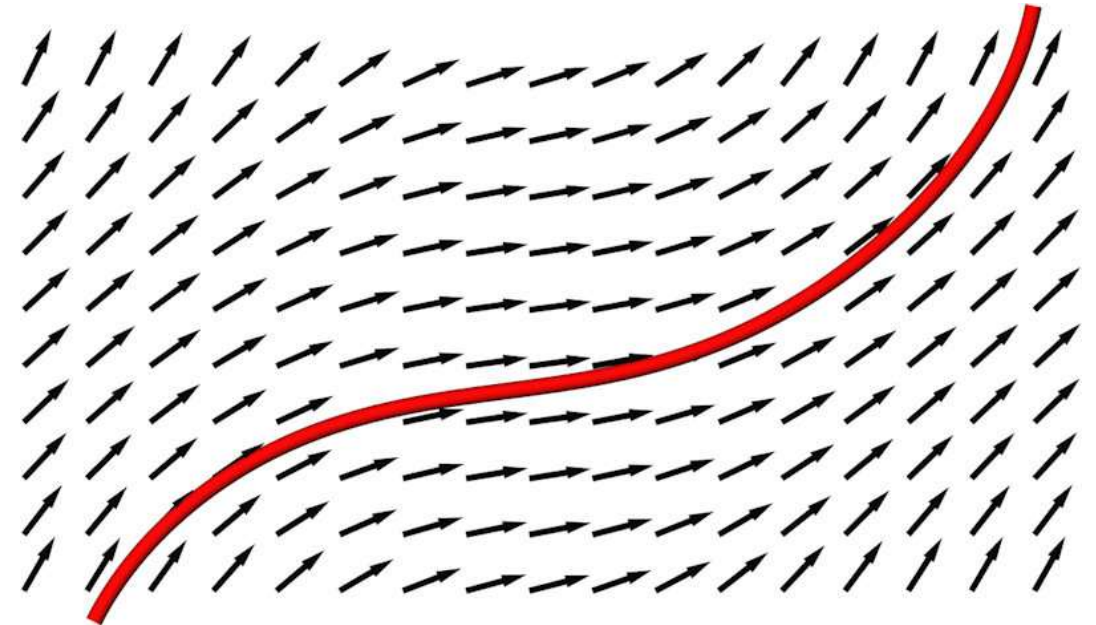
- A **tangent curve** $\mathbf{x}(\tau)$ of the vector field $\mathbf{v}$ is a curve in Euclidean space $\mathbf{E}^{2/3}$ with the property

$$\frac{d}{d\tau} \mathbf{x}(\tau) = \mathbf{v}(\mathbf{x}(\tau))$$

← denotes the tangent vectors of $\mathbf{x}(\tau)$

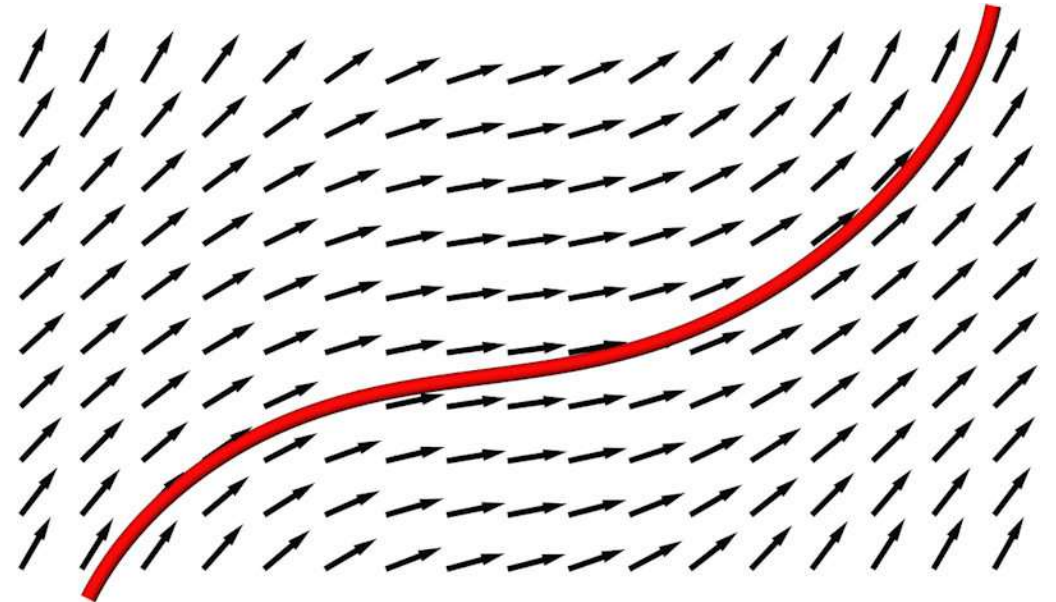
for any τ in the domain of $\mathbf{x}$

- *Interpretation:*
path of a massless particle in a flow described by $\mathbf{v}$

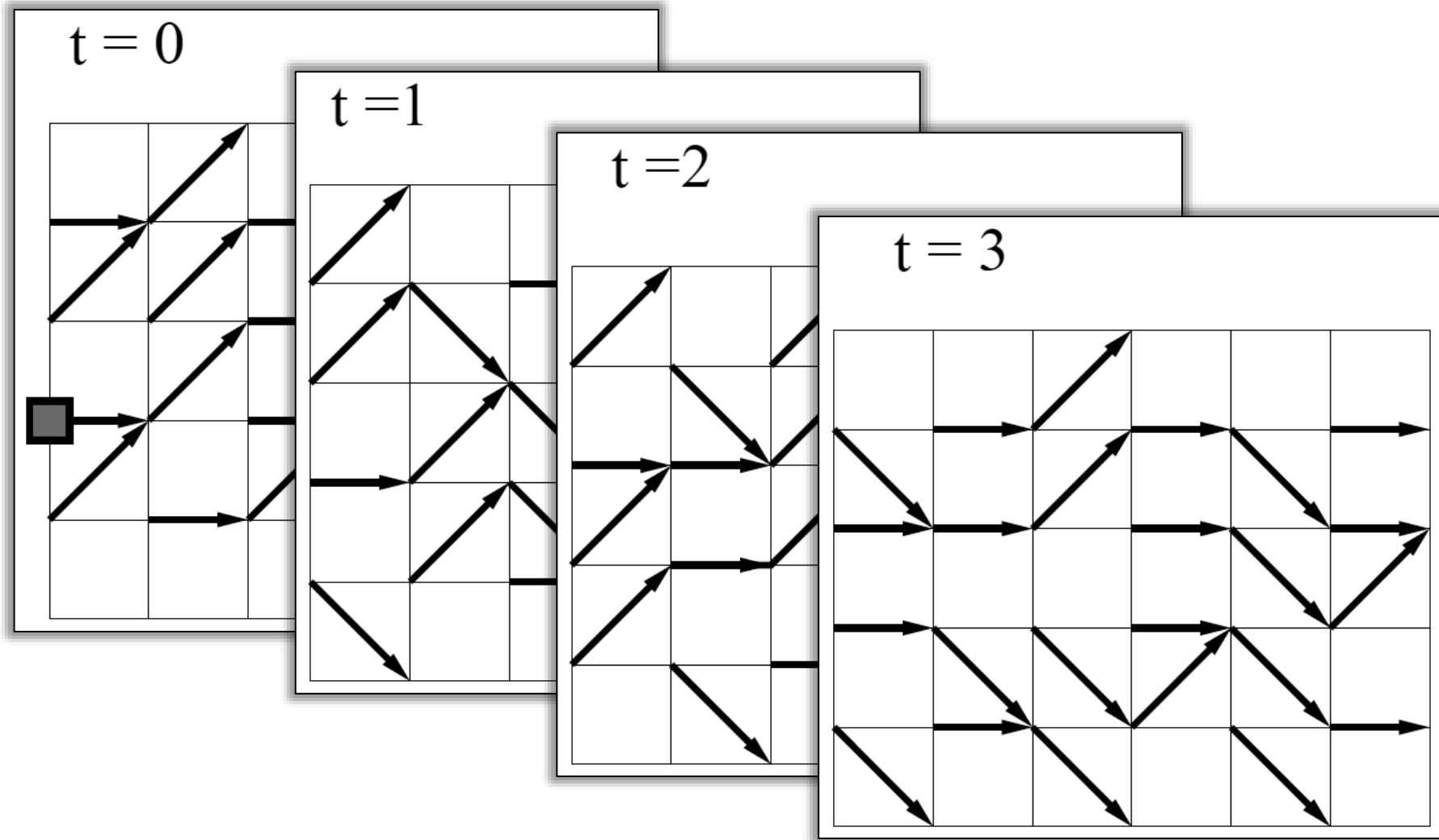


Properties of Tangent Curves

- Given a point in the vector field $\mathbf{v}$, there is one and only one tangent curve through it (except at critical points where $\mathbf{v}=\mathbf{0}$).
- Tangent curves do not intersect each other (except at critical points of $\mathbf{v}$).
- A parametric description of stream lines is usually not possible.
 - We will use numerical integration techniques (Section 11.4)



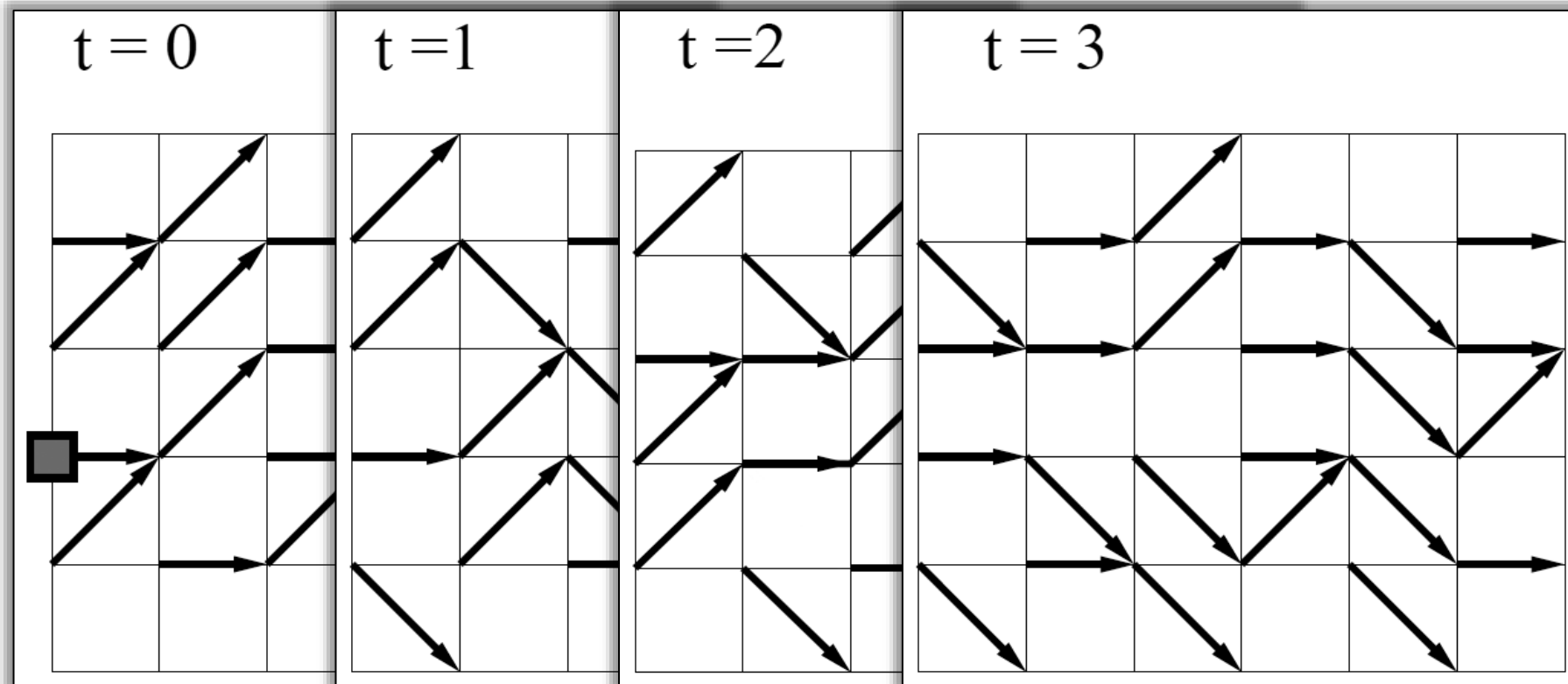
Example: An Unsteady Vector Field



Example: Stream Line

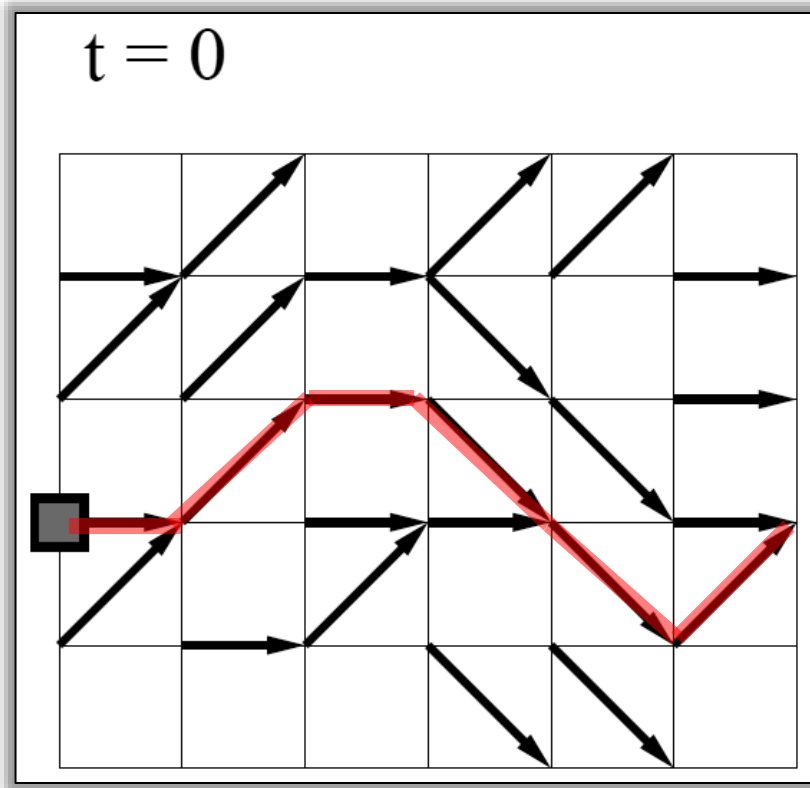
Stream line

curve tangential to the vector field in each point at a fixed time



Example: Stream Line

solution



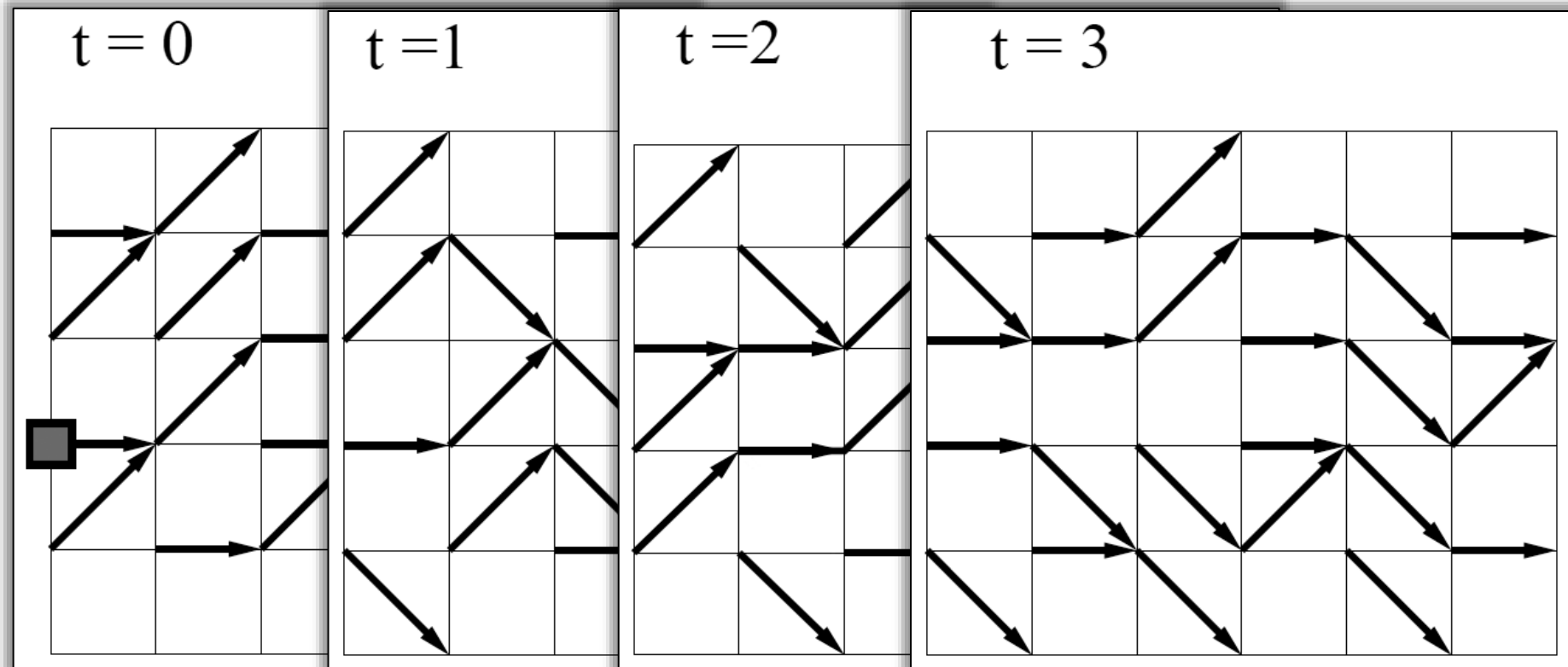
Stream line

curve tangential to the vector field in each point at a fixed time

Example: Path Line

Path line

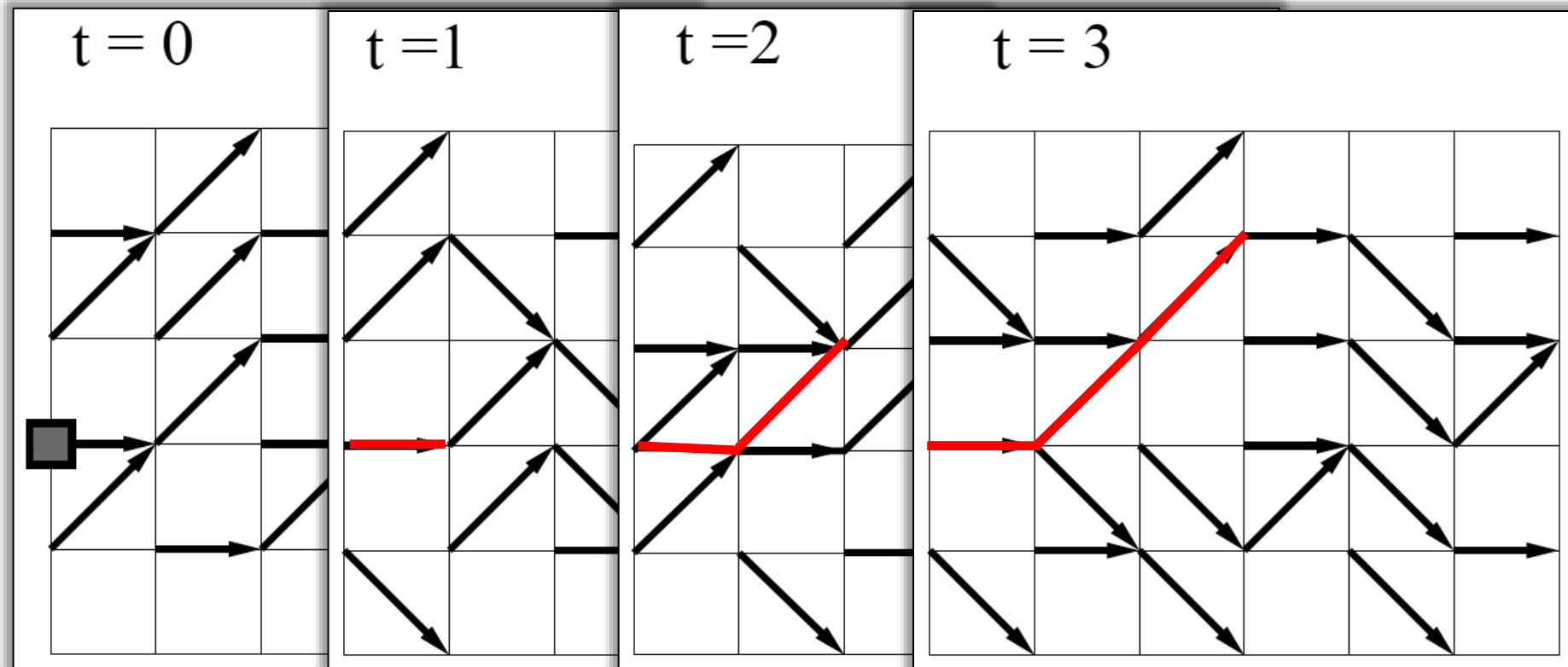
motion of a particle over
time in an unsteady flow field



Example: Path Line

Path line

motion of a particle over
time in an unsteady flow field

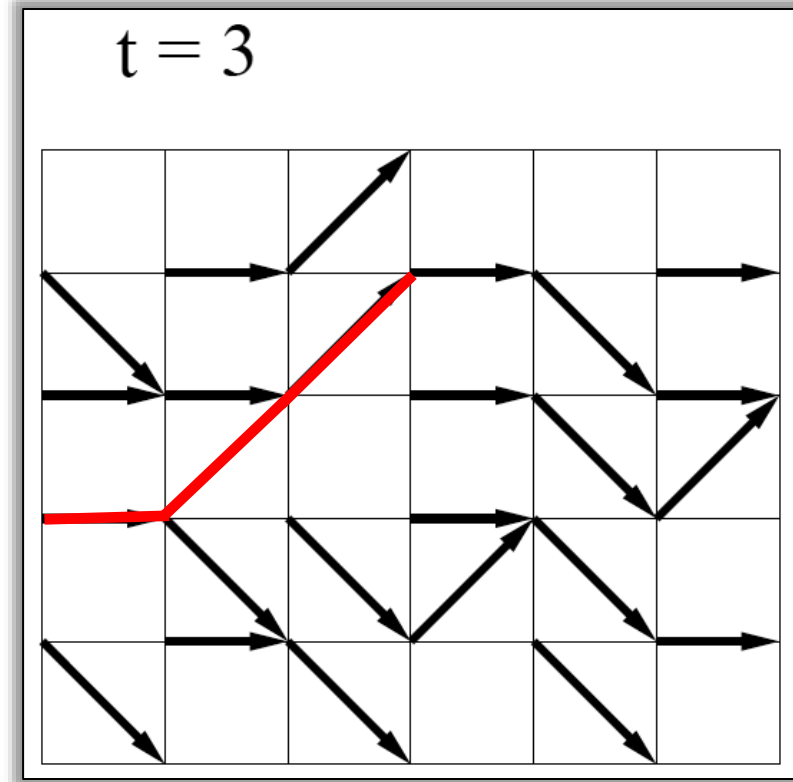


Example: Path Line

Path line

motion of a particle over
time in an unsteady flow field

solution



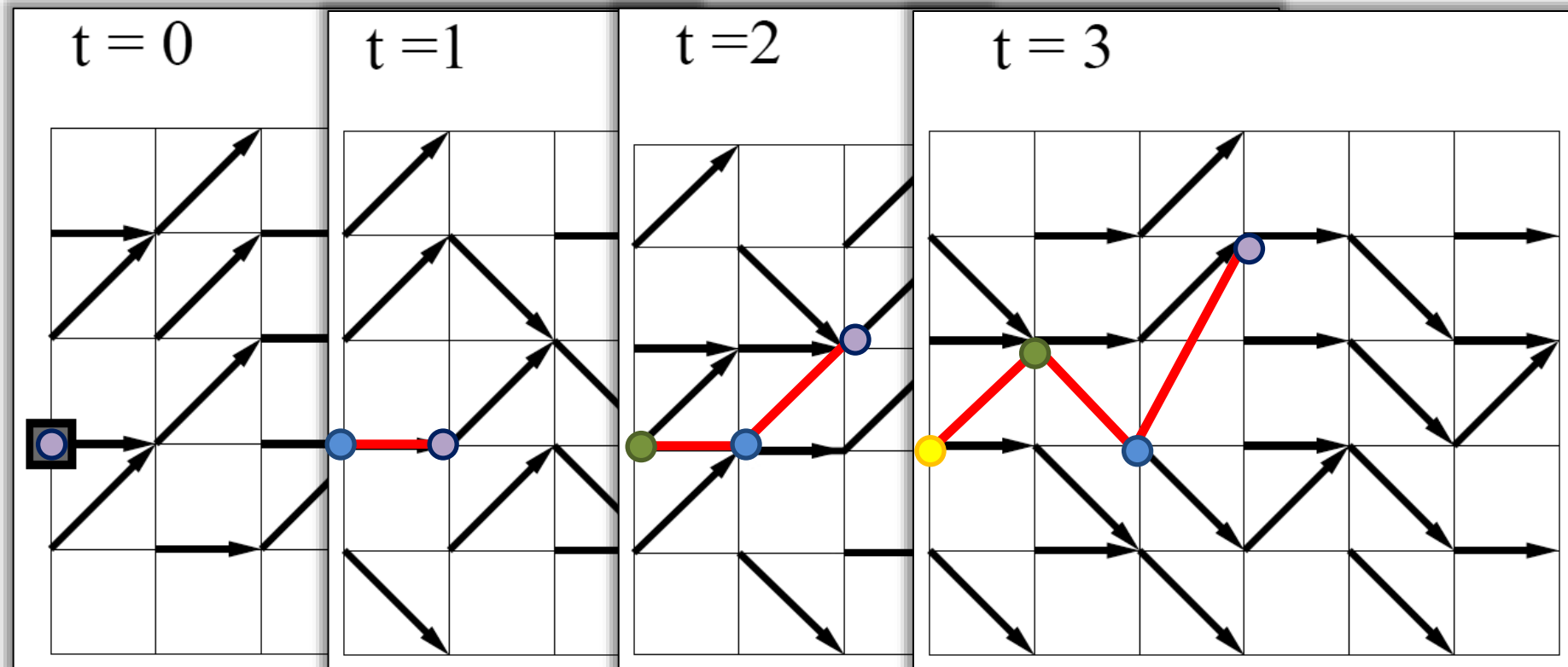
Example: Streak Line

Particle started at...

- $t=0$
- $t=1$
- $t=2$
- $t=3$

Streak line

location of all particles set out at
a fixed point at different times

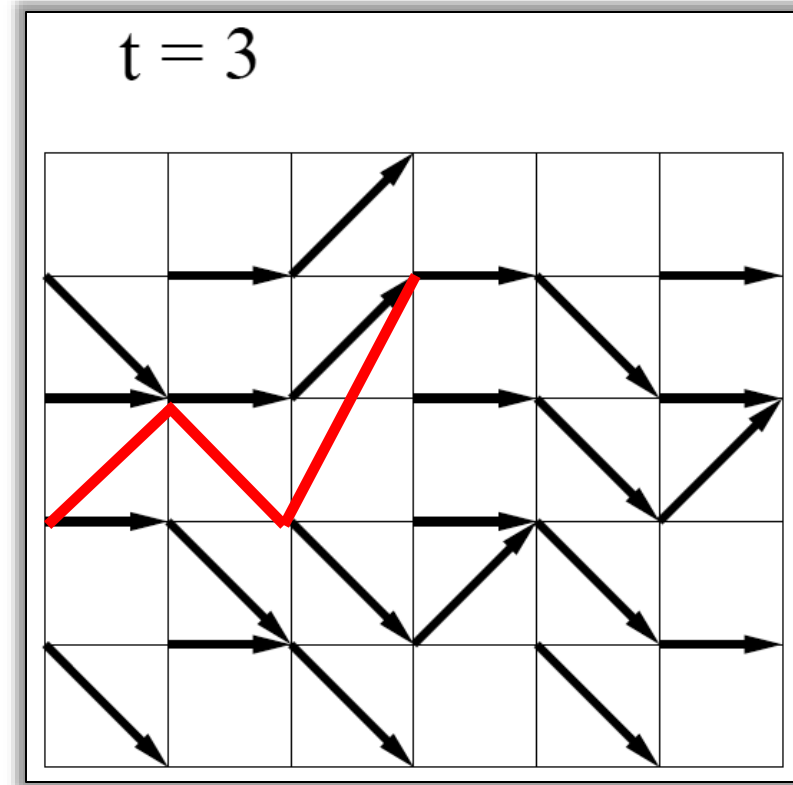


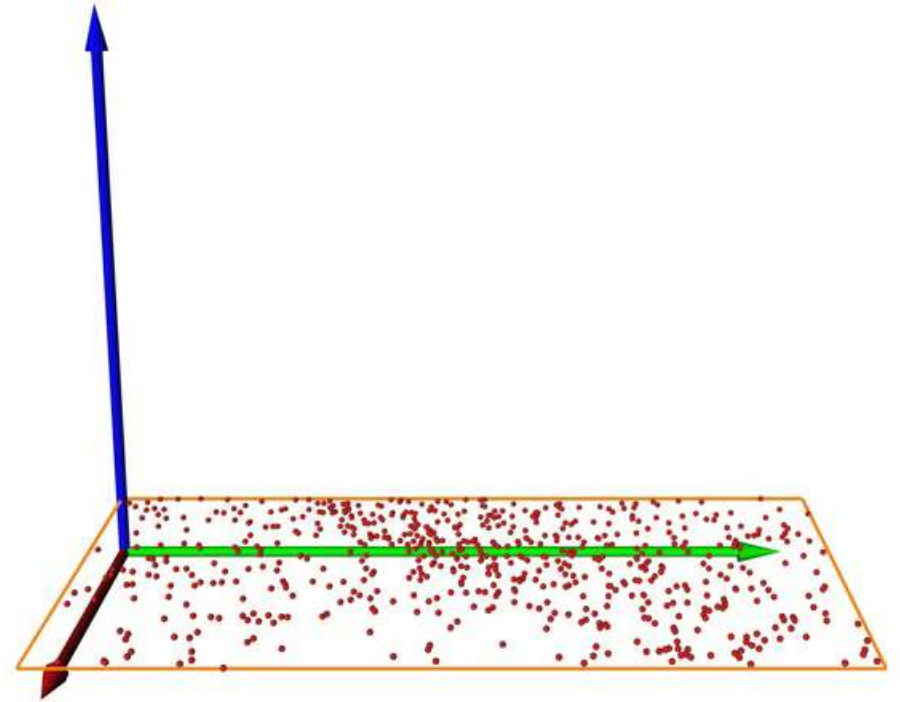
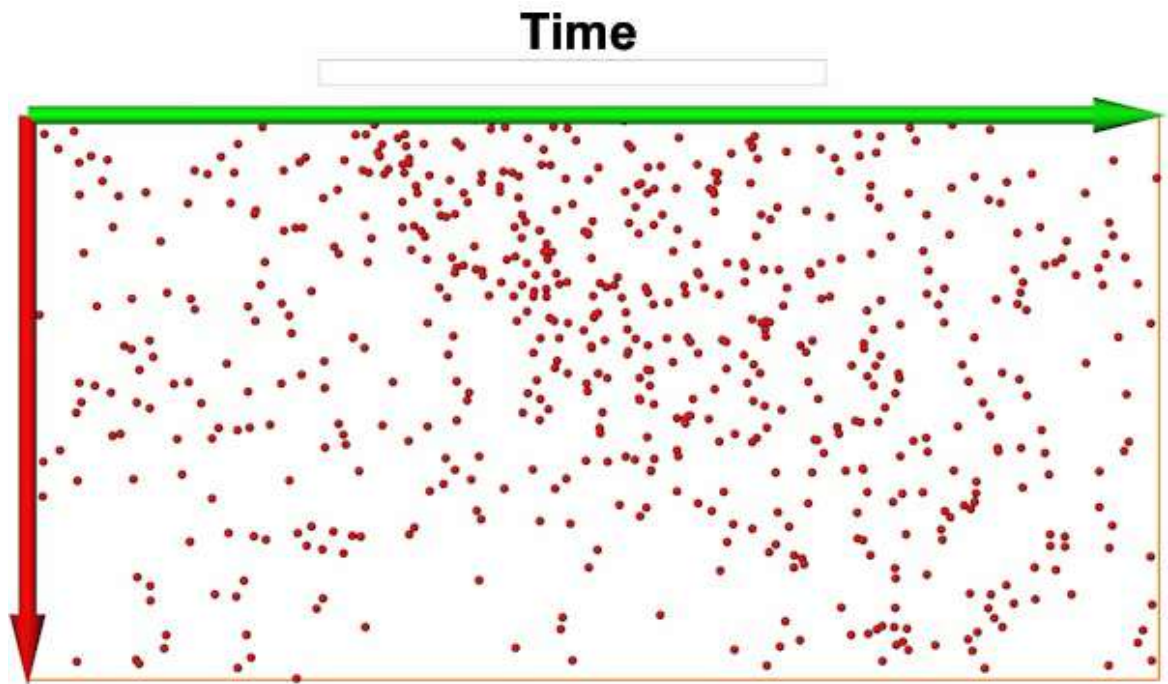
Example: Streak Line

Streak line

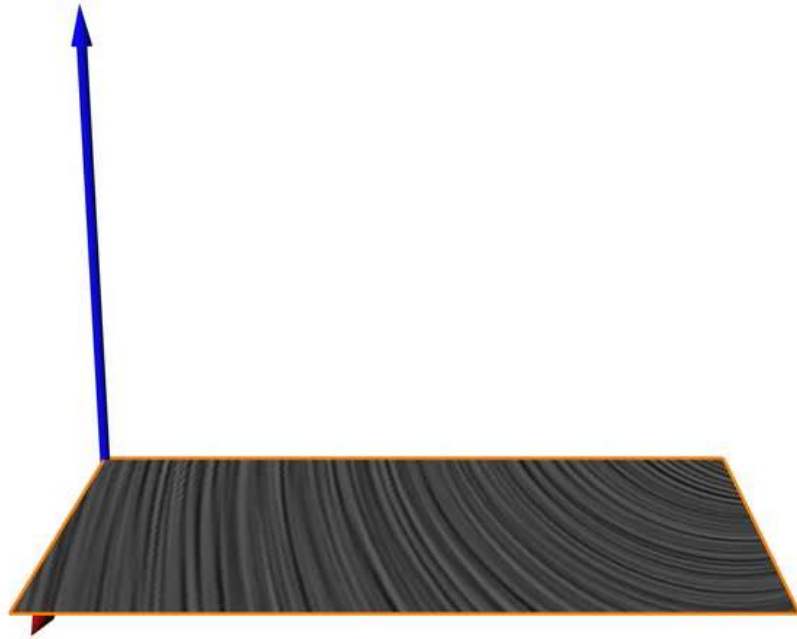
location of all particles set out at a fixed point at different times

solution





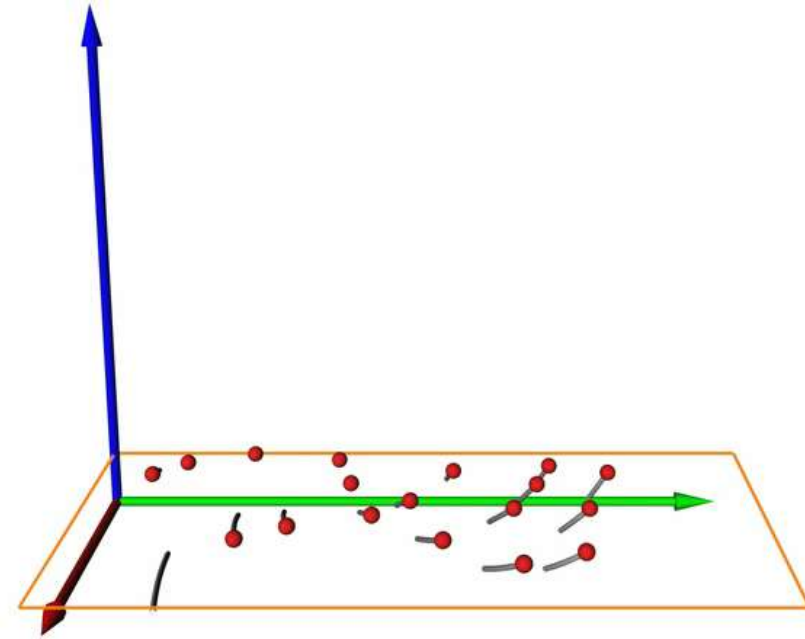
2D time-dependent vector field
particle visualization



stream lines

curve parallel to the vector field in
each point for a **fixed time**

describes motion of a massless
particle in a **steady** flow field



path lines

curve parallel to the vector field in
each point **over time**

describes motion of a massless
particle in an **unsteady** flow field

Unsteady Fields: Time as Additional Dimension

steady vector field

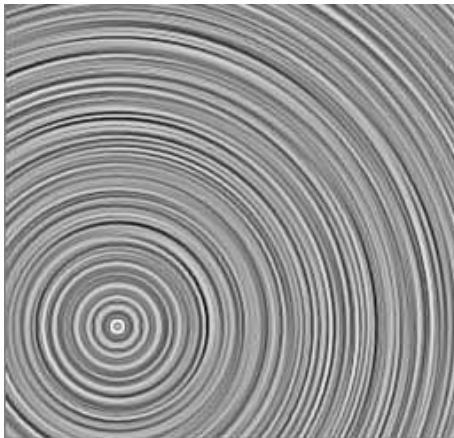
$$\mathbf{v}(x, y)$$

$$\frac{d}{d\tau} \mathbf{x}(\tau) = \mathbf{v}(\mathbf{x}(\tau))$$

with $\mathbf{x}(0) = \mathbf{x}_0$

stream lines

$$\mathbf{v}(x, y) = \begin{pmatrix} u(x, y) \\ v(x, y) \end{pmatrix}$$



unsteady vector field

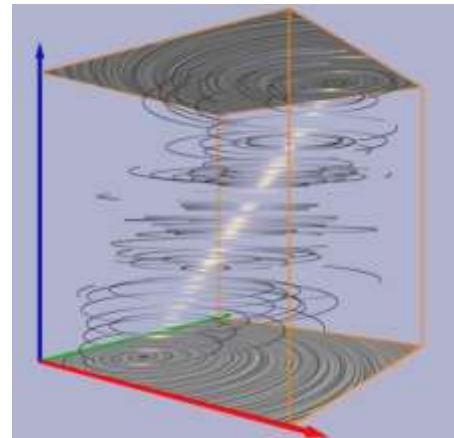
$$\mathbf{v}(x, y, t)$$

$$\frac{d}{d\tau} \mathbf{x}(\tau) = \mathbf{v}(\mathbf{x}(\tau), t_0)$$

with $\mathbf{x}(0) = \mathbf{x}_0$

stream lines

$$\mathbf{s}(x, y, t) = \begin{pmatrix} u(x, y, t) \\ v(x, y, t) \\ 0 \end{pmatrix}$$

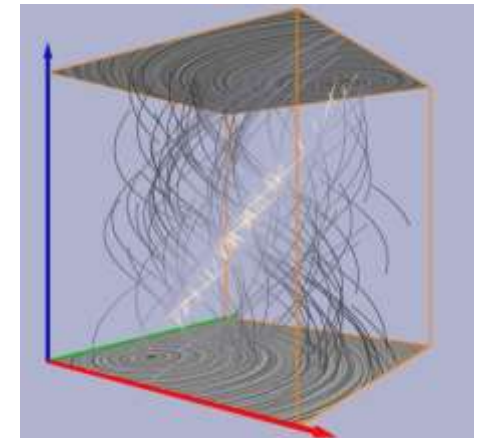


$$\frac{d}{dt} \mathbf{x}(t) = \mathbf{v}(\mathbf{x}(t), t)$$

with $\mathbf{x}(t_0) = \mathbf{x}_0$

path lines

$$\mathbf{p}(x, y, t) = \begin{pmatrix} u(x, y, t) \\ v(x, y, t) \\ 1 \end{pmatrix}$$



Unsteady Fields: Time as Additional Dimension

steady vector field

unsteady vector field

n D unsteady vector field

→ $(n+1)$ D steady vector field

2D

$$\mathbf{v}(x, y) = \begin{pmatrix} u(x, y) \\ v(x, y) \end{pmatrix}$$

$$\mathbf{s}(x, y, t) = \begin{pmatrix} u(x, y, t) \\ v(x, y, t) \\ 0 \end{pmatrix}$$

$$\mathbf{p}(x, y, t) = \begin{pmatrix} u(x, y, t) \\ v(x, y, t) \\ 1 \end{pmatrix}$$

3D

$$\mathbf{v}(x, y, z) = \begin{pmatrix} u(x, y, z) \\ v(x, y, z) \\ w(x, y, z) \end{pmatrix}$$

$$\mathbf{s}(x, y, z, t) = \begin{pmatrix} u(x, y, z, t) \\ v(x, y, z, t) \\ w(x, y, z, t) \\ 0 \end{pmatrix}$$

$$\mathbf{p}(x, y, z, t) = \begin{pmatrix} u(x, y, z, t) \\ v(x, y, z, t) \\ w(x, y, z, t) \\ 1 \end{pmatrix}$$

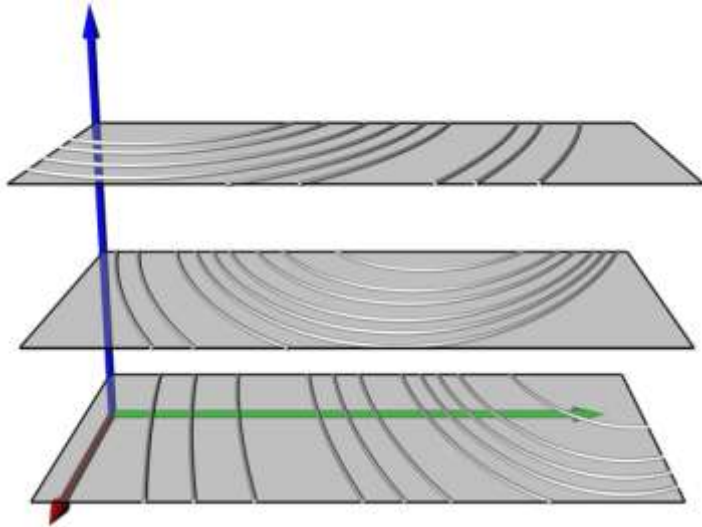


Time

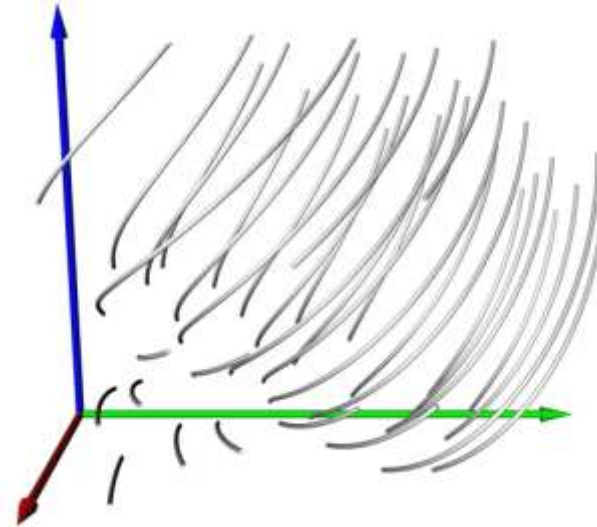


streak line

location of all particles set out at a fixed point at different times

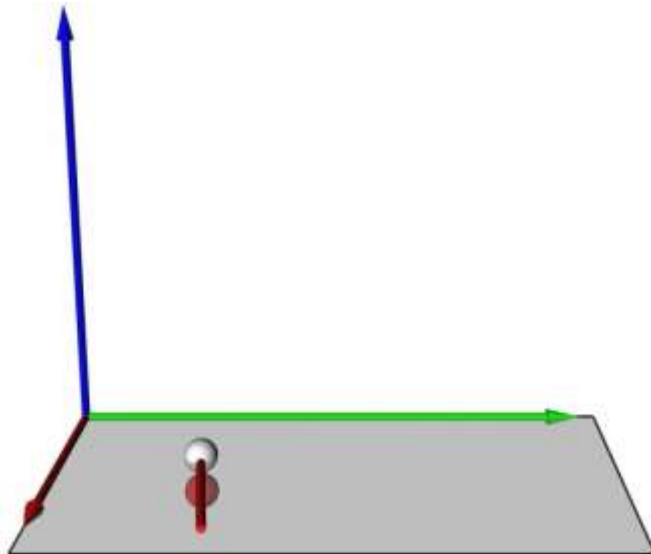


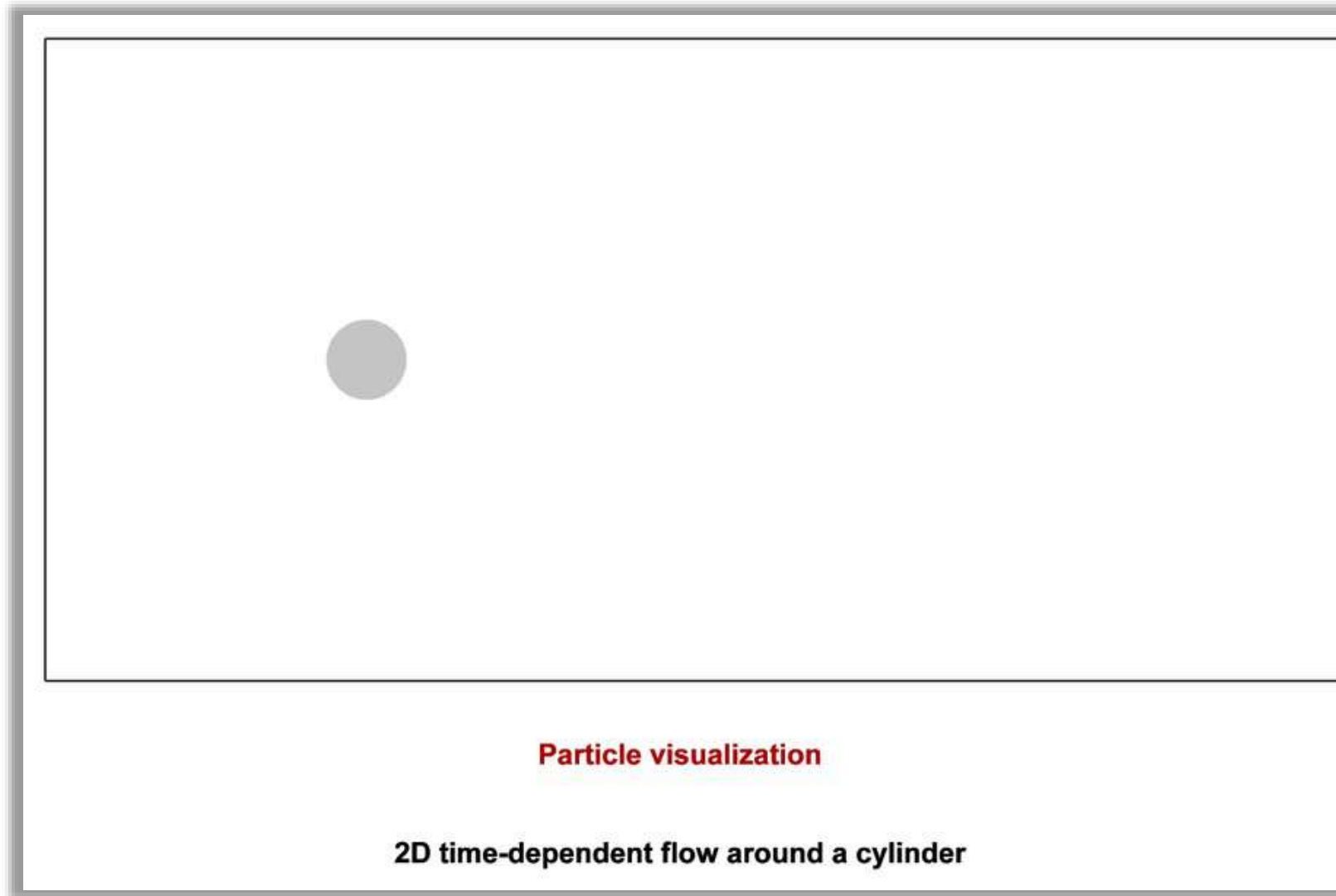
stream lines



path lines

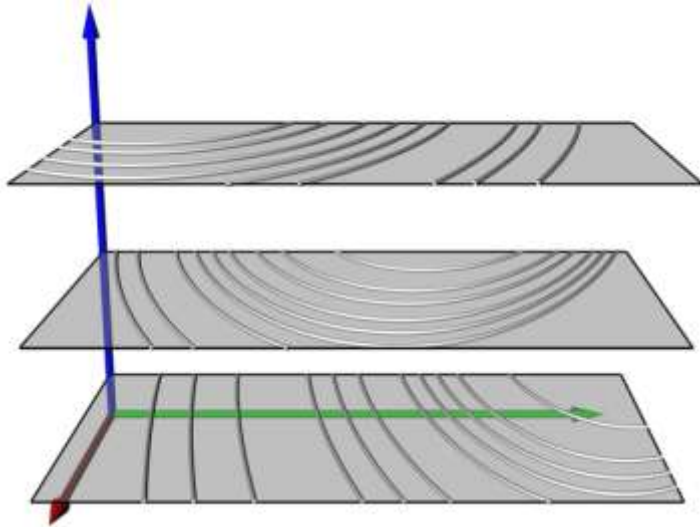
streak lines



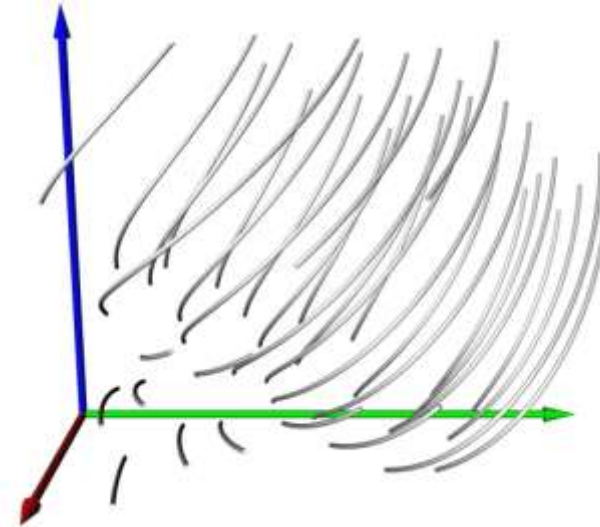


time line

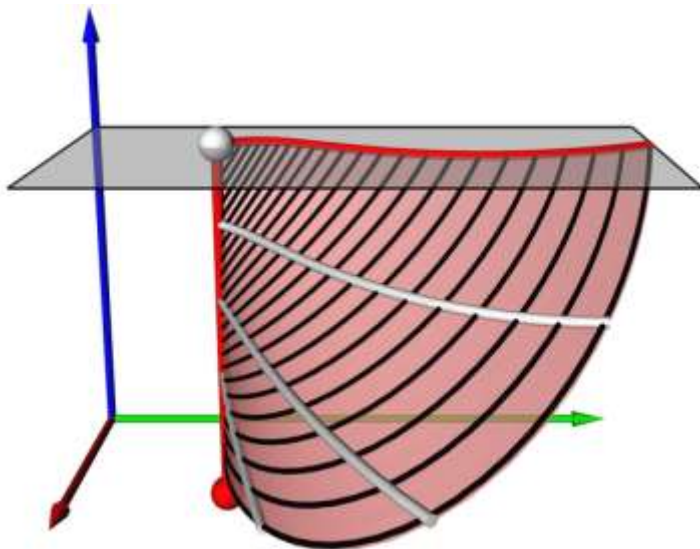
location of all particles set out on a certain line at a fixed time



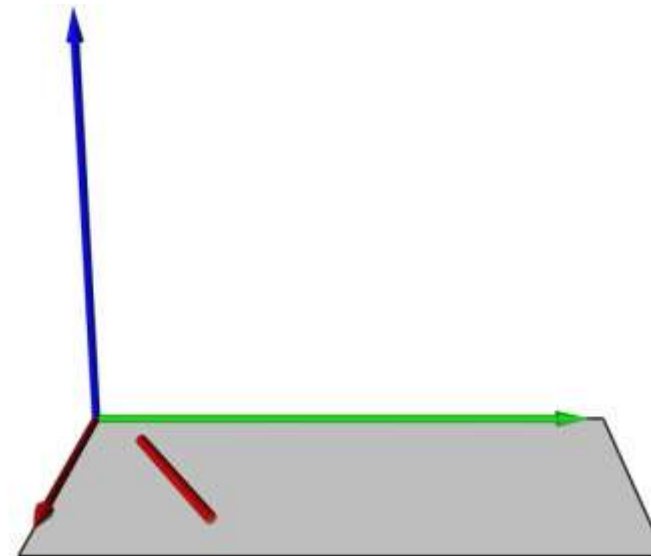
stream lines



path lines



streak lines

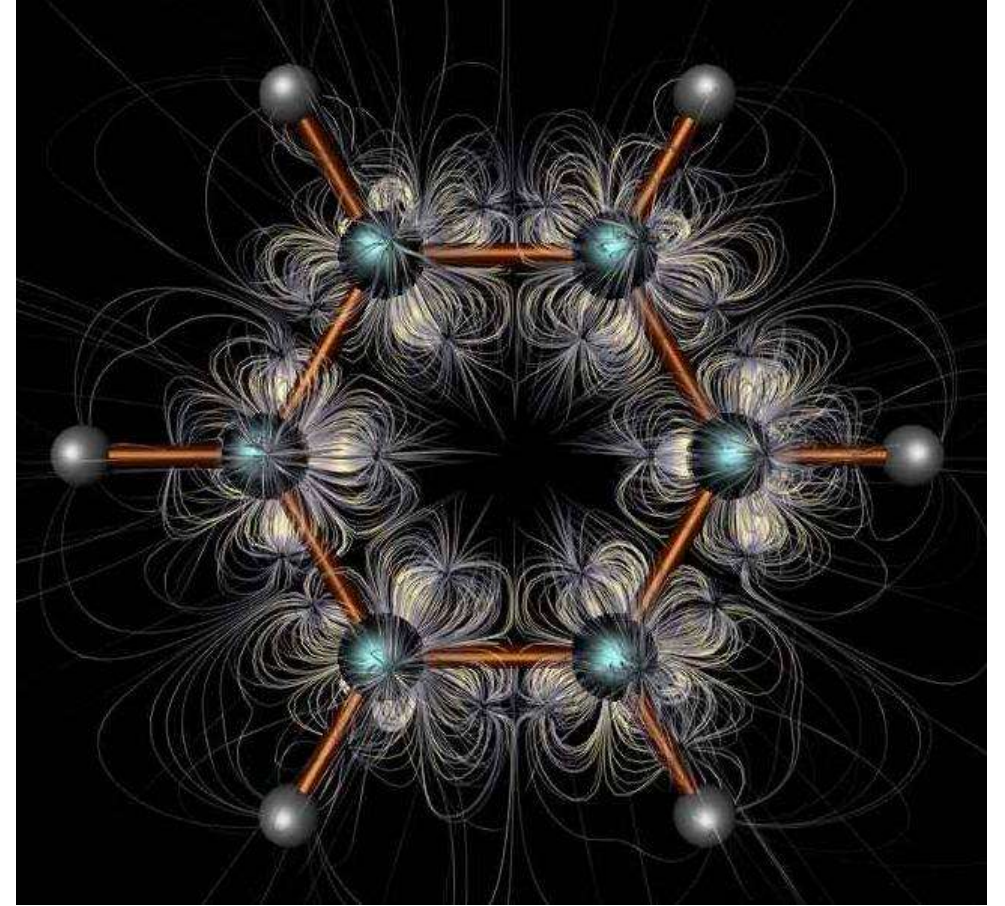
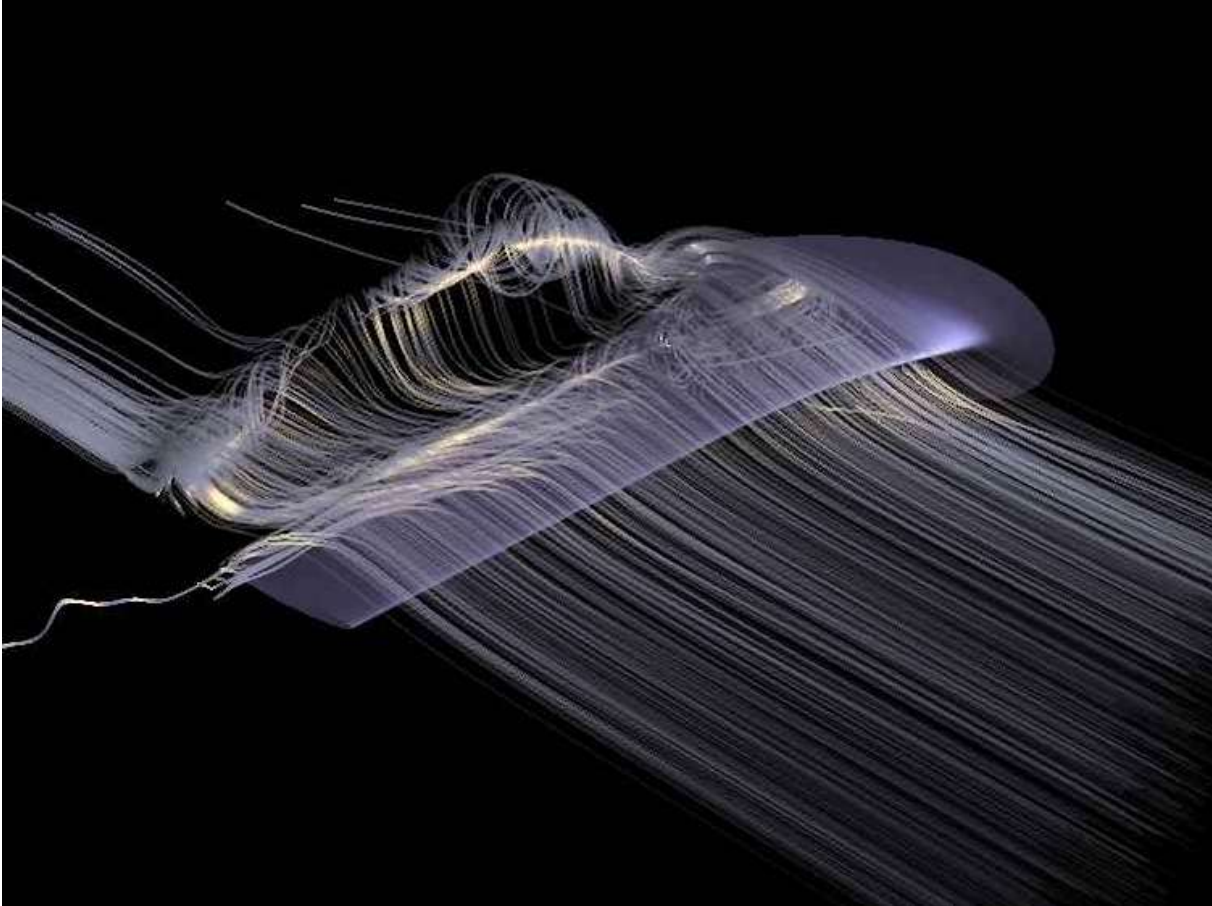


time lines

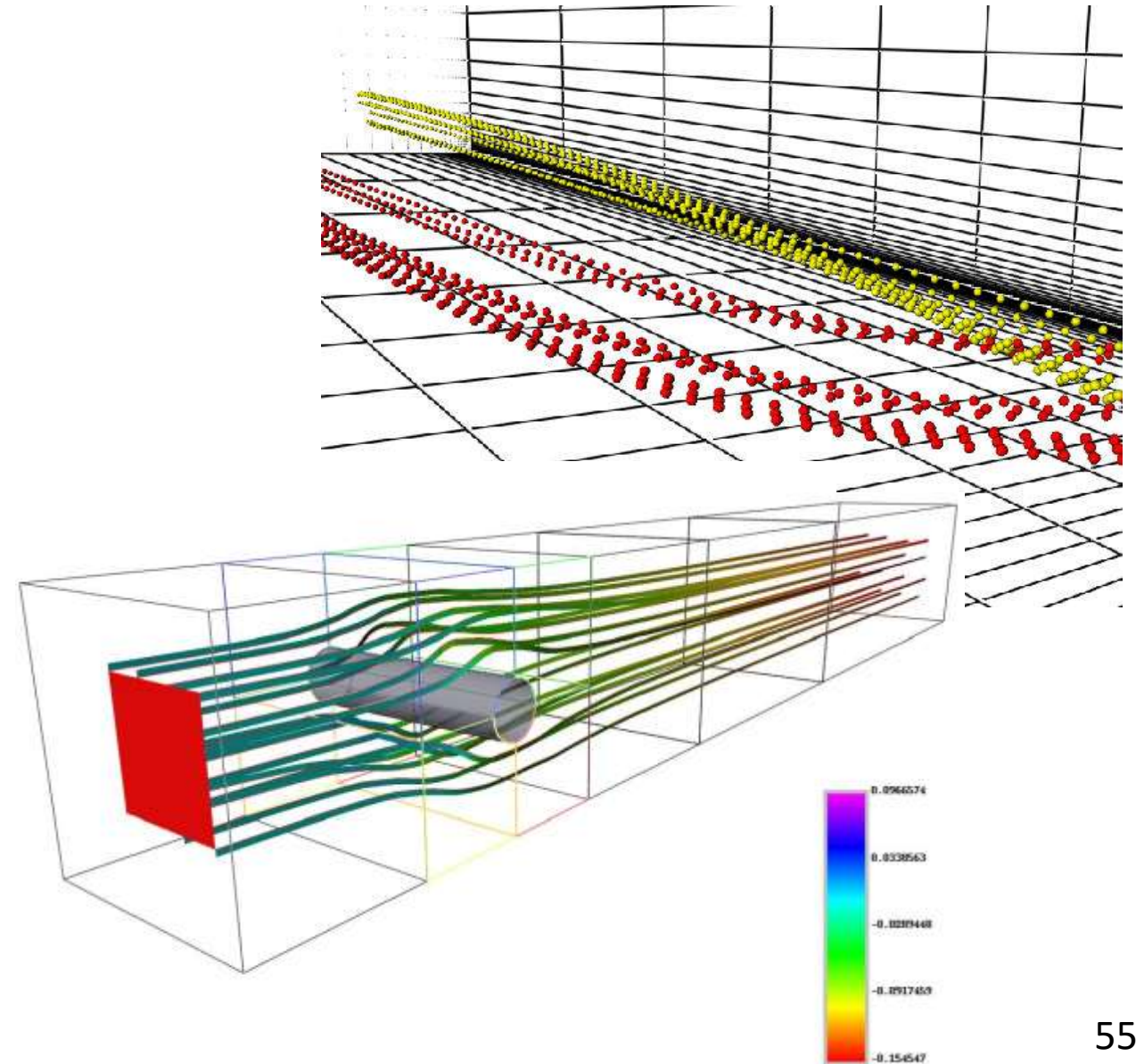
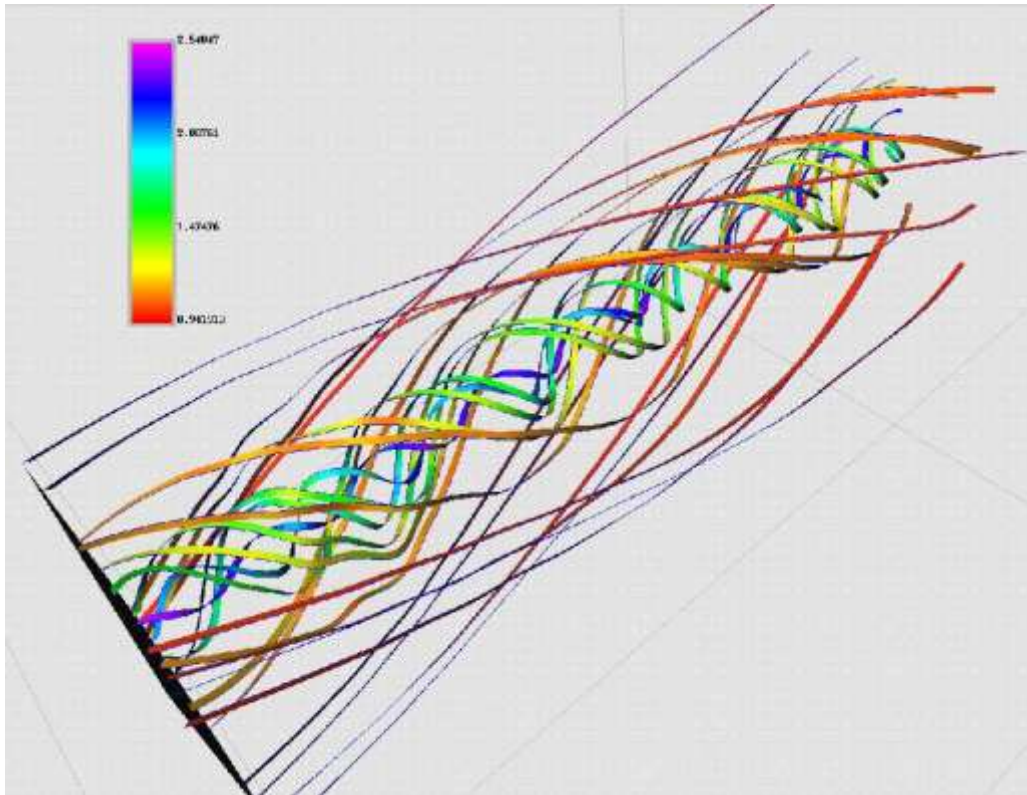
Properties of Characteristic Curves

- **Stream and Path Lines:**
 - There is exactly one stream/path line passing through each non-critical point $(\mathbf{x}, t)$ in space-time.
- **Streak and Time Lines:**
 - Many streak lines / time lines through every point, depending on the seeding structure
- In steady vector fields, stream lines, path lines, and streak lines all coincide.

Examples: Path Lines

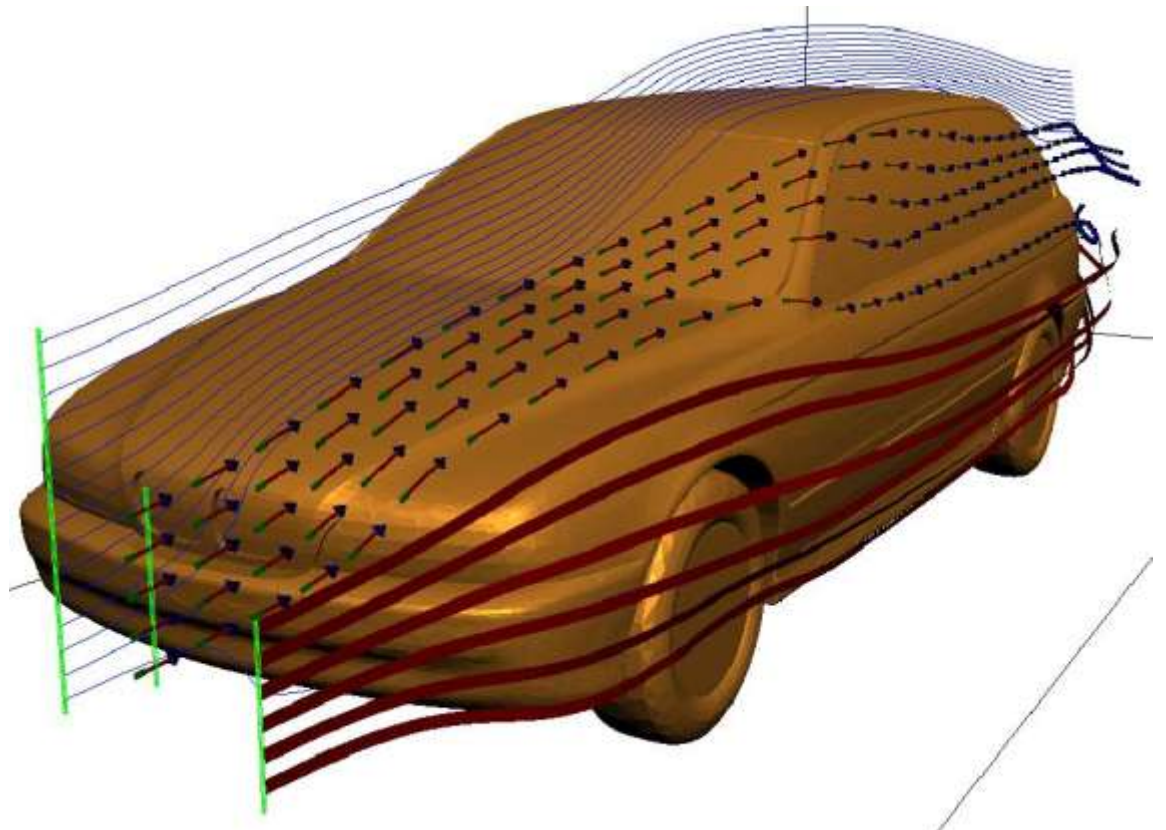
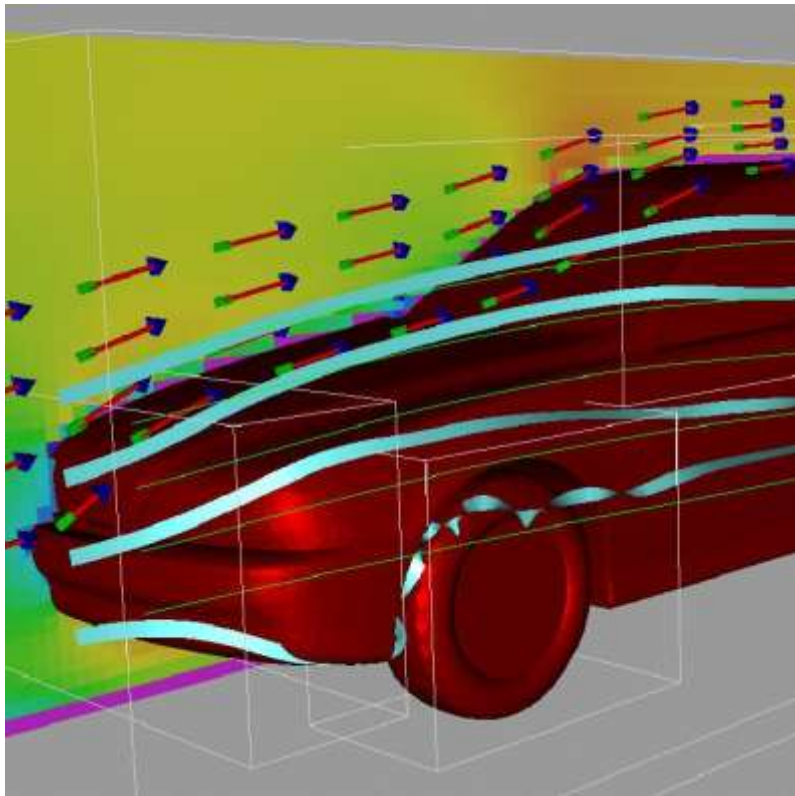


Examples: Streak Lines



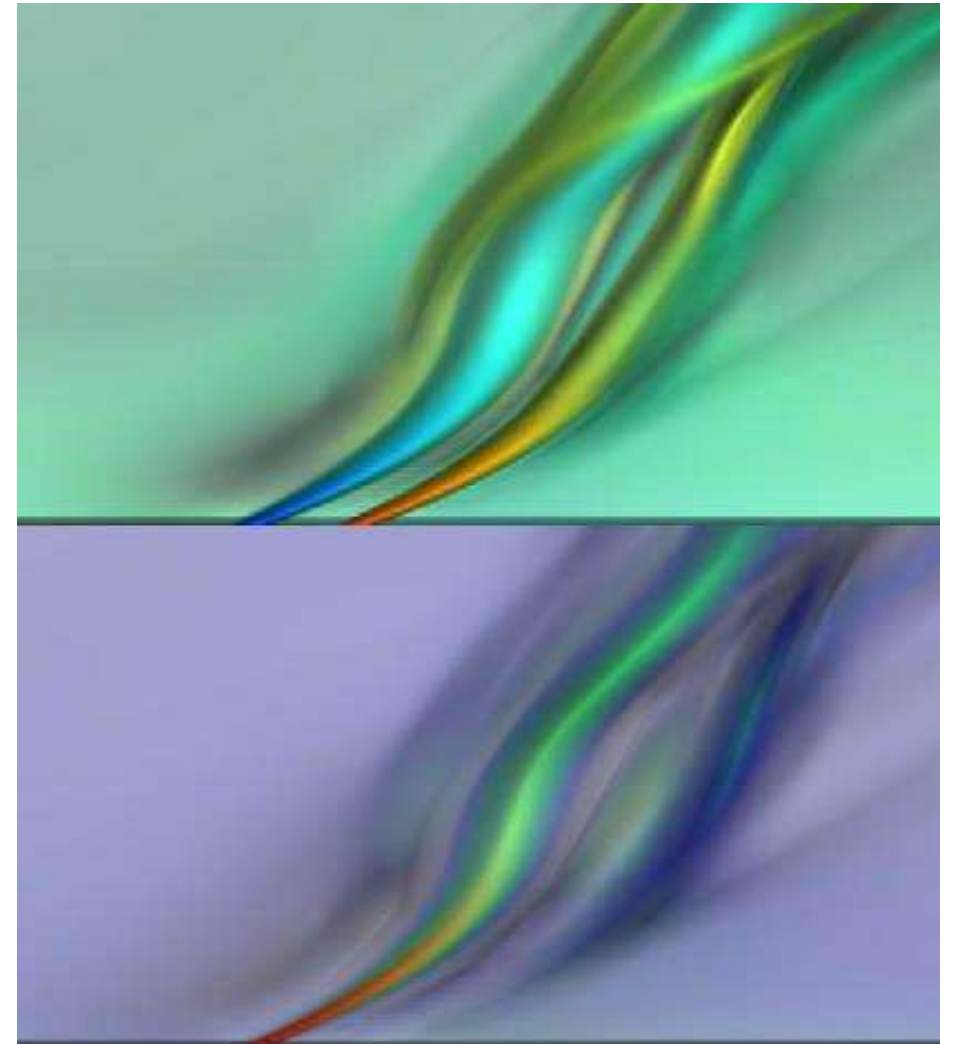
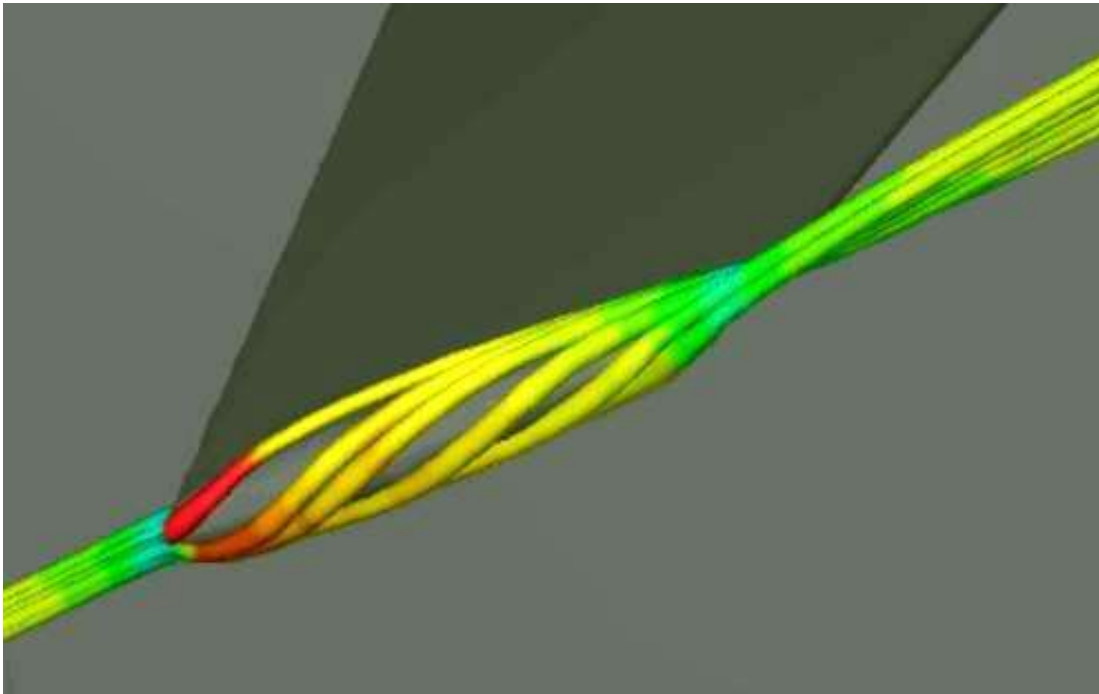
Examples: Stream Ribbons

- Trace a particle through the field
- Attach a line segment orthogonal to the field, rotate it according to curl



Examples: Stream Tubes

- Specify contour, e.g. triangle or circle, and trace it through the flow
- Or, render a fixed cross-sectional geometry around characteristic curve



Section 11.4:

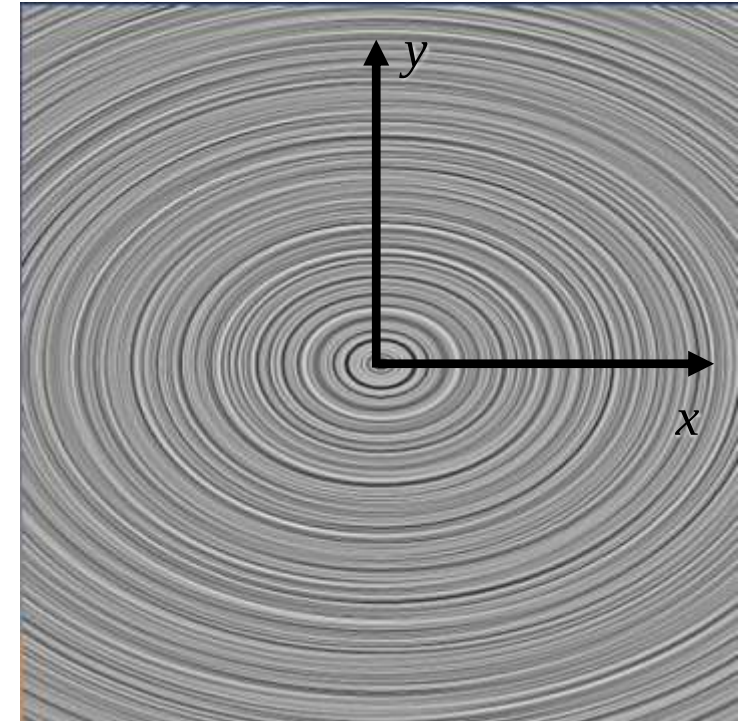
Stream Line Integration

Numerical Integration of Stream Lines

Goal: Piecewise linear approximation of streamline by polyline $\mathbf{x}_i$

- **Testing example:**

- $v(x, y) = \begin{pmatrix} -y \\ \frac{x}{2} \end{pmatrix}$
- exact solution: ellipses
- starting integration from (0,-1)



Reminder: Taylor Approximation

- Smooth functions can be approximated locally:

$$\begin{aligned} f(x) &\approx f(x_0) \\ &+ \frac{d}{dx} f(x_0)(x - x_0) \\ &+ \frac{1}{2} \frac{d^2}{dx^2} f(x_0)(x - x_0)^2 + \dots \\ &\dots + \frac{1}{n!} \frac{d^n}{dx^n} f(x_0)(x - x_0)^n \\ &+ O(x^{n+1}) \end{aligned}$$

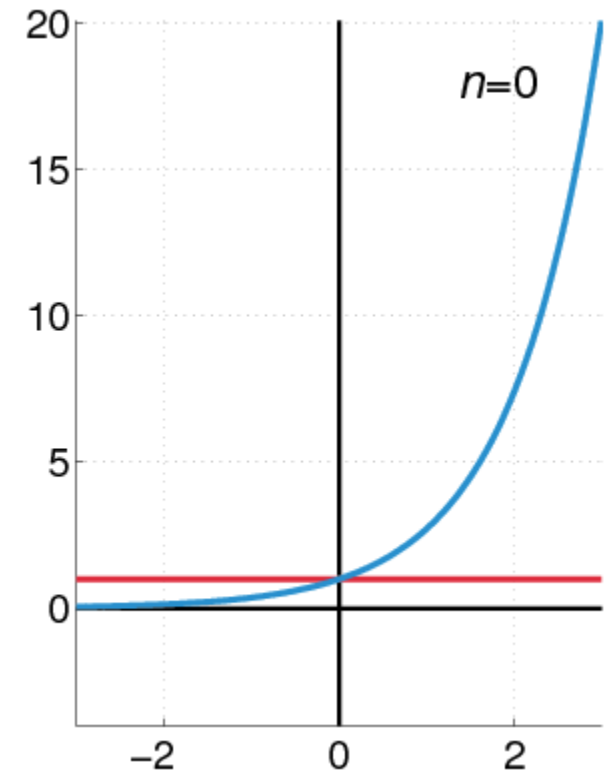
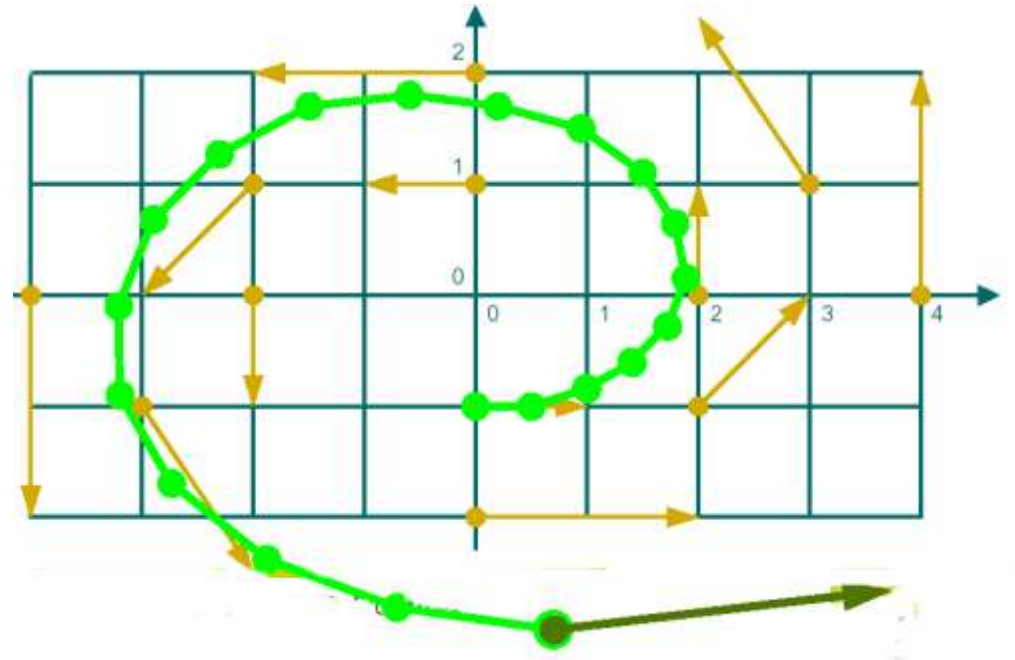


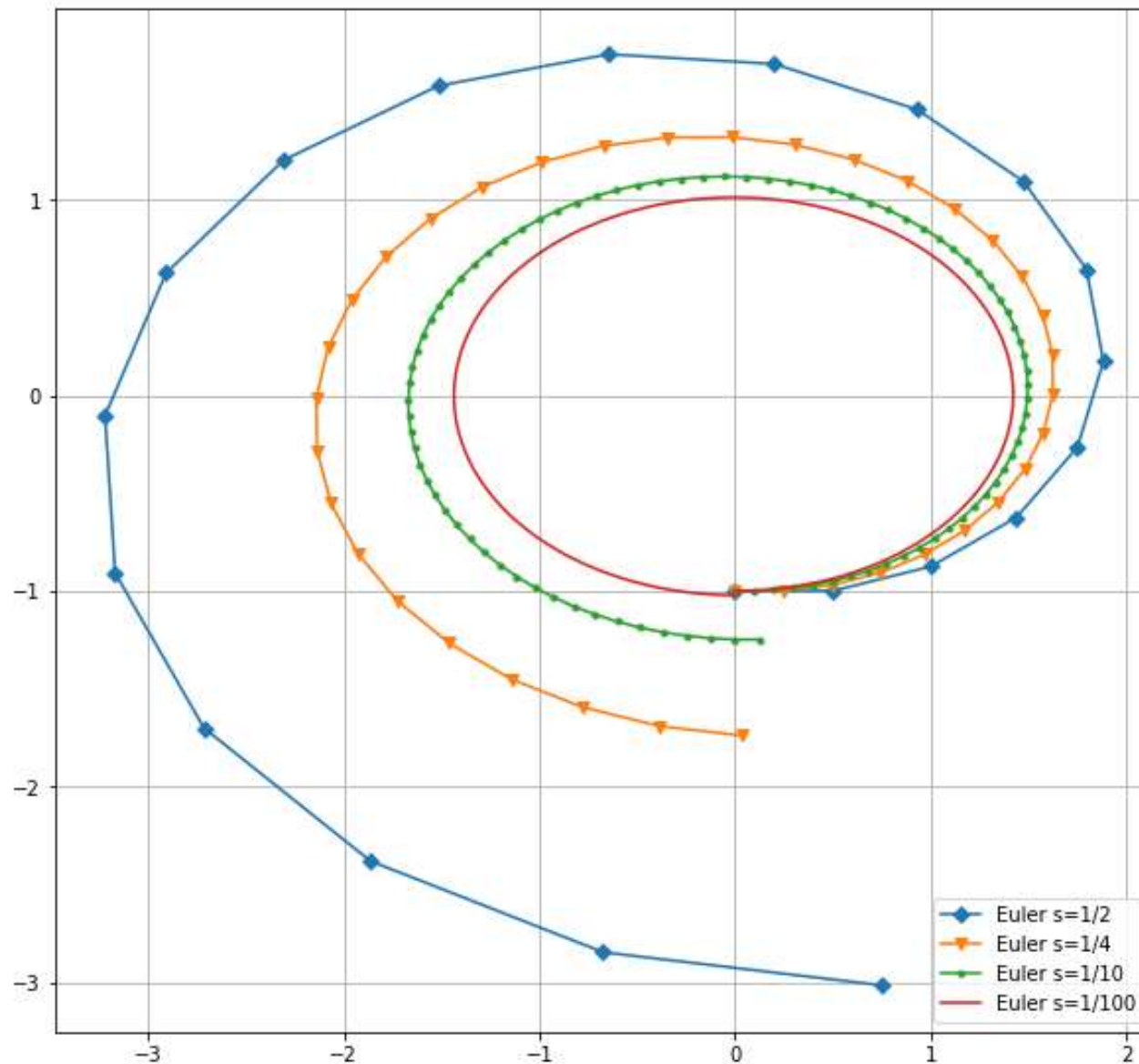
Image Source: Wikipedia

Euler Integration

- Idea: $\mathbf{x}_{i+1} = \mathbf{x}_i + s \mathbf{v}(\mathbf{x}_i)$
- Taylor: $\mathbf{x}(t_0 + s) = \mathbf{x}(t_0) + s\dot{\mathbf{x}}(t_0) + O(s^2)$
- First-order scheme $\rightarrow$ error of order s^2
 - Large numerical error
- s : step size



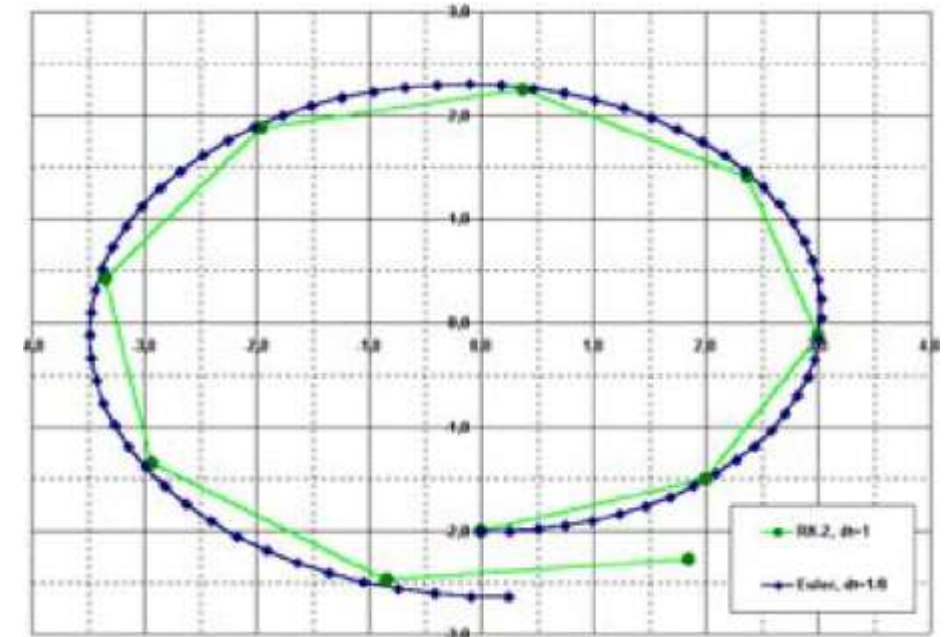
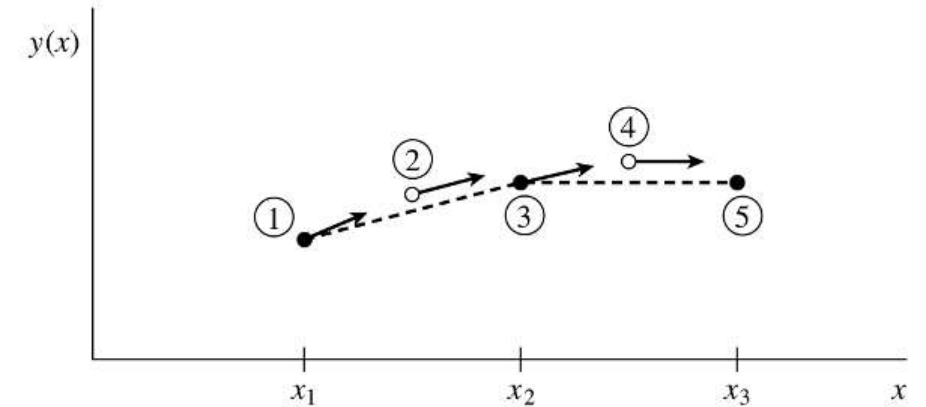
Error in Euler Integration



Second-Order Runge Kutta Integration

$$\mathbf{x}_{i+1} = \mathbf{x}_i + s \mathbf{v}(\mathbf{x}_i + s/2 \mathbf{v}(\mathbf{x}_i))$$

- also known as „midpoint method“
- Idea:
 - go half a (Euler) step forward
 - evaluate vector there
 - use that vector *at the starting point*
- error of order s^3



Top image from [Press et al. 2007]

Analysis of Midpoint Method

- Pretend we have access to the true midpoint:

$$\mathbf{x}(t_0) = \mathbf{x}\left(t_0 + \frac{s}{2}\right) - \frac{s}{2}\dot{\mathbf{x}}\left(t_0 + \frac{s}{2}\right) + \frac{s^2}{8}\ddot{\mathbf{x}}\left(t_0 + \frac{s}{2}\right) + O(s^3)$$

$$\mathbf{x}(t_0 + s) = \mathbf{x}\left(t_0 + \frac{s}{2}\right) + \frac{s}{2}\dot{\mathbf{x}}\left(t_0 + \frac{s}{2}\right) + \frac{s^2}{8}\ddot{\mathbf{x}}\left(t_0 + \frac{s}{2}\right) + O(s^3)$$

- Subtract:

$$\mathbf{x}(t_0 + s) = \mathbf{x}(t_0) + s\dot{\mathbf{x}}\left(t_0 + \frac{s}{2}\right) + O(s^3)$$

- Consider error in estimate of $\dot{\mathbf{x}}\left(t_0 + \frac{s}{2}\right)$:

$$\mathbf{v}\left(\mathbf{x}\left(t_0 + \frac{s}{2}\right) + \mathbf{r}\right) = \mathbf{v}\left(\mathbf{x}\left(t_0 + \frac{s}{2}\right)\right) + \mathbf{J}\mathbf{r} + O(\|\mathbf{r}\|^2)$$

- From Euler scheme, we know that $\|\mathbf{r}\| = O(s^2)$

Fourth-Order Runge Kutta Integration

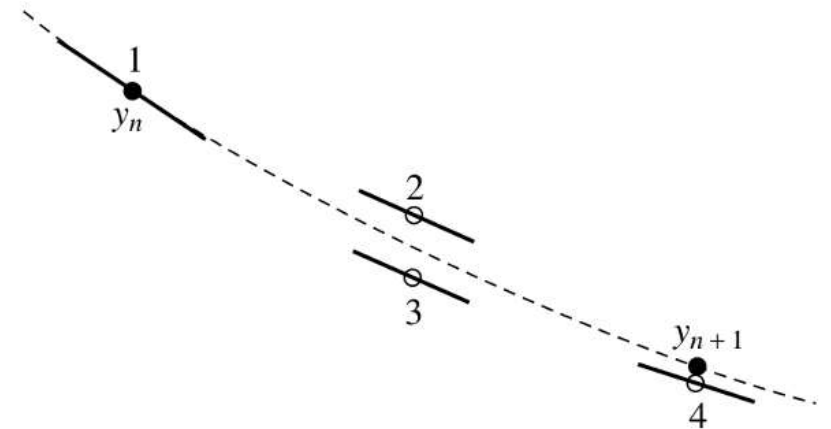
- Standard in many applications
 - Error term reduced to order s^5
- Idea: step is a convex combination of 4 vectors

$$\mathbf{x}_{i+1} = \text{RK4}(\mathbf{x}_i) = \mathbf{x}_i + s \cdot \left(\frac{\mathbf{v}_1}{6} + \frac{\mathbf{v}_2}{3} + \frac{\mathbf{v}_3}{3} + \frac{\mathbf{v}_4}{6} \right)$$

with

$$\mathbf{v}_1 = \mathbf{v}(\mathbf{x}_i) \quad , \quad \mathbf{v}_2 = \mathbf{v}\left(\mathbf{x}_i + \frac{s}{2} \mathbf{v}_1\right)$$

$$\mathbf{v}_3 = \mathbf{v}\left(\mathbf{x}_i + \frac{s}{2} \mathbf{v}_2\right) \quad , \quad \mathbf{v}_4 = \mathbf{v}(\mathbf{x}_i + s \mathbf{v}_3)$$



Adaptive Stepsize: Stepsize Doubling

Idea: Small steps are only needed in highly curved regions

- Attempt the same step with two different step sizes (one big step vs. two small steps)

$$\begin{aligned} x_1 &= RK4(x, 2s) \\ x_2 &= RK4(RK4(x, s), s) \end{aligned} \Rightarrow x(t + 2s) = \begin{cases} x_1 + (2s)^5 \phi + O(s^6) \\ x_2 + 2(s)^5 \phi + O(s^6) \end{cases}$$

- Indication of truncation error: $\Delta = \|x_2 - x_1\|$
- If error Δ is larger than a given tolerance level τ , the step is attempted again with a smaller step size s
- Otherwise the step is taken and s' is estimated for the next integration step:

$$s' = s \cdot \rho \cdot \sqrt[5]{\frac{\tau}{\Delta}}, \text{ with safety factor } \rho < 1$$

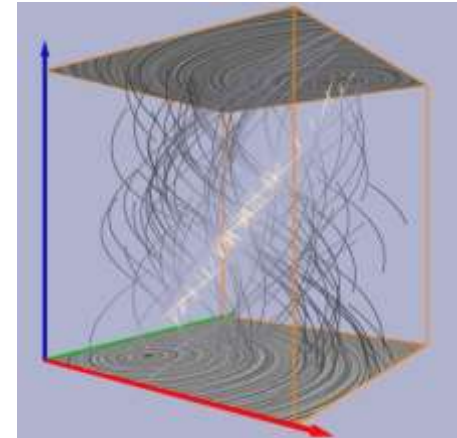
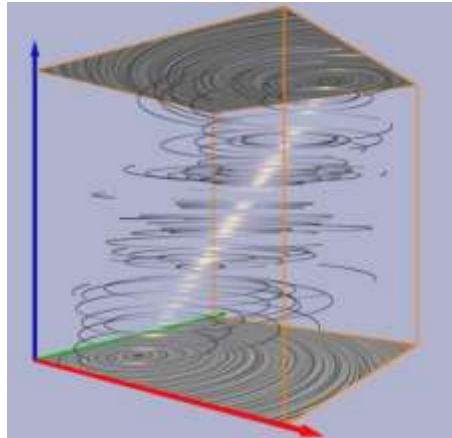
Embedded Runge-Kutta

- **Main idea:** Take step with schemes of different order and estimate error as difference between the two results $\Delta = \|x - x^*\|$
 - *Trick:* Certain pairs of schemes can re-use function evaluations from higher-order scheme in the lower-order scheme
 - Less computational effort than stepsize doubling
 - Originally invented by Merson and Fehlberg
 - Typical example “RK5(4)”:
 x is computed with fifth order, x^* with fourth order RK
 - Details and equations given in [Press et al. 2007]

Line Integration in Unsteady Flows

- stream lines, path lines: similar to steady case (integrate **s**, **p** respectively)

$$\mathbf{s}(x, y, t) = \begin{pmatrix} u(x, y, t) \\ v(x, y, t) \\ 0 \end{pmatrix}$$



$$\mathbf{p}(x, y, t) = \begin{pmatrix} u(x, y, t) \\ v(x, y, t) \\ 1 \end{pmatrix}$$

- streak lines, time lines:
 - represent lines as polygons
 - every vertex of polygon has to be integrated simultaneously in **p**
 - adaptive refinement necessary

Computational effort
much higher than for
stream or path lines

Summary: Stream Line Integration

- Stream line integration is performed with numerical schemes such as Euler or Runge-Kutta
 - Order defines how quickly error is reduced with decreasing step size
 - Fourth-order Runge Kutta is a widely used standard, but best option depends on the application
- Estimates of the local error can be used for adaptive stepsize
 - Embedded methods faster than step size doubling
- Adaptive refinement needed for streak and time lines

Section 11.5:

Stream Line Seeding and Selection

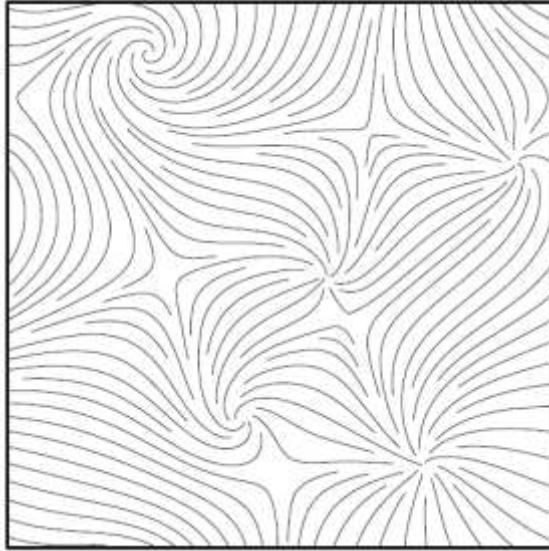
Stream Line Seeding

- **Problem:** which stream/path/streak/time lines to visualize?
 - too few: important details get lost
 - too many: overload, visual clutter
 - wrong location: possibly lost detail and clutter
- Simple approaches:
 - start on regular grid points
 - start on grid points plus random jitter
 - start randomly

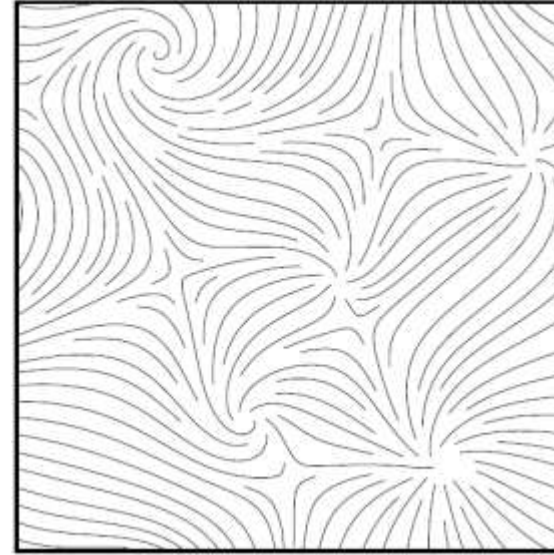


Seed points on
uniform grid

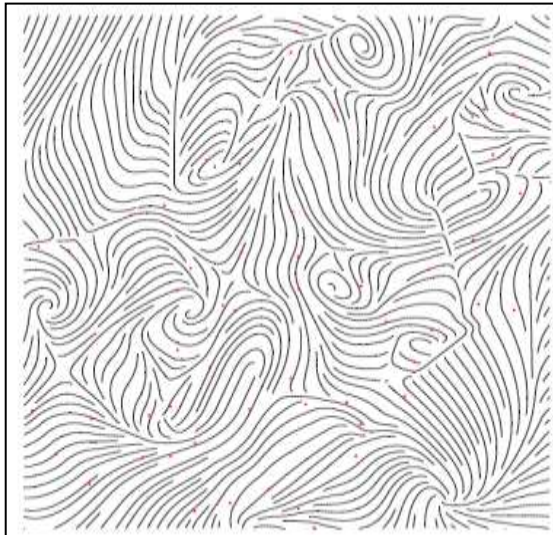
Examples of Stream Line Seeding



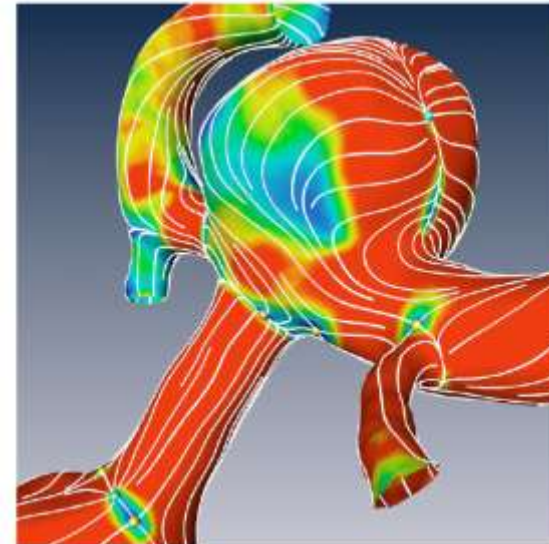
Turk and Banks, 1996



Jobard et al., 1997



Mebarki et al., 2005



Rosanwo et al., 2009

Image-Guided Stream Line Placement

- **[Turk/Banks 96]:** Attempt uniform coverage
 - start with streamlets (very short stream lines)
 - apply a series of energy-decreasing elementary operations: combine, delete, create, lengthen, shorten streamlets
 - energy: difference between low-pass filtered version of current placements and uniform grey image

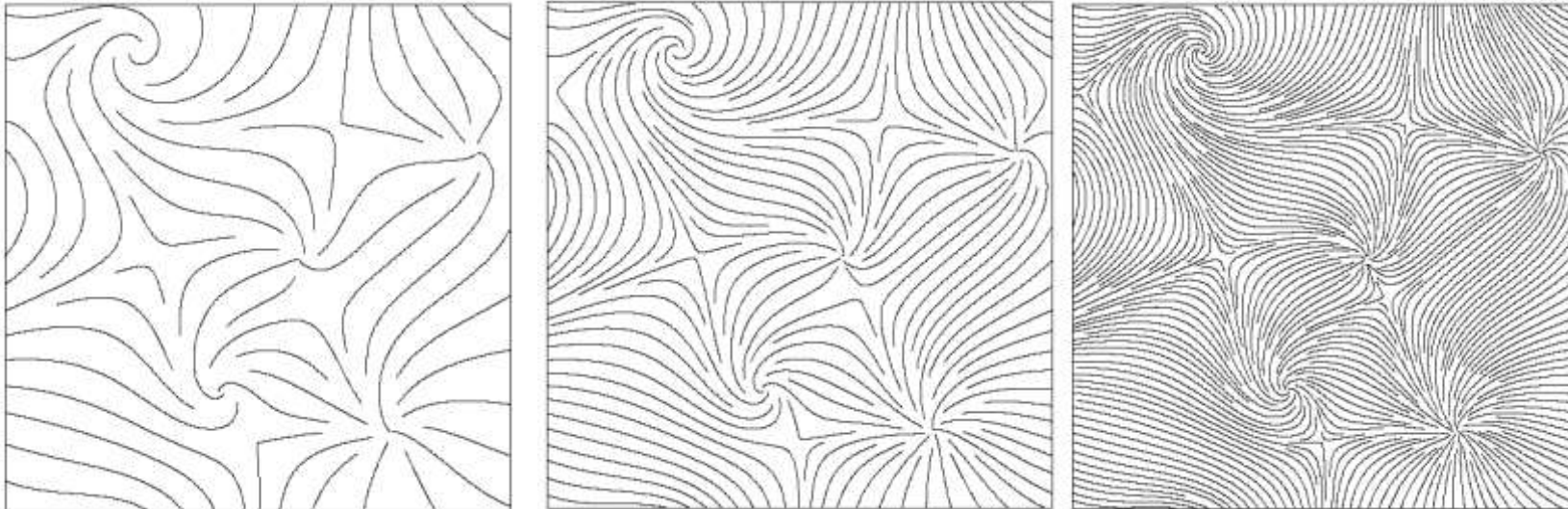


Illustration: Image-Guided Placement

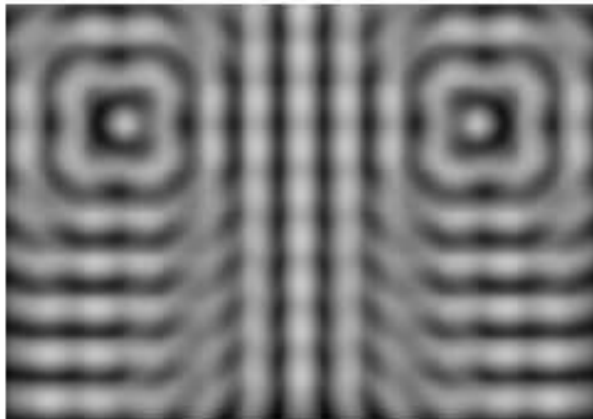
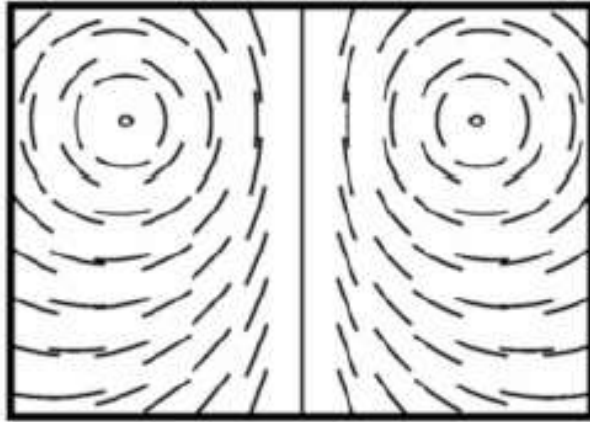


Figure 2: (a) Short streamlines with centers placed on a regular grid (top); (b) filtered version of same (bottom).

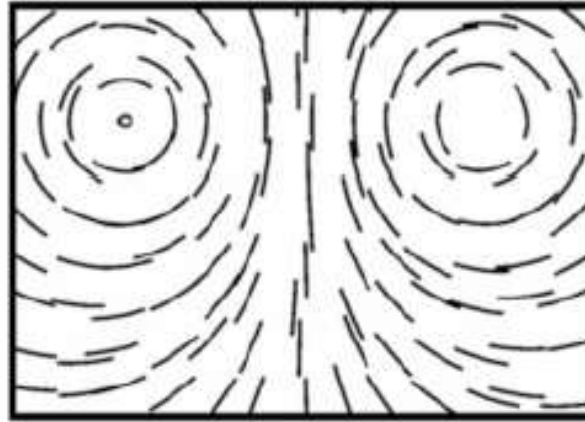


Figure 3: (a) Short streamlines with centers placed on a jittered grid (top); (b) filtered version showing bright and dark regions (bottom).

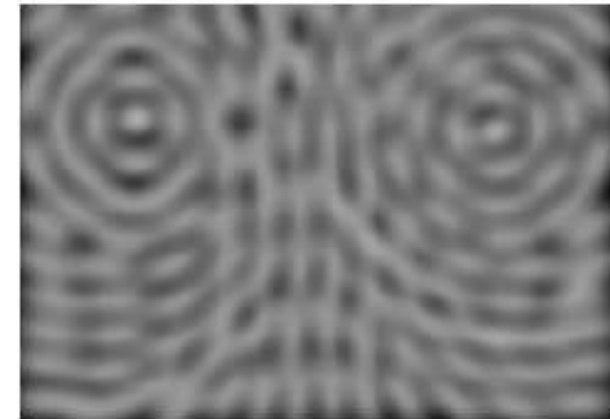
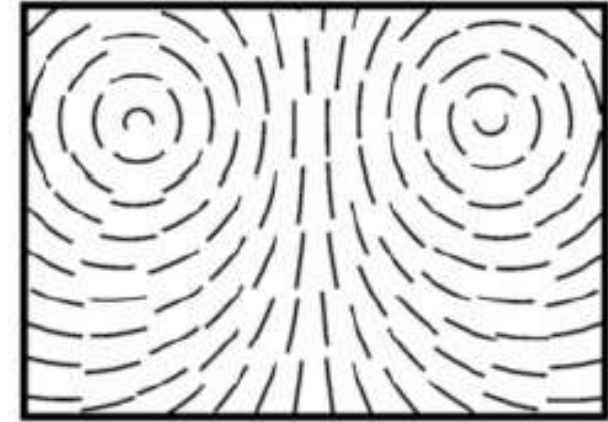


Figure 4: (a) Short streamlines placed by optimization (top); (b) filtered version showing fairly even gray value (bottom).

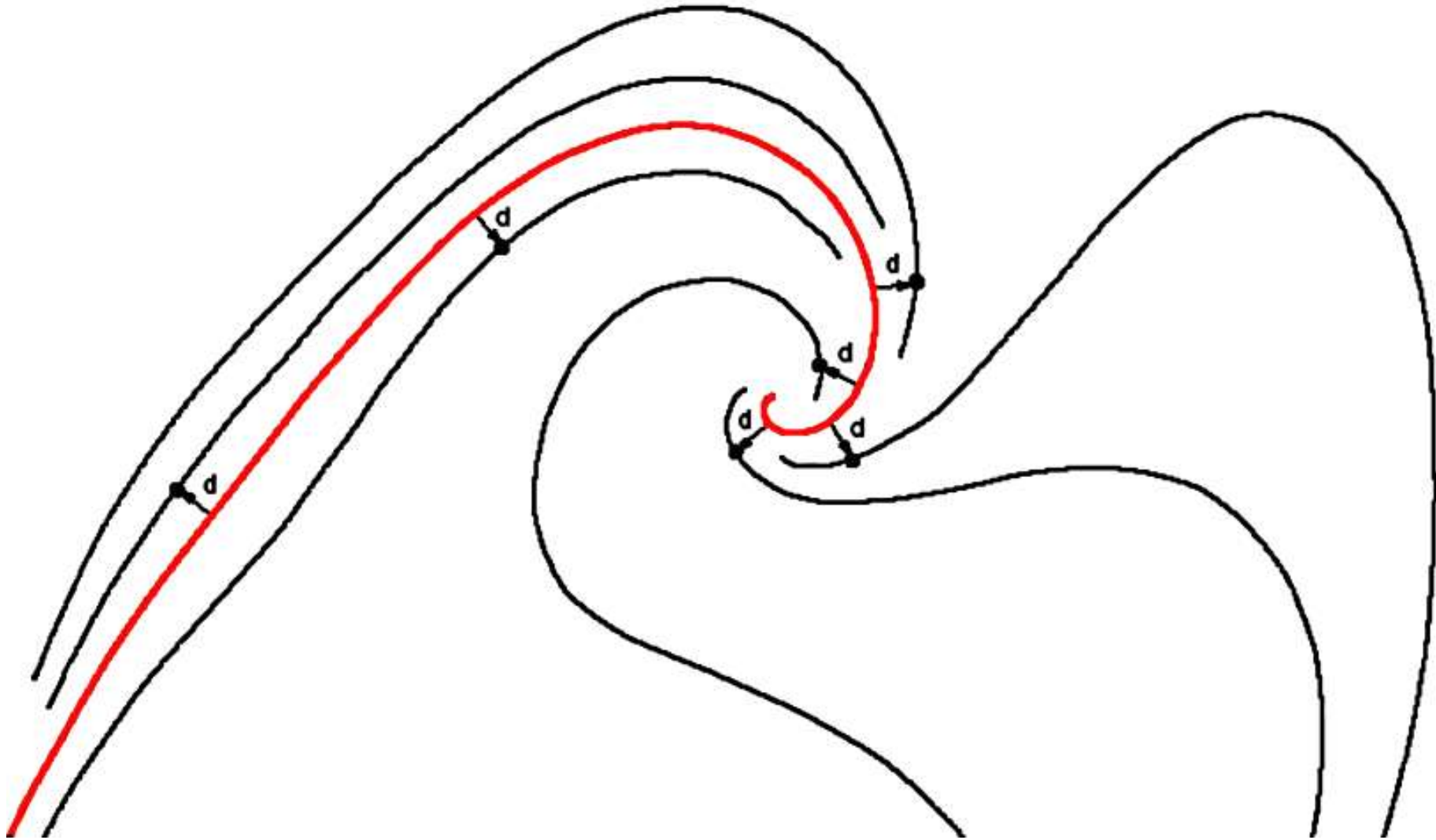
Uniform Glyph Seeding



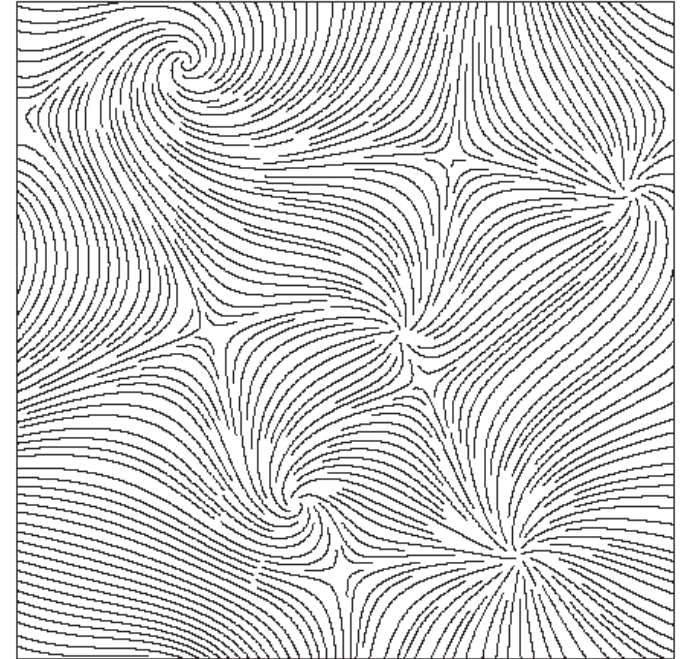
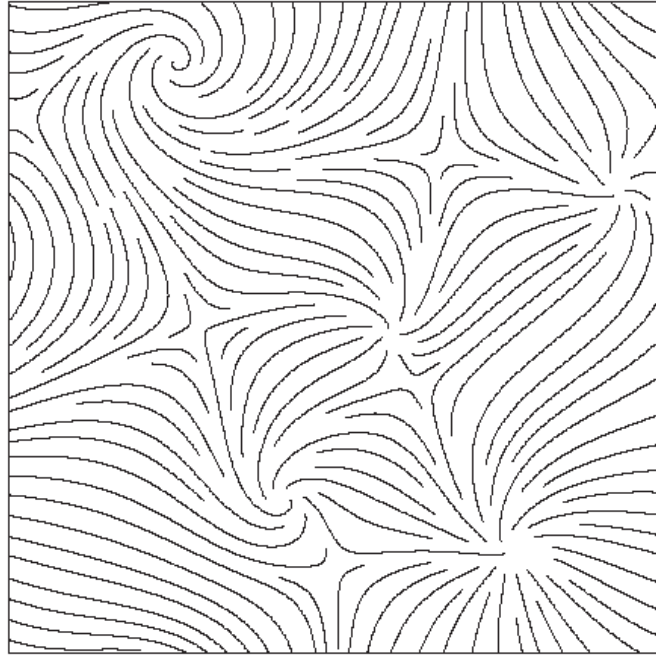
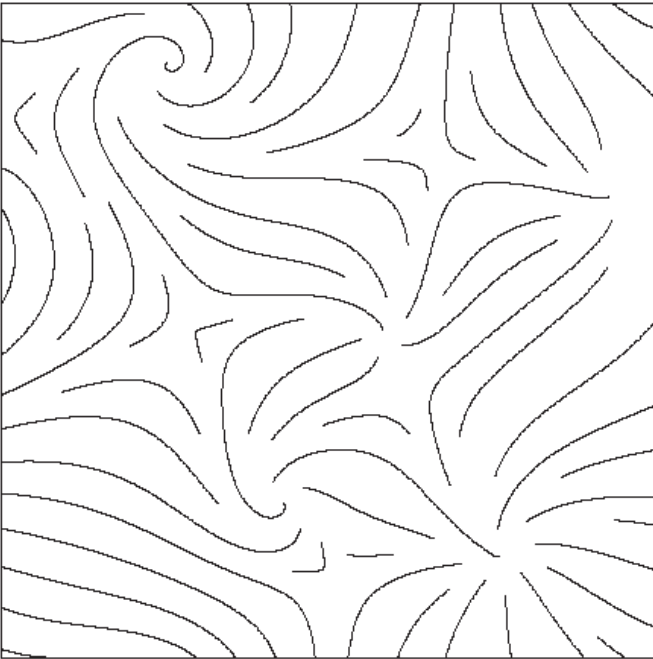
Evenly Spaced Stream Lines

- Greedy approach by [Jobart/Lefer 97]:
 - placement of new stream lines in the neighborhood of already present stream lines
 - Start integration of an arbitrary stream line in forward and backward direction until it
 - Leaves the domain
 - Ends in a critical point
 - Comes closer than d_{test} to an already drawn stream line
 - Next stream line: starts in a point with distance d_{sep} to first stream line
 - Ends when no further stream lines are possible
 - 2 steering parameters d_{sep} , d_{test} with $d_{\text{test}} < d_{\text{sep}}$

Illustration: Distance between Stream Lines



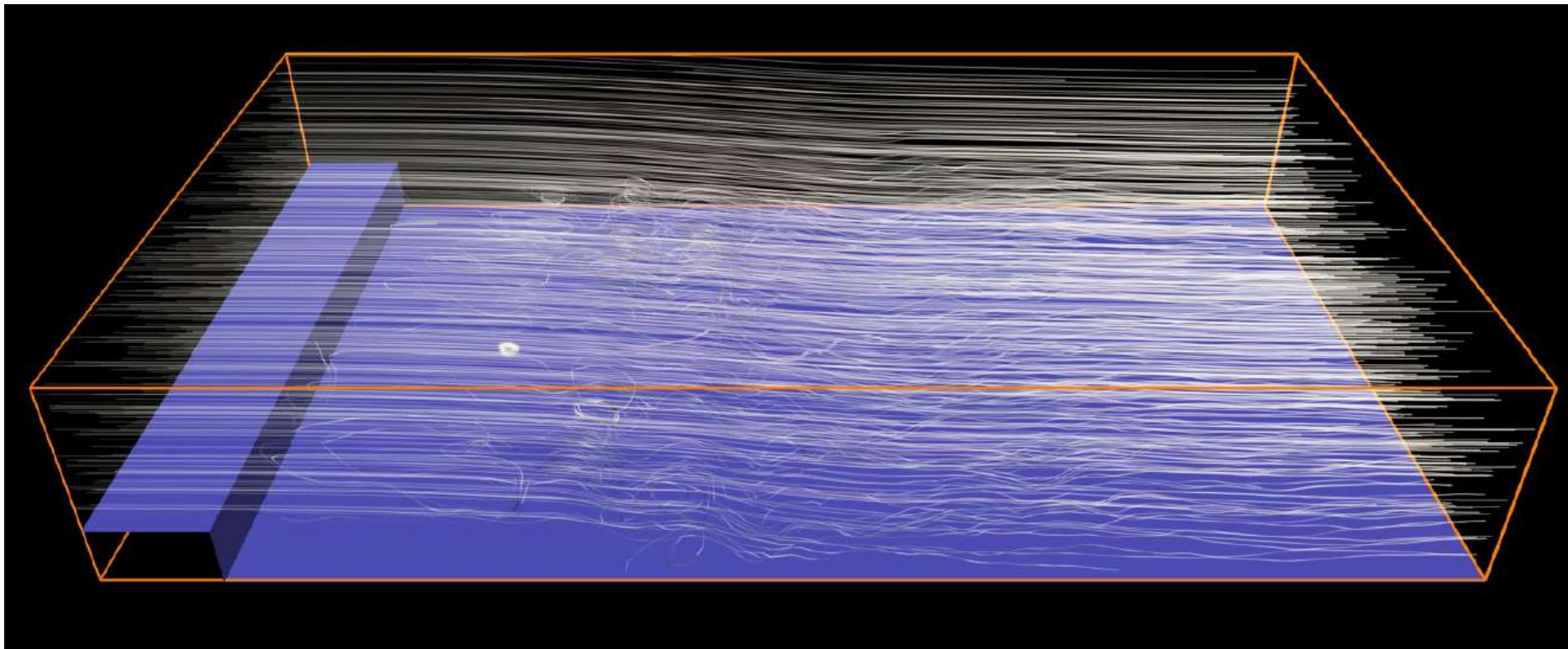
Result: Evenly Spaced Stream Lines



Results on the same vector field, with different distance threshold parameters.

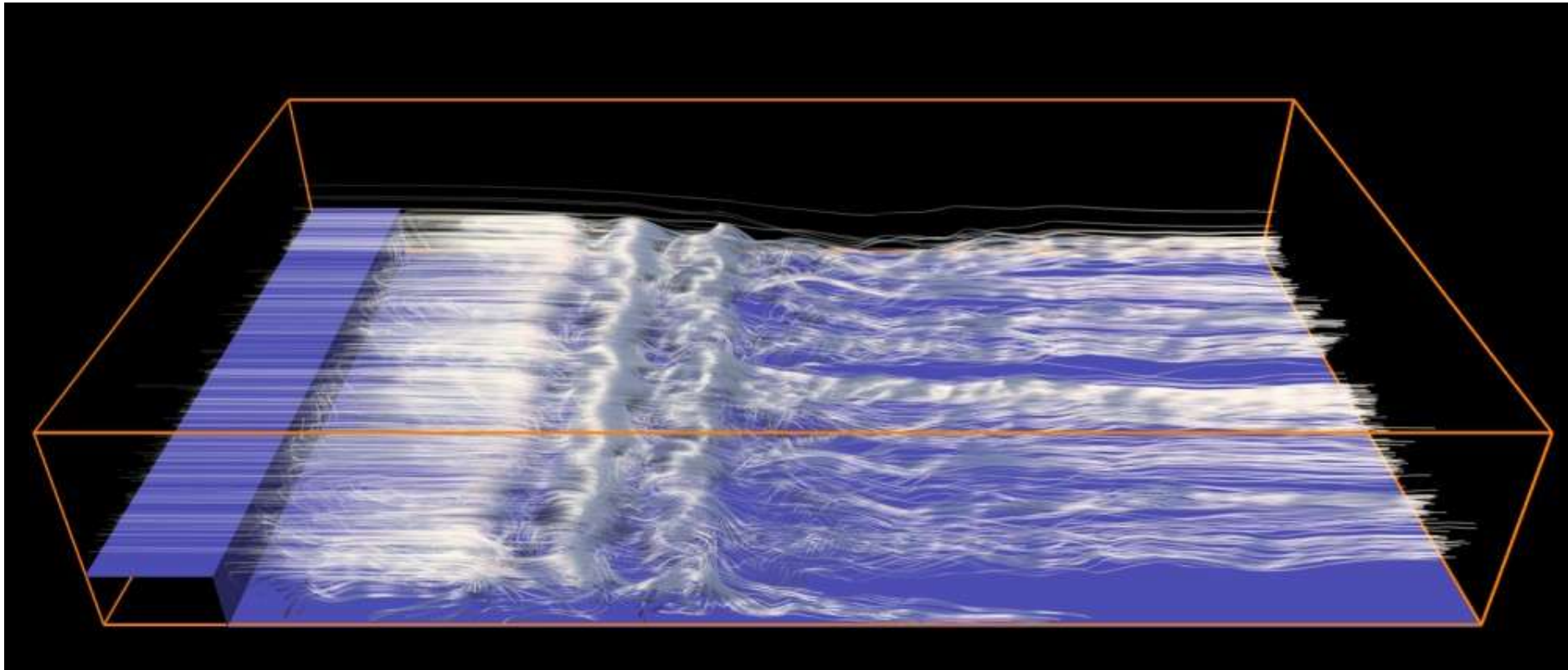
Stream Line Seeding in 3D

- **Stream line seeding 3D:**
 - evenly-spaced does not make sense
 - starting on uniform grid:



Idea: Focus on Regions of Interest

- In 3D, it can make sense to seed field lines mostly in regions where something “interesting” happens (such as turbulence)
 - How to detect those?



Background: Vortex Detection

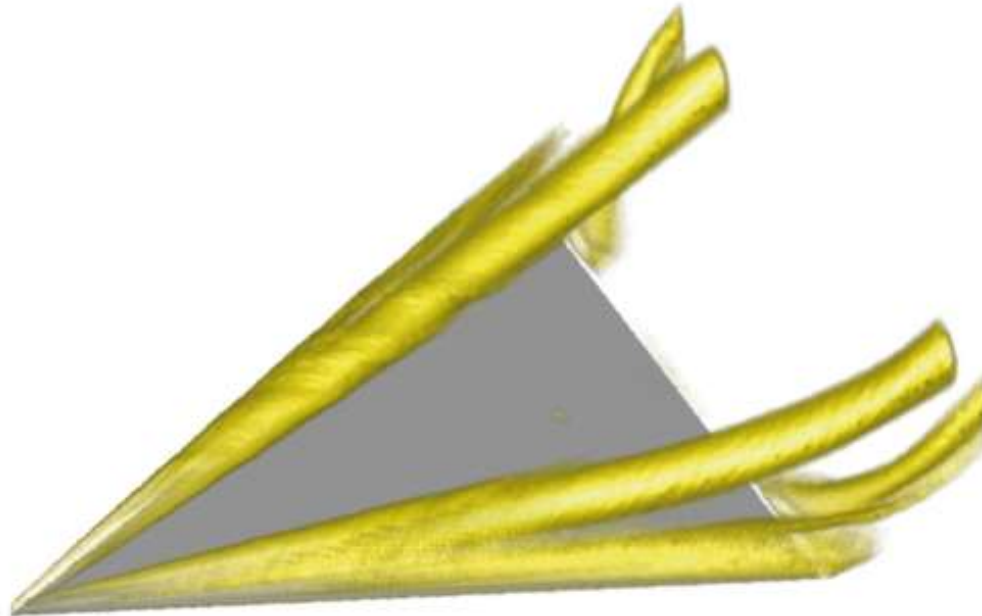
Intuitively, a vortex is a region of swirling flow behavior



Vorticity-based Vortex Detection

- **Idea:** Use curl to measure strong rotation
- Resulting region-based visualization:
 - Isosurface of vorticity, $\|\nabla \times \mathbf{v}\| \geq \theta_\omega$

Threshold on vorticity magnitude



Vortex-Strain Duality

- $\mathbf{J}_v$ can be decomposed into $\mathbf{J}_v = \mathbf{S} + \mathbf{\Omega}$

- a symmetric (shear) part: $\mathbf{S} = \frac{1}{2}(\mathbf{J}_v + \mathbf{J}_v^T)$

- an antisymmetric (rotational) part:

$$\mathbf{\Omega} = \frac{1}{2}(\mathbf{J}_v - \mathbf{J}_v^T) = \frac{1}{2} \begin{pmatrix} 0 & -\omega_3 & \omega_2 \\ \omega_3 & 0 & -\omega_1 \\ -\omega_2 & \omega_1 & 0 \end{pmatrix}$$

$$\text{Note: } \begin{pmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \end{pmatrix} = \mathbf{curl} \mathbf{v} = \begin{pmatrix} w_y - v_z \\ u_z - w_x \\ v_x - u_y \end{pmatrix}$$

- Vortex-Strain Duality:
 - $\|\mathbf{\Omega}\| \gg \|\mathbf{S}\|$: high vortical activity
 - $\|\mathbf{S}\| \gg \|\mathbf{\Omega}\|$: region of high strain

Q-based Vortex Detection

- Q-criterion

$$Q = \frac{1}{2} (\|\Omega\|^2 - \|S\|^2)$$

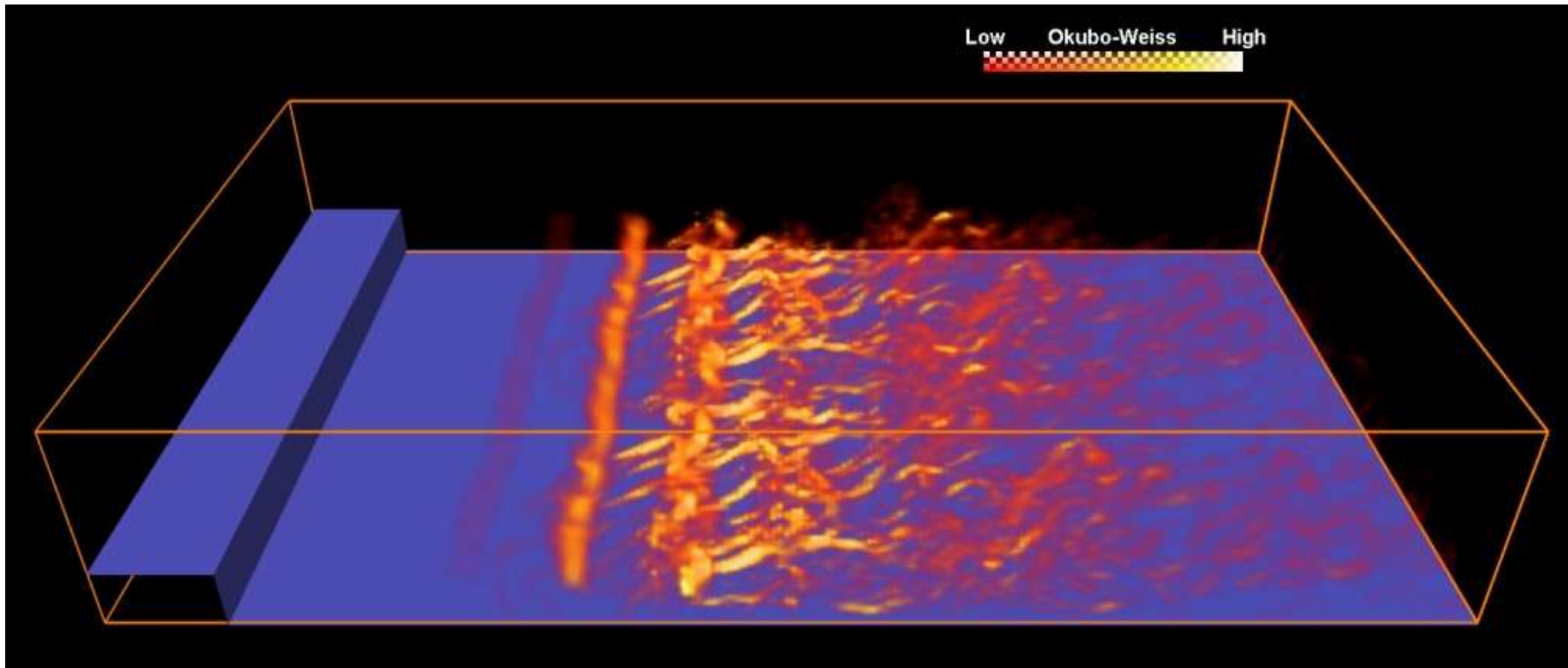
- where $\|\cdot\|$ denotes the Frobenius norm
- $Q > 0$: vorticity dominates the rate of strain

$Q > 0$



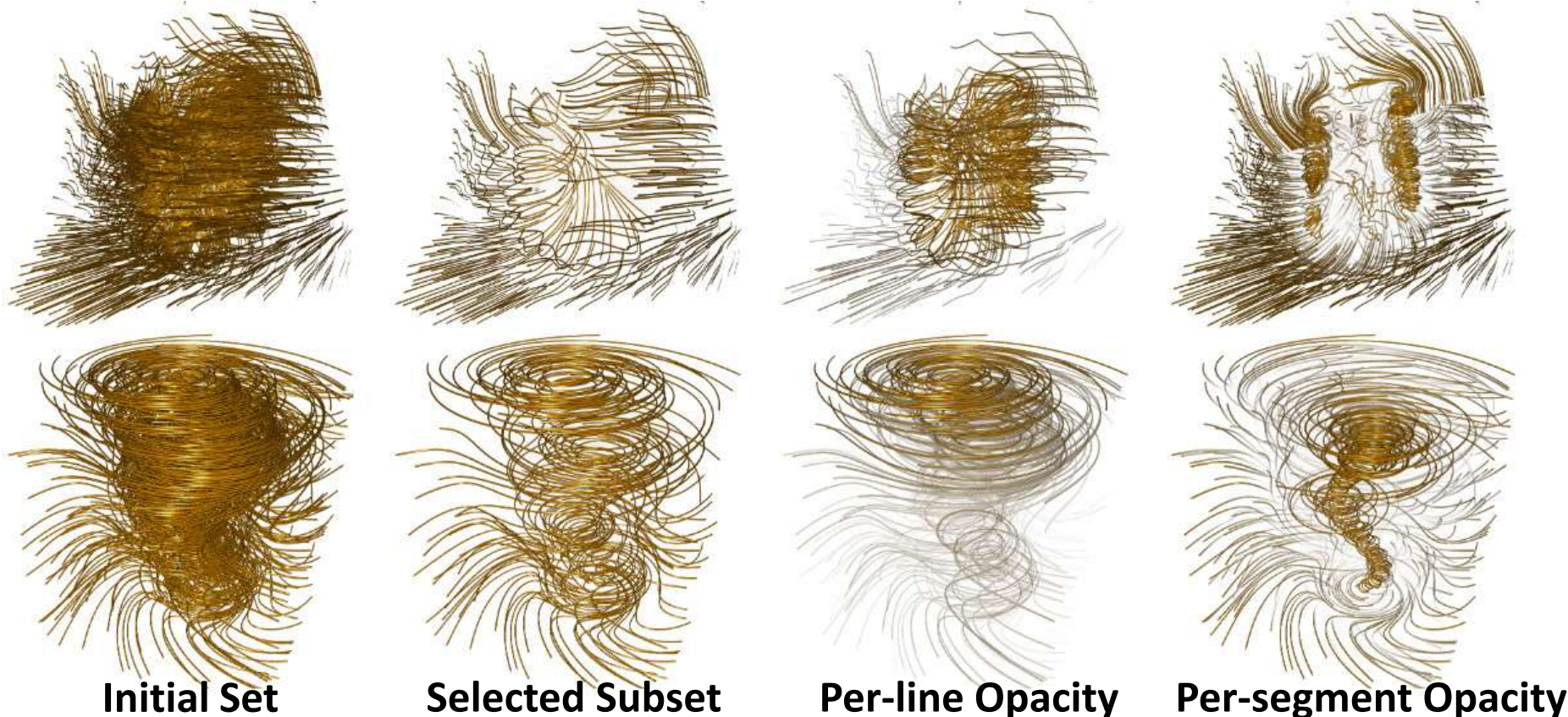
Use for Seeding Characteristic Curves in 3D

- In 3D, it can make sense to seed field lines mostly in regions of turbulence



Streamline Selection

- *Alternative strategy:* Integrate a large set of initial streamlines,
 - **select a subset** that shows the main features with minimal occlusion
 - **optimize opacity** per line or line segment



Example: Opacity Optimization

- For line segments i, j with adjacency a_{ij} , define
 - importance scores $g_i \in [0,1]$ (e.g., based on length, curvature)
 - occlusion indicators $h_{ij} \in [0,1]$ for i occluding jand user-specified weights p, q, r, s
to compute opacities α_i that minimize the overall energy

$$p \sum (\alpha_i - 1)^2$$

Avoid empty solutions

$$+q \sum \sum (\alpha_i (1 - g_i)^\lambda h_{ij} g_j)^2$$

Hide unimportant segments in front of important ones

$$+r \sum \sum (\alpha_i (1 - g_i)^\lambda h_{ji} g_j)^2$$

Hide unimportant segments behind important ones

$$+s \sum \sum a_{ij} (\alpha_i - \alpha_j)^2$$

Encourage smooth opacity variations along the curves

Summary: Stream Line Seeding / Selection

- In 2D, different strategies exist to achieve a uniform distribution of streamlines
 - Optimization- and image based
 - Greedy distance based
- In 3D, uniform distributions are less effective
 - Seed close to “interesting” features
 - Basic approaches to vortex detection: curl, Q
 - Seed a large initial set, select a relevant subset or fade out less important curves

Section 11.6:

Stream Line Rendering

Rendering of 3D Stream Lines

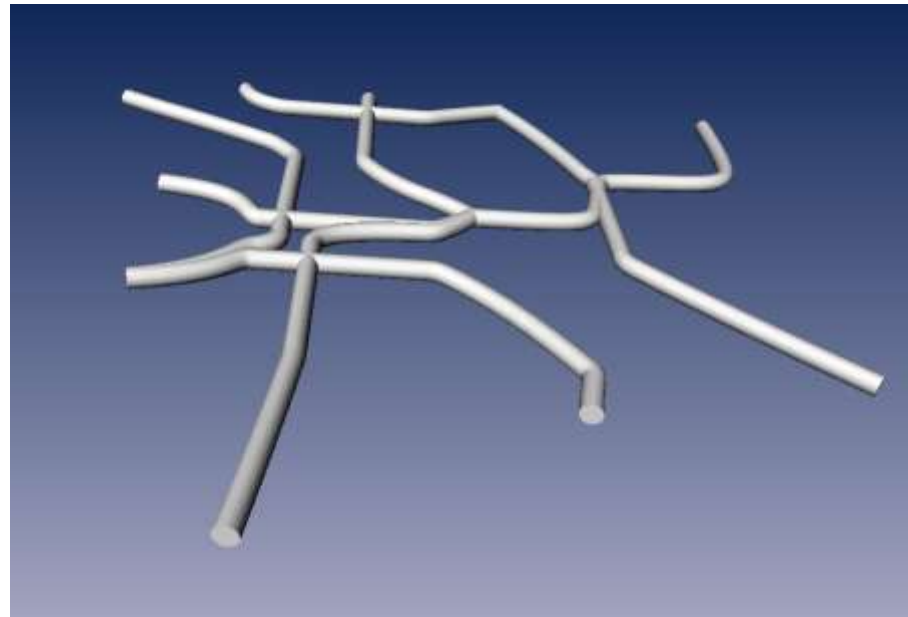
- **Getting a 3D impression of 3D stream lines:**
 - Important for spatial perception
 - Required to gain insight into the data



Stream Tubes

Classic approach to achieve 3D impression:

- Render stream tubes instead of stream lines
- Circular cross-section
- Requires a large number of triangles!



Stylized Line Primitives

More recent approach: Use programmable shaders

- Send a single triangle strip to the GPU
- Orient the strip on the GPU such that it lies in the viewing plane
- Use fragment shader to compute cylinder-like shading & depth

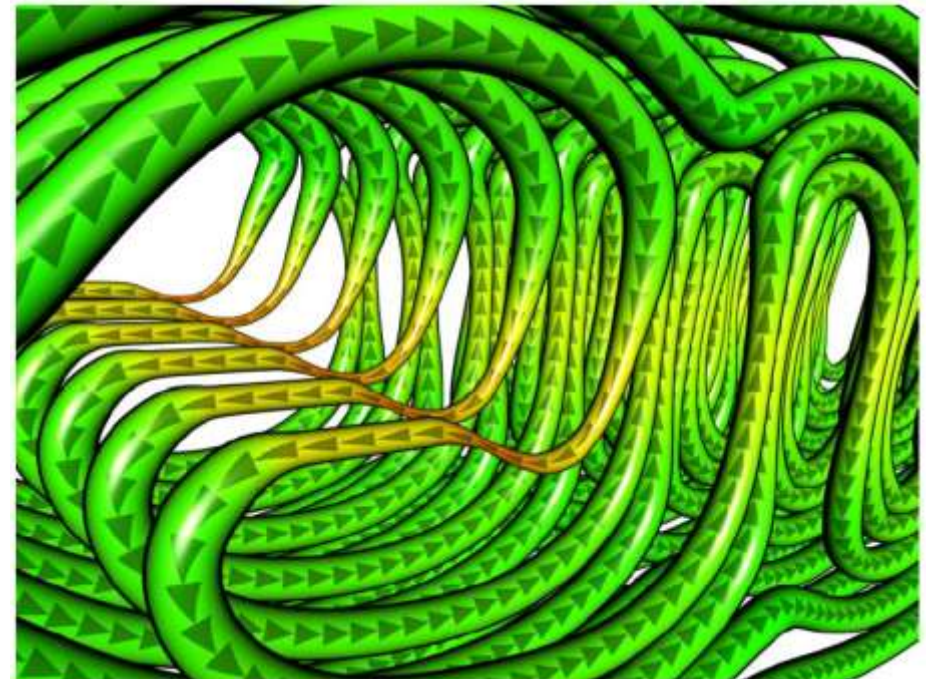


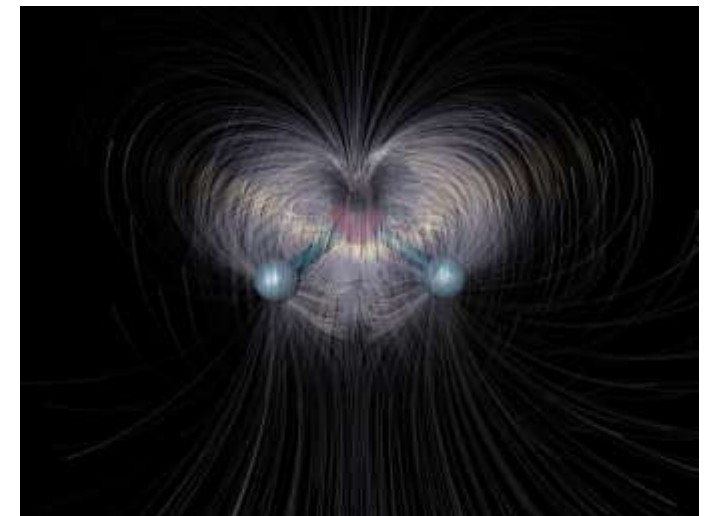
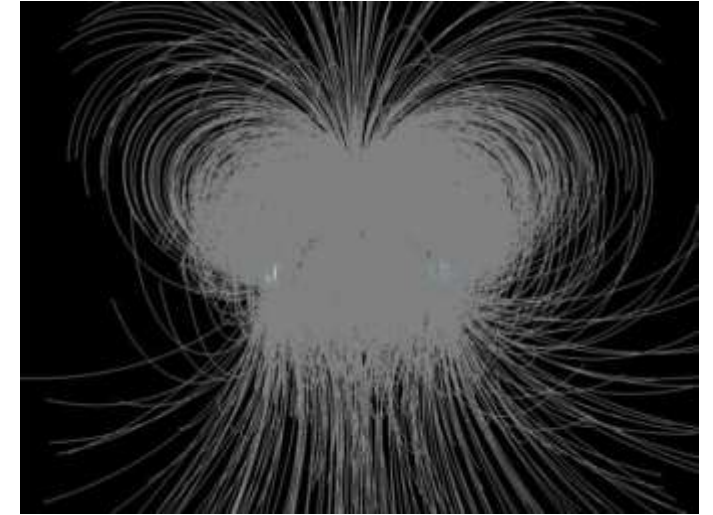
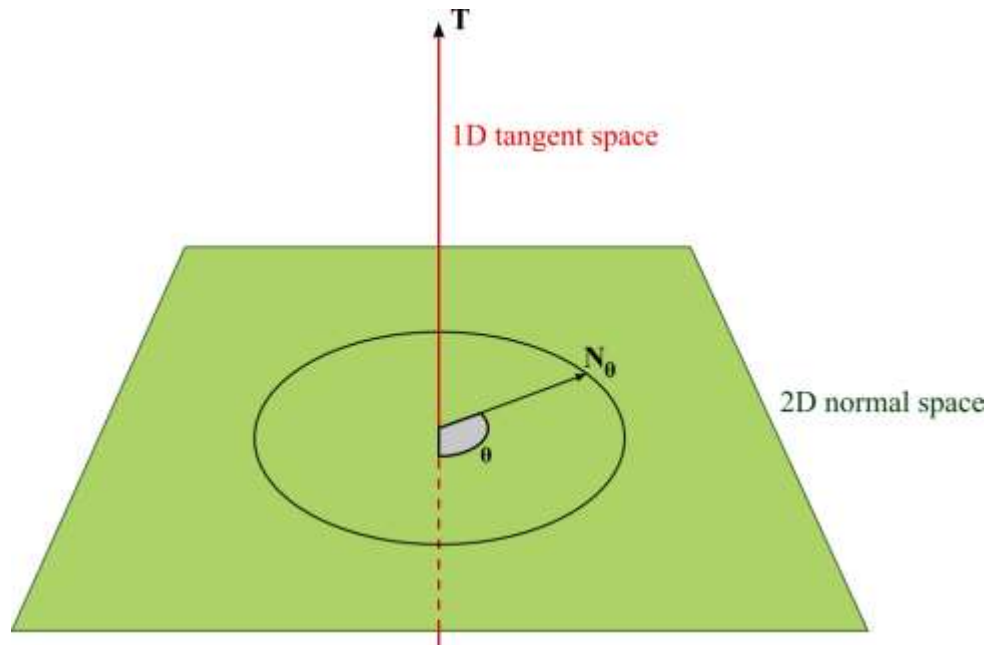
Image from
Visualization with stylized line primitives
Carsten Stoll, Stefan Gumhold
IEEE Visualization 2005

Illuminated Streamlines

Alternative: Shade 1D curves directly

Problem: normal vector required for shading, but not uniquely defined

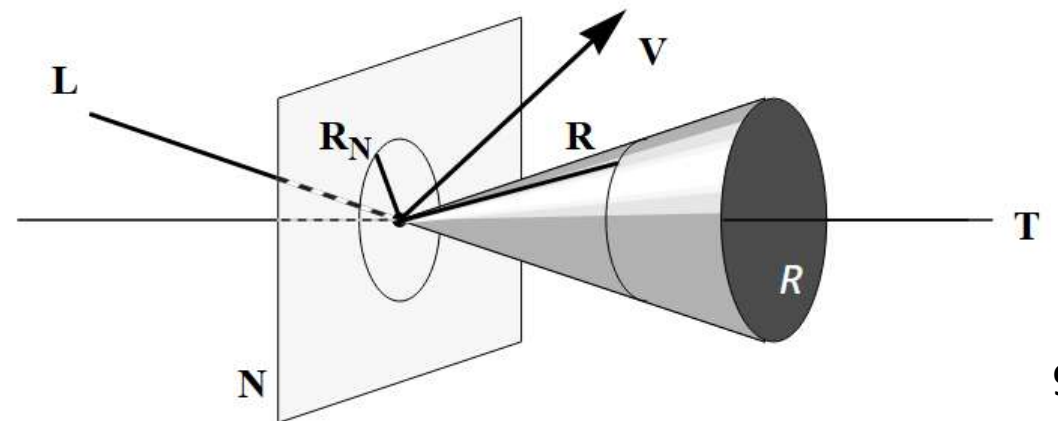
– instead: 2D normal space



Shading Curves

Shading model by [Zoeckler et al. 1996], based on [Banks 1994]:

- **Diffuse term:** Light energy arriving in a small neighborhood of a point on the curve depends on $\sin(\alpha)$, where α is the angle between light direction L and tangent direction T
 - Amounts to picking the unique normal vector N in the L - T plane
 - Resulting images tend to be overly bright and low contrast, compensated by introducing an exponent: $(L \cdot N)^p$ (e.g., with $p = 2$)
- **Specular term:** Reflected light forms a cone, pick reflected light vector R that corresponds to the same normal N as used above



Computing Stream Line Illumination

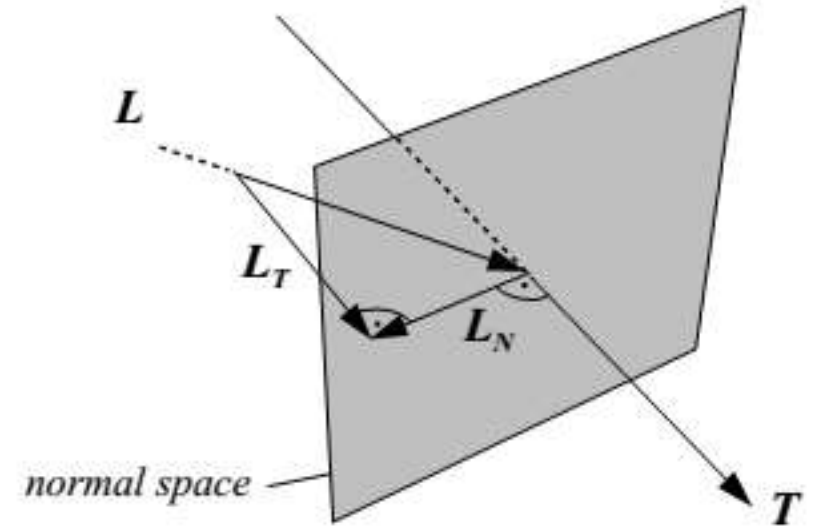
Computation in [Zoeckler et al. 1996]:

- Thanks to Pythagoras, we do not have to compute N
- Computation of diffuse term:

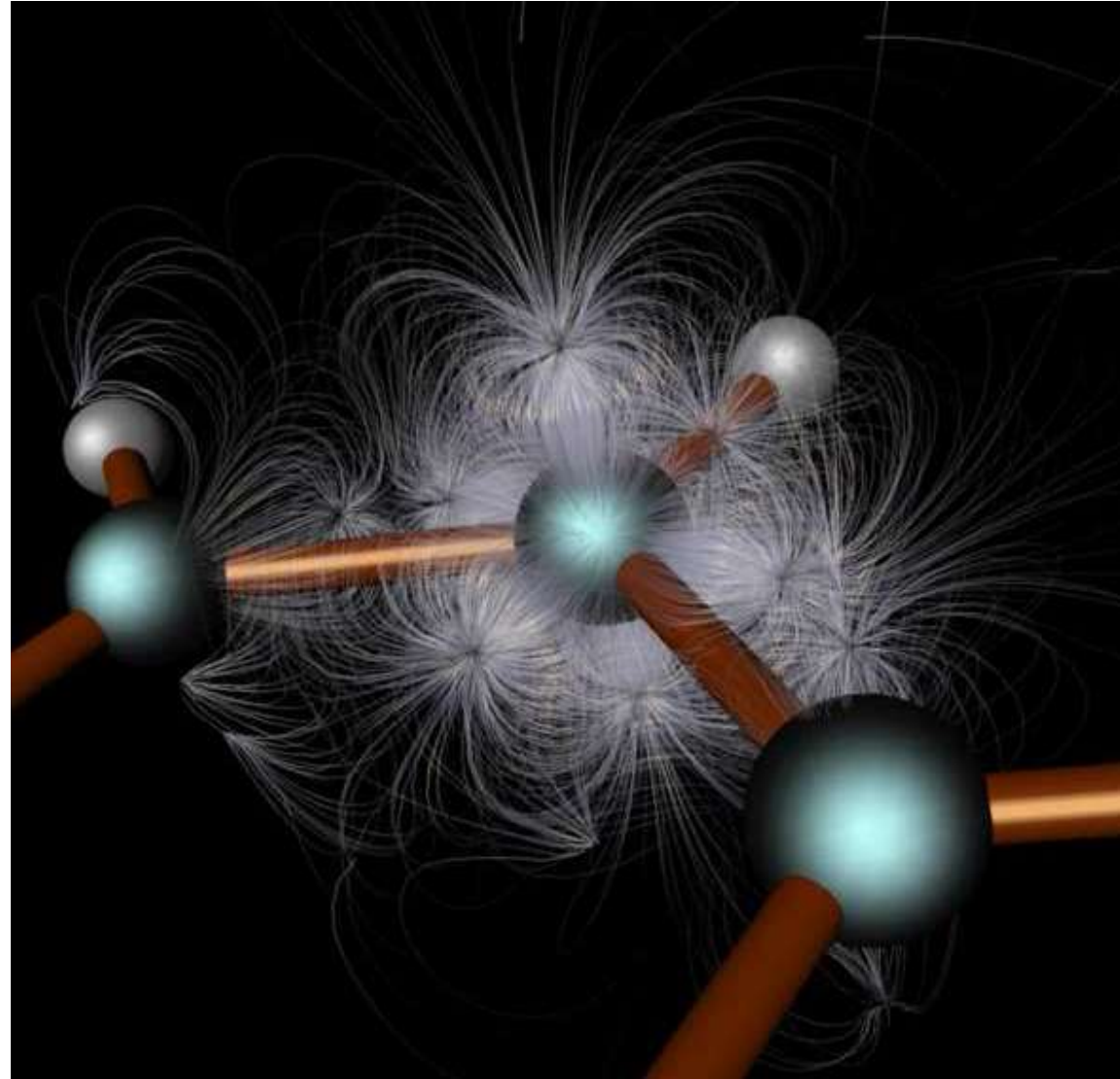
$$L \cdot N = \|L_N\| = \sqrt{1 - (L \cdot T)^2}$$

- In specular term, $R_N = L_N, R_T = -L_T$

$$\begin{aligned} V \cdot R &= V \cdot (L_N - L_T) = V \cdot ((L \cdot N)N - (L \cdot T)T) \\ &= (L \cdot N)(V \cdot N) - (L \cdot T)(V \cdot T) \\ &= \sqrt{1 - (L \cdot T)^2} \sqrt{1 - (V \cdot T)^2} - (L \cdot T)(V \cdot T) \end{aligned}$$



Example: Illuminated Stream Lines



Summary: Stream Line Rendering

Shading required to convey shape of three-dimensional curves

1. Rendering as cylindric tubes
 - Using proxy geometry to speed up rendering
2. Illuminating streamlines directly
 - Define normal vector to lie within L-T plane
 - Pythagorean theorem for fast implementation

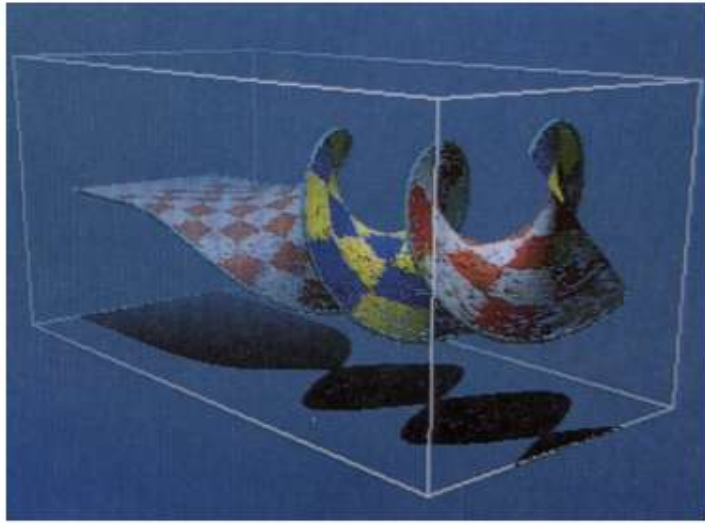
Section 11.7: Integral Surfaces

Integral Surfaces in Time-Dependent Flows

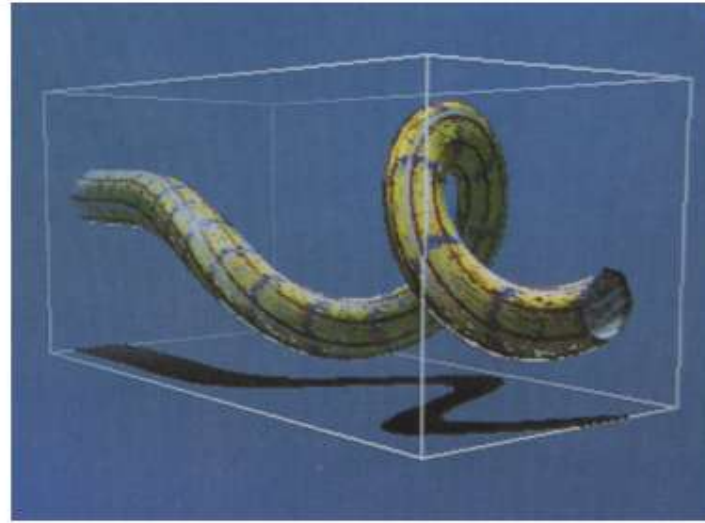
In analogy to characteristic curves, we can obtain characteristic surfaces by seeding accordingly:

- **Stream surfaces**
 - Trace points on a seeding curve through a vector field at a fixed time
- **Path surfaces**
 - Trace points on a seeding curve through an unsteady vector field
- **Streak surfaces**
 - Continuously emit particles along a seeding curve, take a snapshot at a particular time
- **Time surfaces** (surfaces in motion)
 - Emit particles along a surface once, take a snapshot at a given time

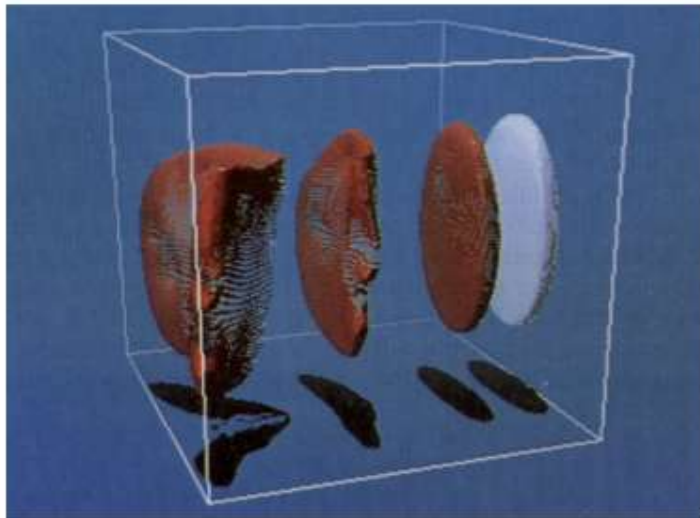
Examples: Stream Surfaces and Objects



a)



b)



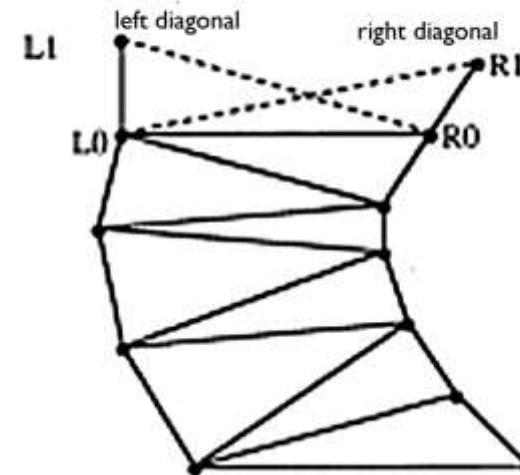
c)

- a) *stream surface*
- b) *stream tube*
- c) *stream object in motion*

Stream Surface Integration

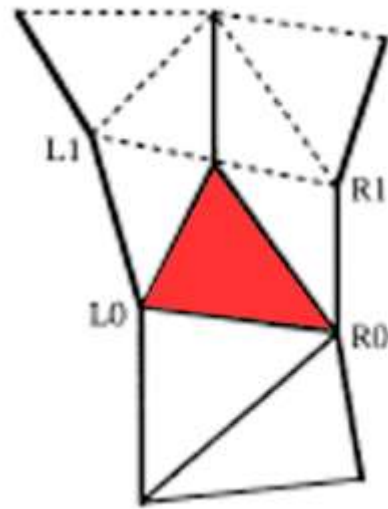
Basic Algorithm for stream surface integration:

- Seeding structure: polygonal curve
- apply one step of stream line integration for each vertex of the polygon
 - creates new polygonal curves
- triangulate strip between the two polygonal curves
 - build ribbon in a greedy manner
 - in each step, pick shorter diagonal



Stream Surfaces: Adapting Vertices

Inserting or removing vertices: if vertices on front line are too close or too far away from next



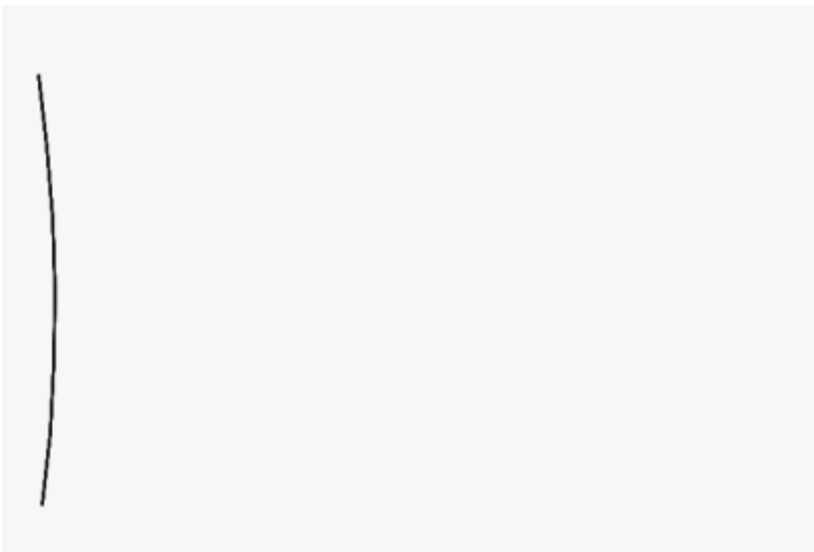
Strategies for inserting new vertices:

- between two current vertices
- between two previous vertices + integrate (more accurate)
- between corresponding vertices at seeding structure, integrate them (most accurate but slowest)

steady vector field

$$\mathbf{v}(\mathbf{x})$$

stream surface



General stream surfaces
[Hultquist, Vis 1992]

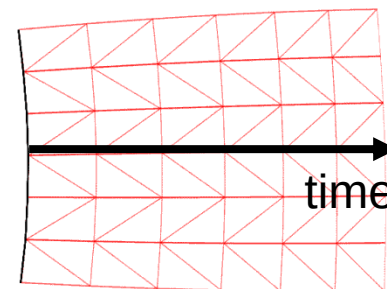
Topological considerations
[Stalling, Phd thesis, 1998]

High accuracy
[Garth et al., Vis 2008]

unsteady vector field

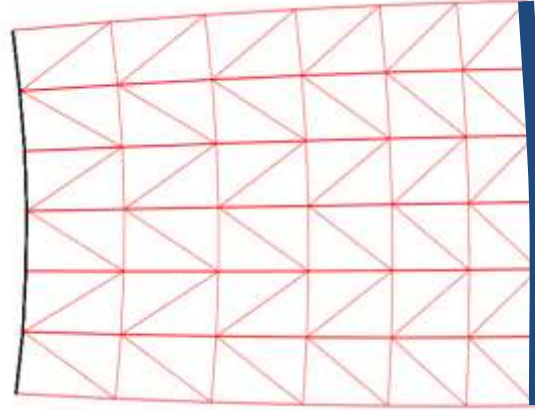
$$\mathbf{v}(\mathbf{x}, t)$$

path surface



streak surface

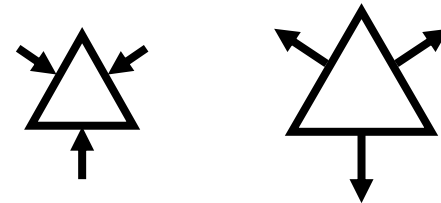
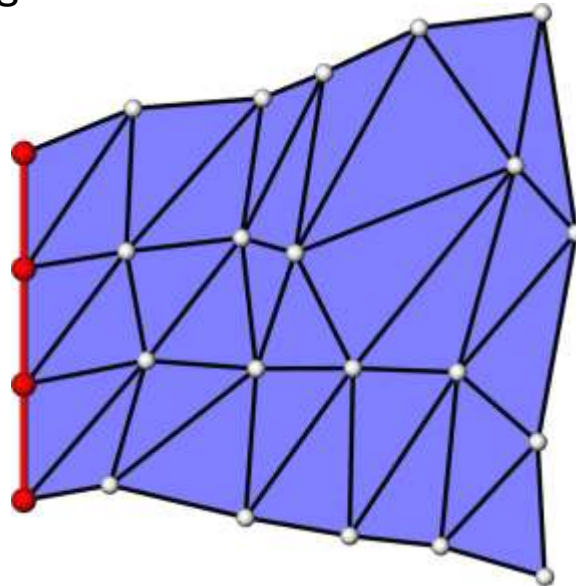




Adaptive insertion/removal of points at **front line**

stream & path surfaces

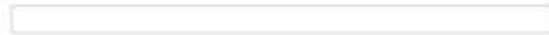
streak surfaces



Complete remeshing in every time step is necessary to stay within resolution constraints



Time



streak line

streak surface



Smoke Surfaces

- Idea by [von Funck et al. 2008]: Render streak surfaces semi-transparently
 - Adaptive transparency provides visual impression of smoke
 - Avoids having to re-mesh during integration
 - Interactive performance



Artistic photographs by W. Brennan

$$\alpha = 1 - e^{-\int_{t_0}^{t_1} \kappa(t) dt}$$

extinction coefficient κ

κ const. inside prism

$$\alpha = 1 - e^{-(t_1 - t_0)\kappa}$$

$$(t_1 - t_0) = \frac{h}{\cos \gamma}$$

with $\gamma = \angle(\mathbf{r}, \mathbf{n})$

$$\alpha(h) = 1 - e^{-\frac{h\kappa}{\cos \gamma}}$$

Taylor expansion

$$\alpha(h) = \alpha(0) + h \frac{d\alpha}{dh}(0)$$

$$\alpha(h) = \frac{h\kappa}{\cos \gamma}$$

particle model

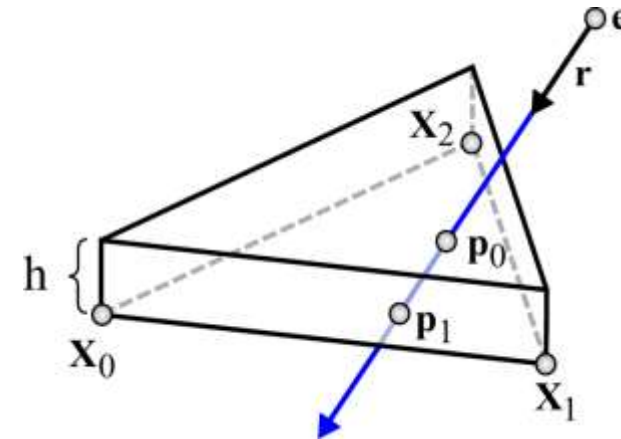
$$\kappa = c \frac{n_p}{\text{area}(\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2)}$$

of particles n_p

$$\alpha_{\text{density}} = \frac{k}{\text{area}(\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2) \cos \gamma}$$

with $k = h c n_p$

The optical model of smoke

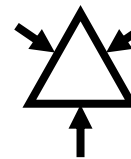
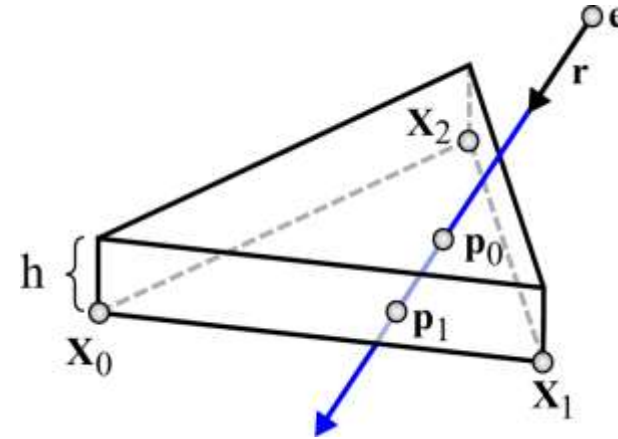


triangle/prism $(\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2)$

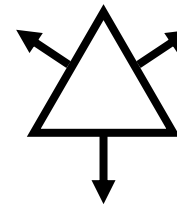
height h

viewing ray $\mathbf{p}(t) = \mathbf{e} + t\mathbf{r}$

The optical model of smoke



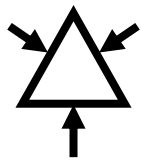
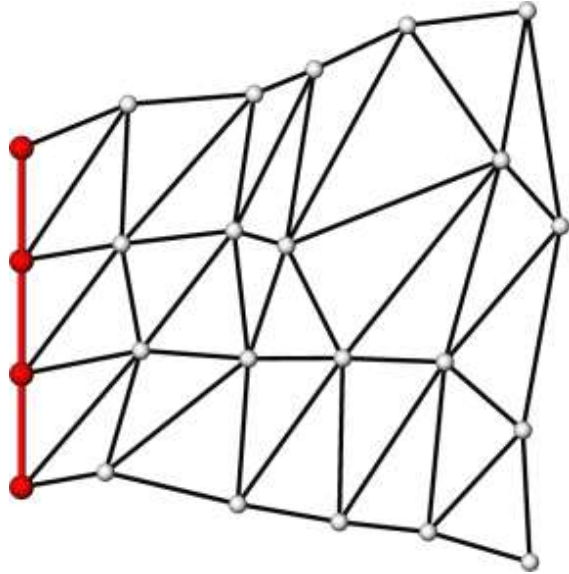
more opaque



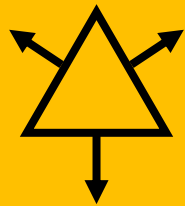
less opaque
less visible

$$\alpha_{density} = \frac{k}{\text{area}(x_0, x_1, x_2) \cos \gamma}$$

Streak surface

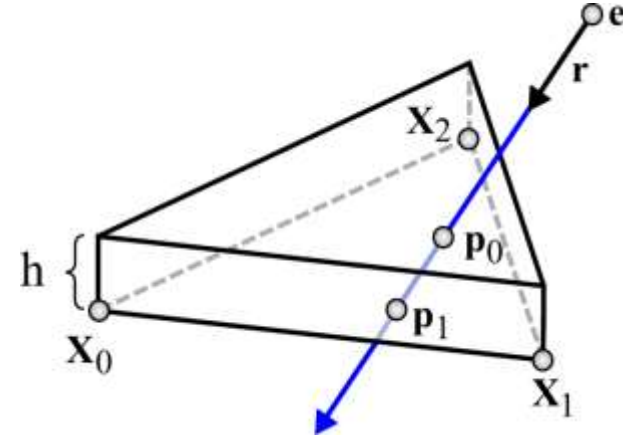


no real problem

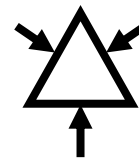


too coarse

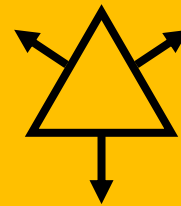
The optical model of smoke



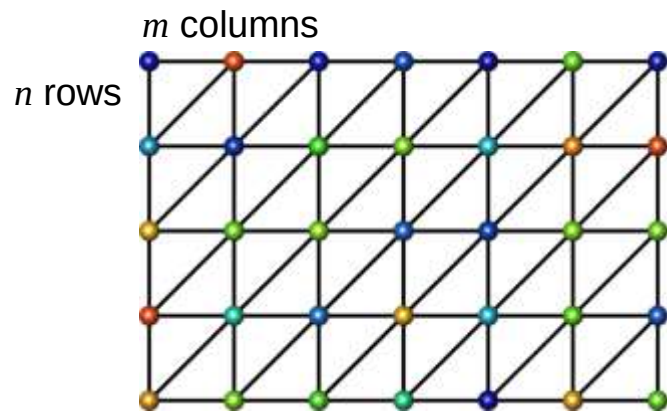
$$\alpha_{density} = \frac{k}{\text{area}(\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2) \cos \gamma}$$



more opaque



less opaque
less visible



fixed size

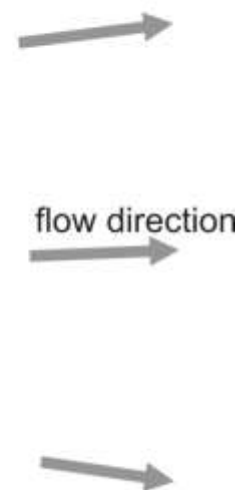
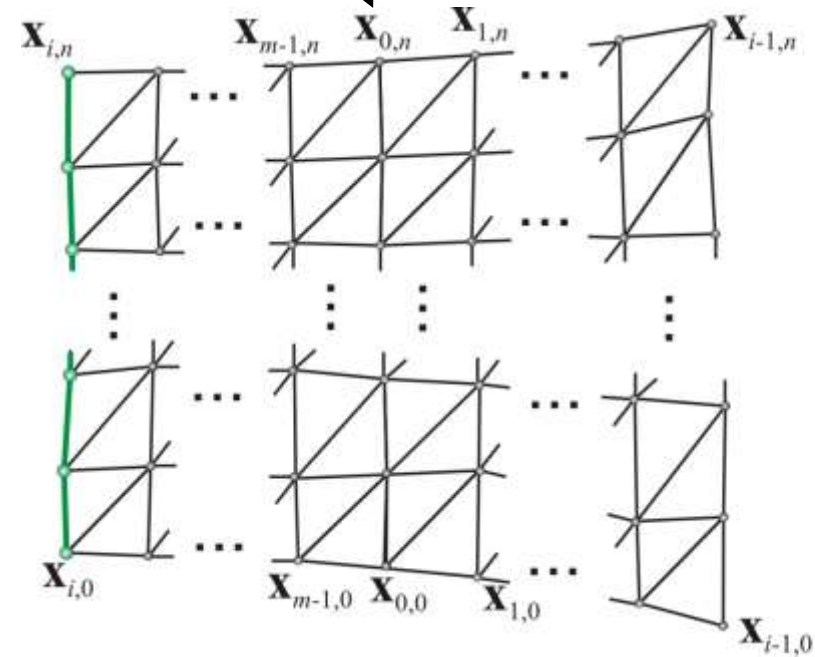
vertex array

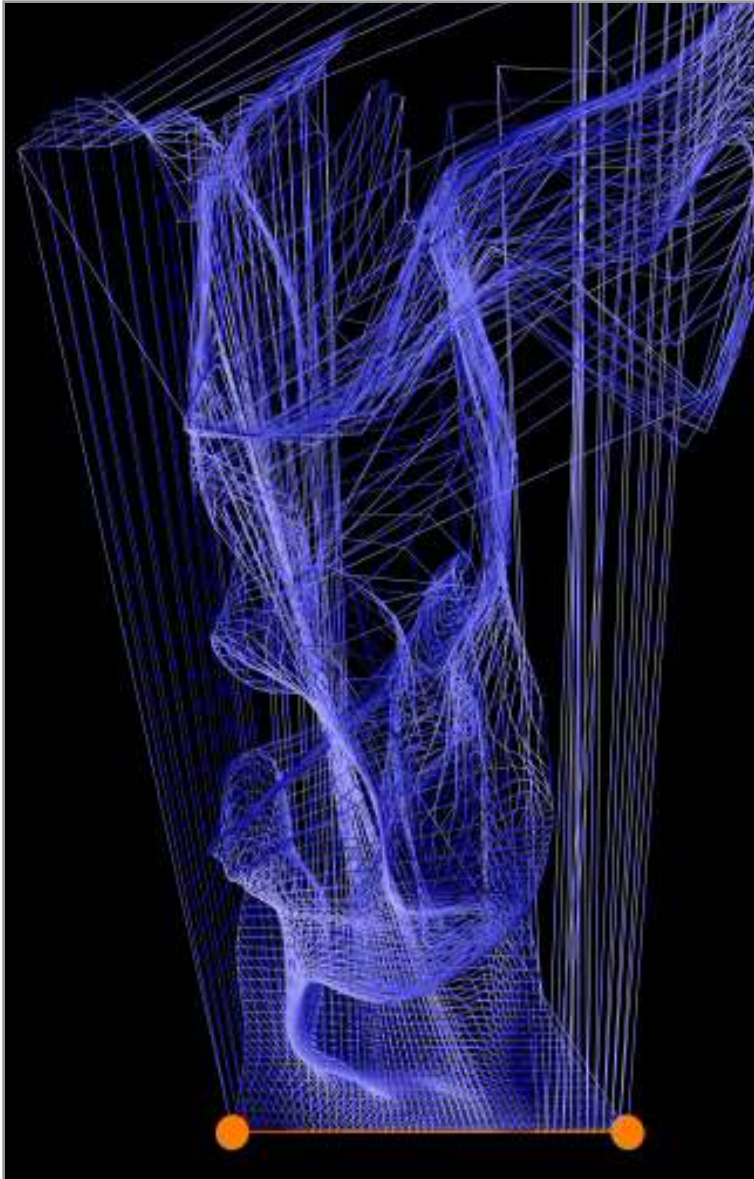
opacity

fixed content

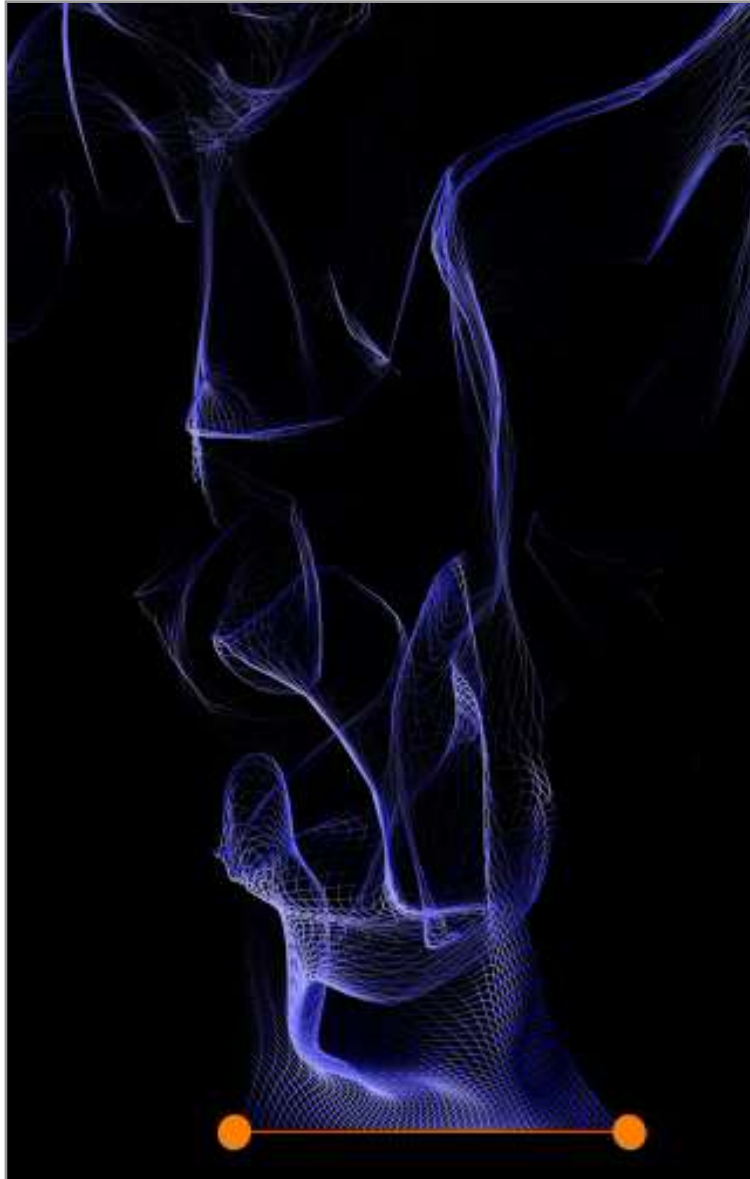
index array
(connectivity)

colors





opaque streak surface



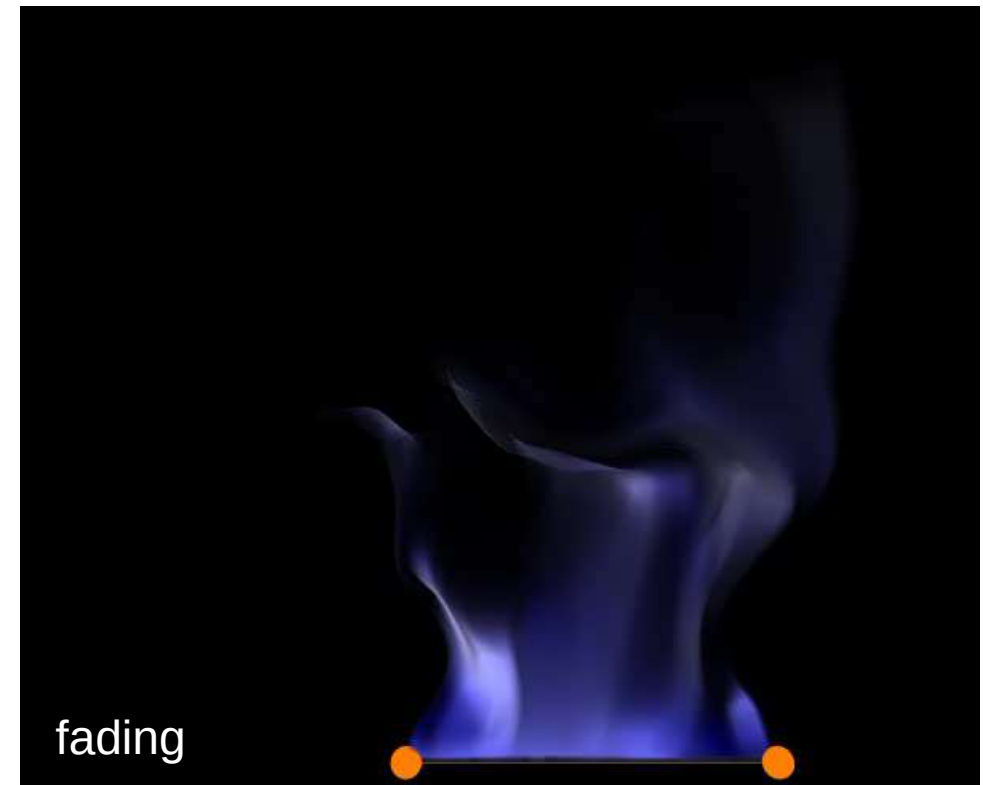
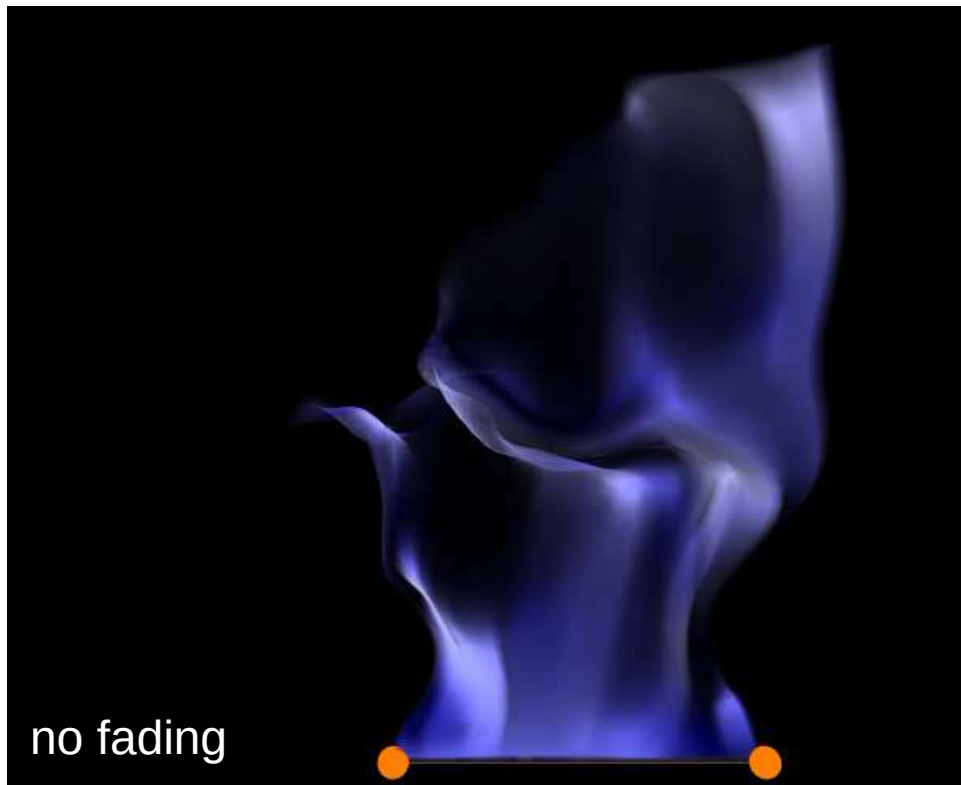
semi-transparent



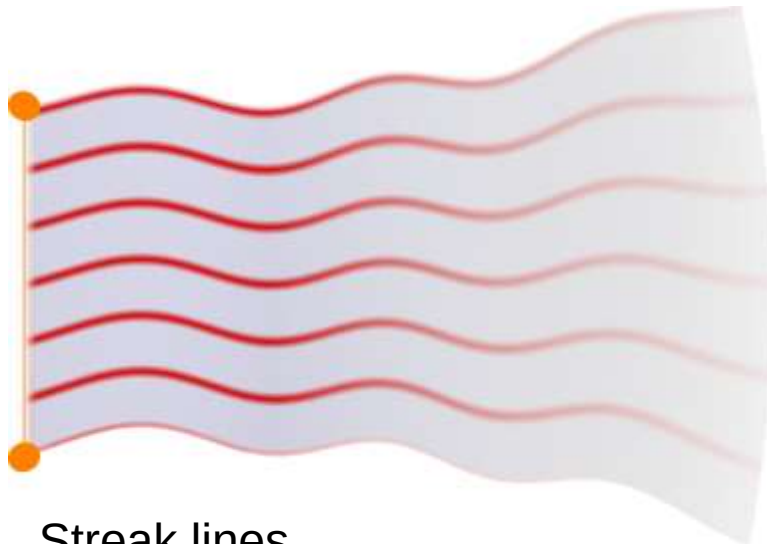
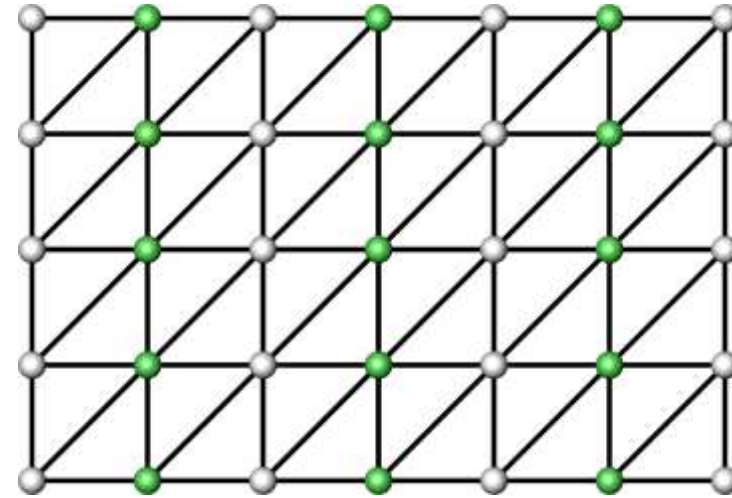
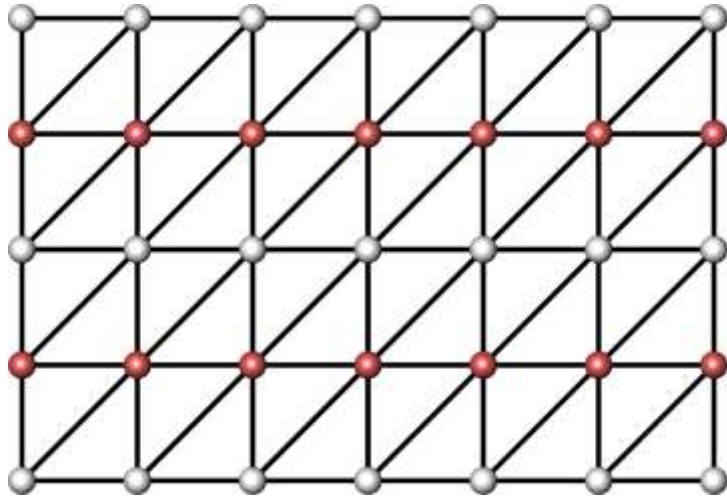
smoke surface

Optional Fading Over Time

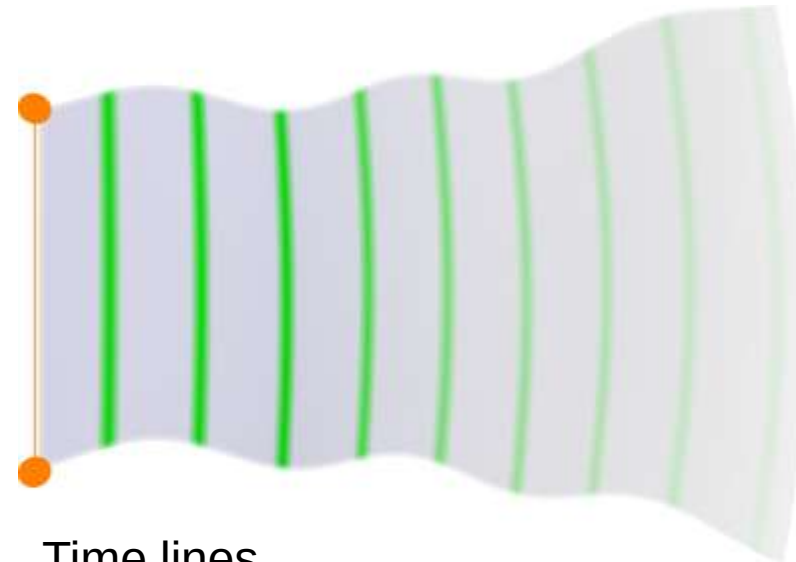
- Fade out smoke with time: $\alpha_{fade} = 1 - \frac{t}{t_{max}}$
- Optionally, can also fade out poorly shaped triangles or ones in high-curvature regions



Streak and Time Lines

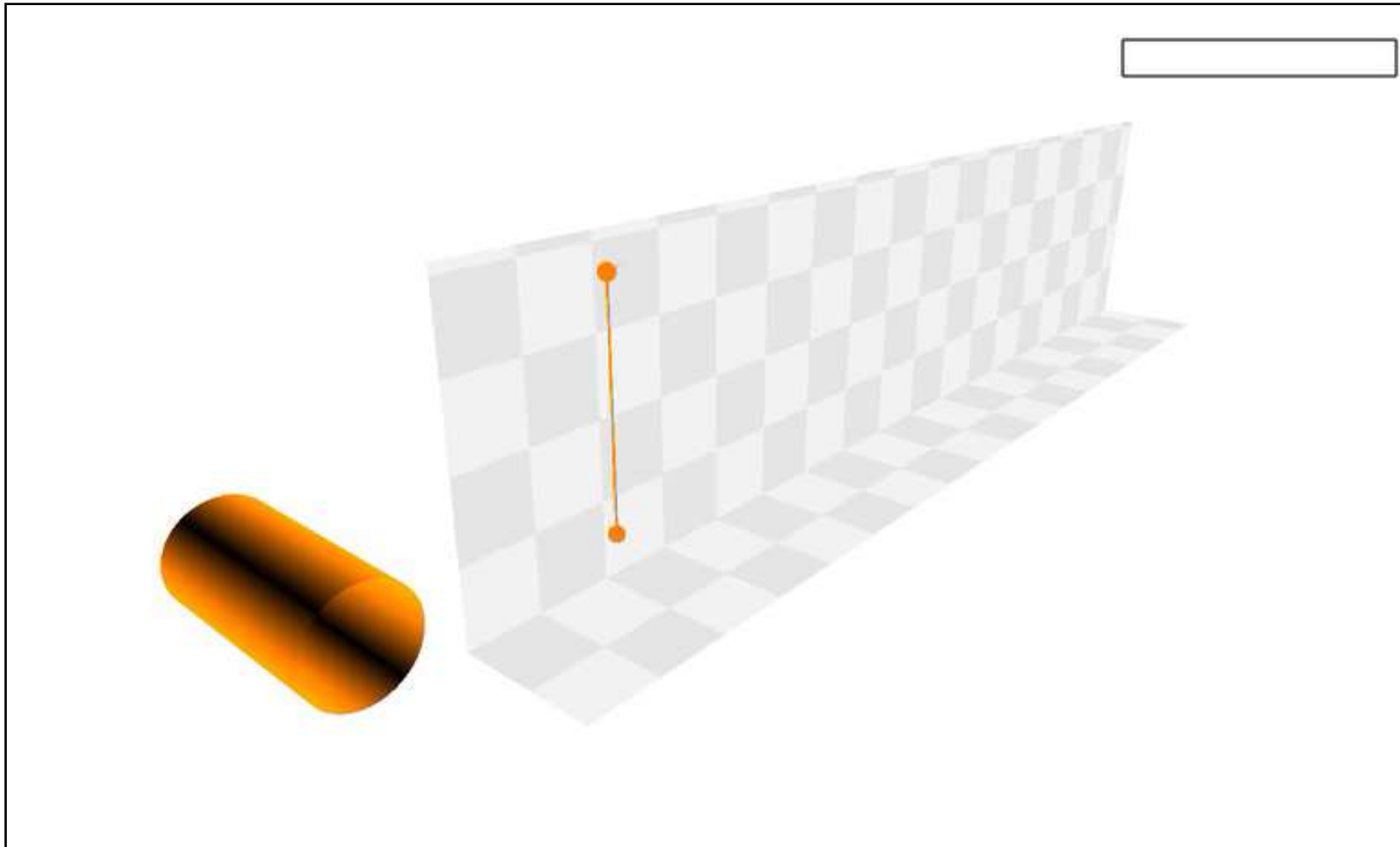


Streak lines



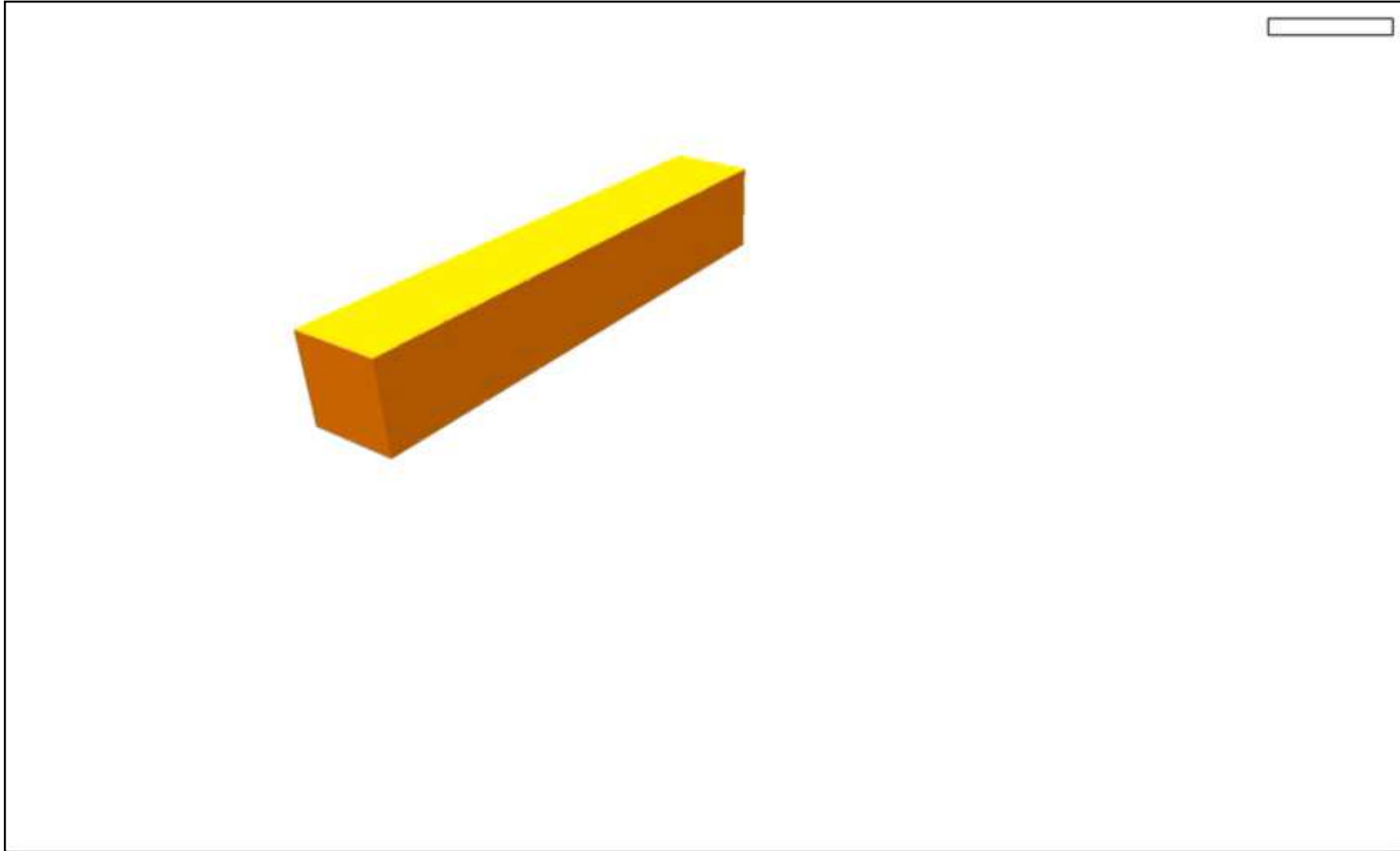
Time lines

Flow behind a Circular Cylinder



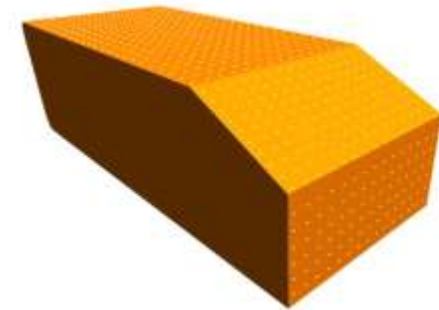
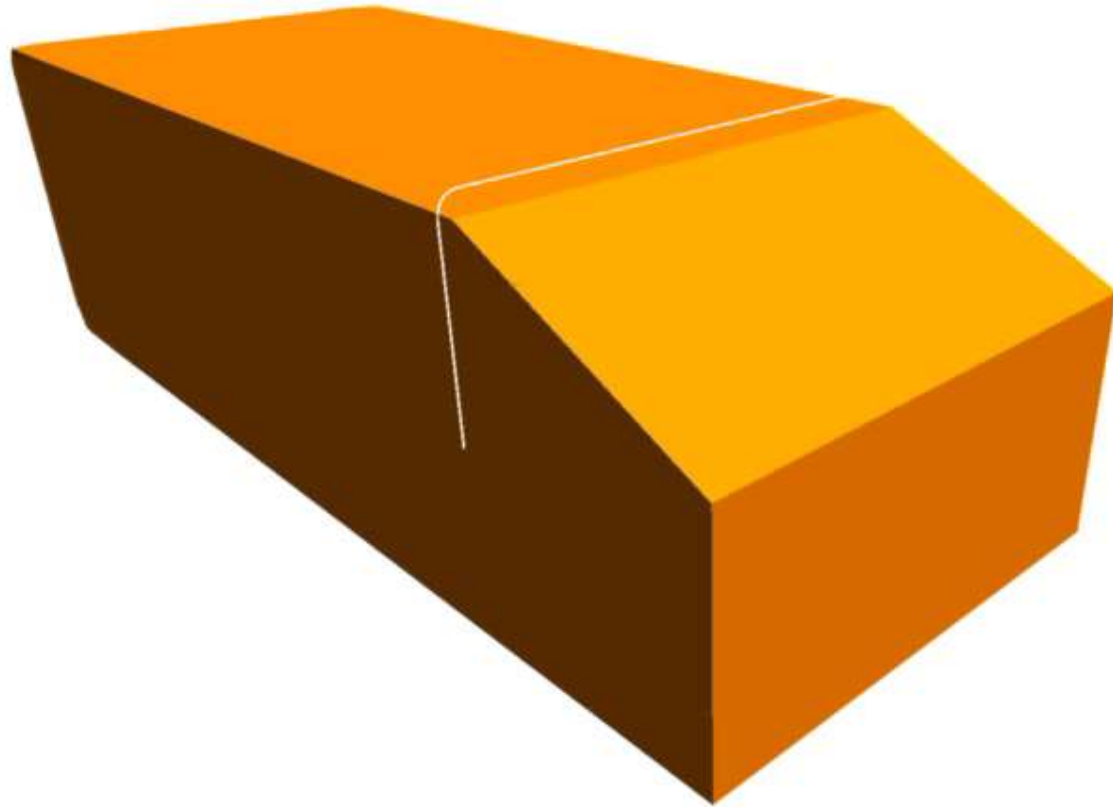
Interactive Session

Flow behind a Square Cylinder



Open source data set. Available from <http://cfd.cineca.it/>
Data courtesy of Camarri, Salvetti (U Pisa), Buffoni (P Torino), Iollo (U Bordeaux).

Ahmed Body



Summary: Integral Surfaces

- Defined in analogy to characteristic curves
 - Essentially: line integration + meshing / adaptive refinement
- In turbulent flow, high-quality streak surfaces and time surfaces require expensive re-meshing throughout the integration process
- Typically require interactive seeding
- Rendering with a smoke metaphor (“smoke surfaces”) can avoid re-meshing and produce interactive results

References: Flow Visualization in General

- Part IV of Charles Hansen, Chris Johnson (Eds): Visualization Handbook. Academic Press, 2004
- Frits Post, Benjamin Vrolijk, Helwig Hauser, Robert Laramée, Helmut Doleisch: Feature extraction and visualisation of flow fields. Eurographics 2002 State of the Art Reports, pp. 69-100.
- Frits Post, Benjamin Vrolijk, Helwig Hauser, Robert Laramée, Helmut Doleisch: The State of the Art in Flow Visualization: Feature Extraction and Tracking. Computer Graphics Forum 22(4):775-792, 2003
- Robert Laramée, Helwig Hauser, Helmut Doleisch, Benjamin Vrolijk, Frits Post, Daniel Weiskopf: The State of the Art in Flow Visualization: Dense and Texture-Based Techniques. Computer Graphics Forum 23(2):203-221, 2004

References: Glyphs and Numerical Integration

- Willem de Leeuw, Jarke van Wijk: A probe for local flow field visualization. Proc. IEEE Visualization 1993, pp. 39-45
- R.M. Kirby, H. Marmanis, D.H. Laidlaw: Visualizing multivalued data from 2D incompressible flows using concepts from painting. Proc. IEEE Visualization 1999, pp. 333-340
- William H. Press, Saul A. Teukolsky, William T. Vetterling, Brian P. Flannery: Numerical Recipes in C++: The Art of Scientific Computing, 3rd Edition, Cambridge University Press, 2007

References: Streamline Placement and Rendering

- G. Turk, D. Banks: Image-guided streamline placement. Proc. SIGGRAPH 1996, pp. 453–460
- B. Jobard, W. Lefer: Creating evenly–spaced streamlines of arbitrary density. Proc. EG Workshop on Visualization in Scientific Computing 1997
- M. Zöckler, D. Stalling, and H. Hege: Interactive visualization of 3D-vector fields using illuminated streamlines. IEEE Visualization 1996, pp. 107–113
- Carsten Stoll, Stefan Gumhold, Hans-Peter Seidel: Visualization with stylized line primitives. Proc. IEEE Visualization 2005
- W. von Funck, T. Weinkauff, H. Theisel, H.-P. Seidel: Smoke Surfaces: An Interactive Flow Visualization Technique Inspired by Real-World Flow Experiments. IEEE Transactions on Visualization and Computer Graphics 14(6), 2008

Exam-like Questions

1. Given a vector field $\mathbf{v}$, what information do we gain from computing the divergence $\nabla \cdot \mathbf{v}$ and the curl $\nabla \times \mathbf{v}$?
2. Why is it computationally more expensive to compute streak lines in unsteady vector fields compared to path lines?
3. State an advantage of embedded Runge-Kutta for streamline integration compared to integration with the midpoint method.
4. Why are popular 2D streamline placement strategies rarely effective in three dimensions?